

From Theory to Practice in Model Merging

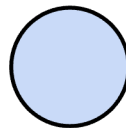
任星宇

2026.8.20

吴泰霖团队组会报告

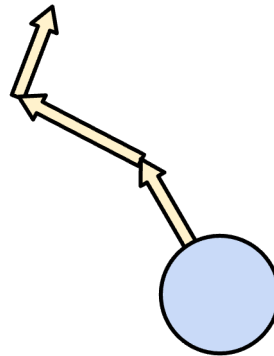
课题组主页: <https://ai4s.lab.westlake.edu.cn/>

What is Model Merging?



Pre-trained model

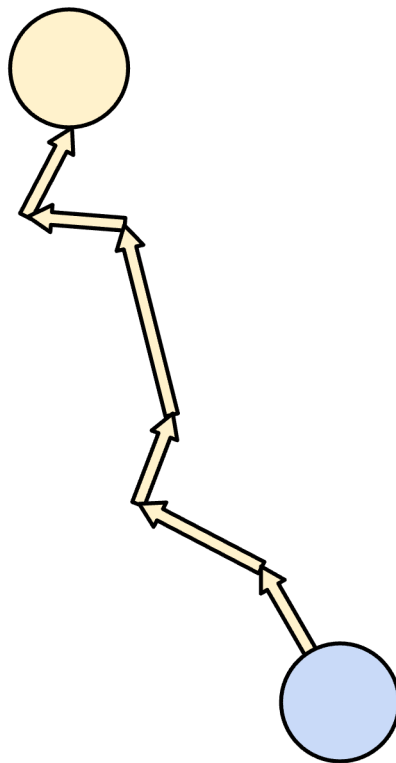
What is Model Merging?



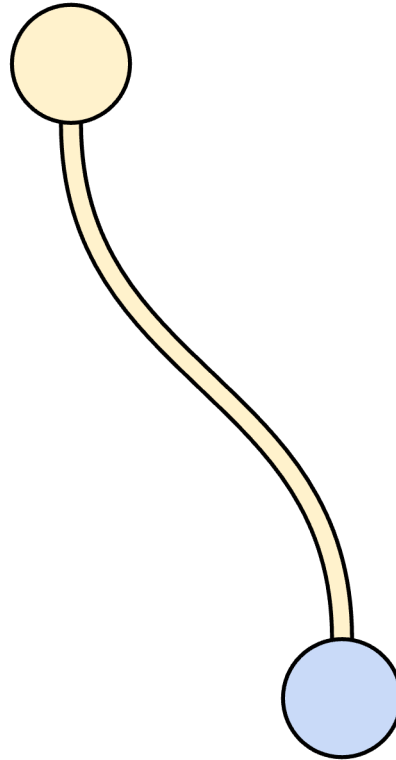
What is Model Merging?



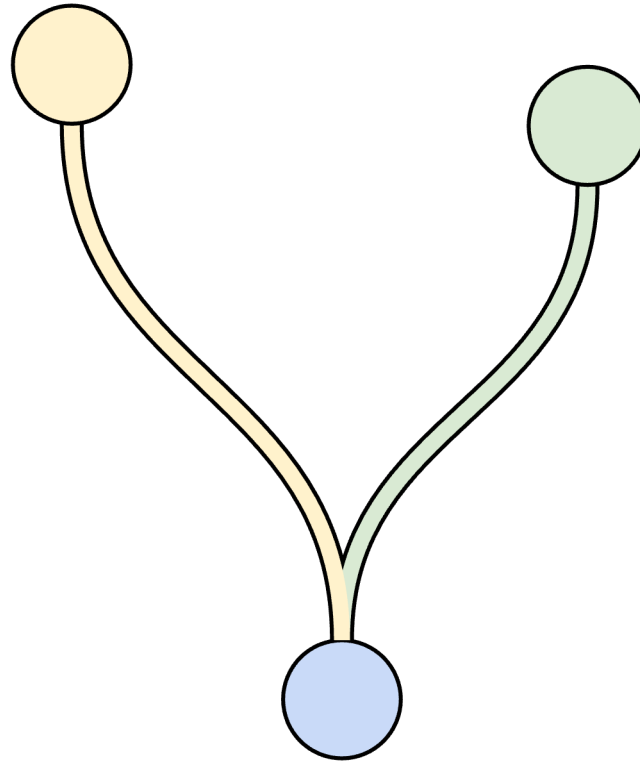
Fine-tuned model



What is Model Merging?



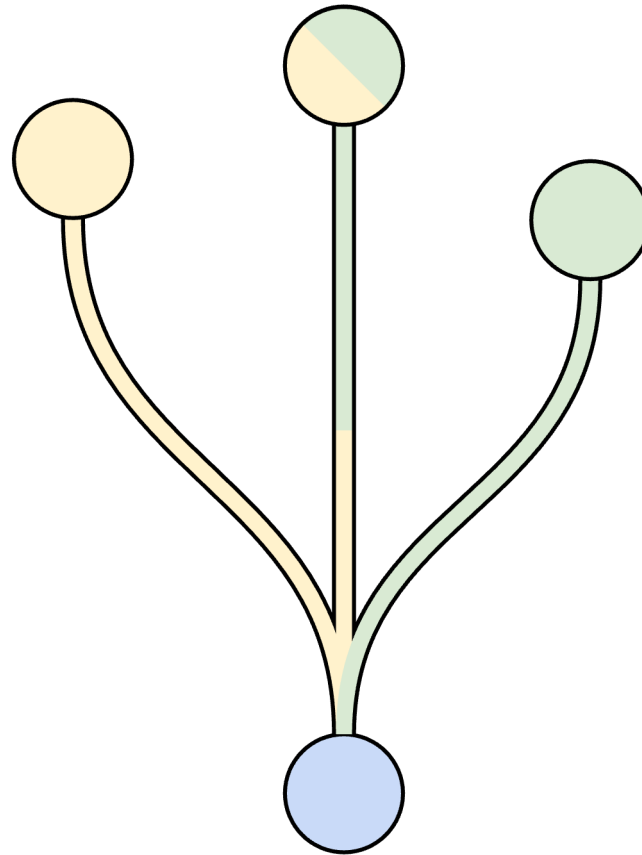
What is Model Merging?



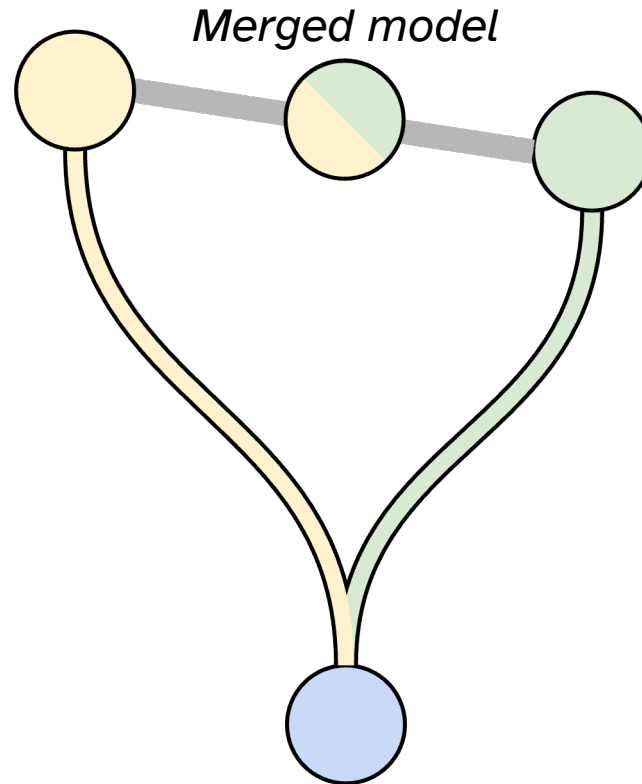
What is Model Merging?



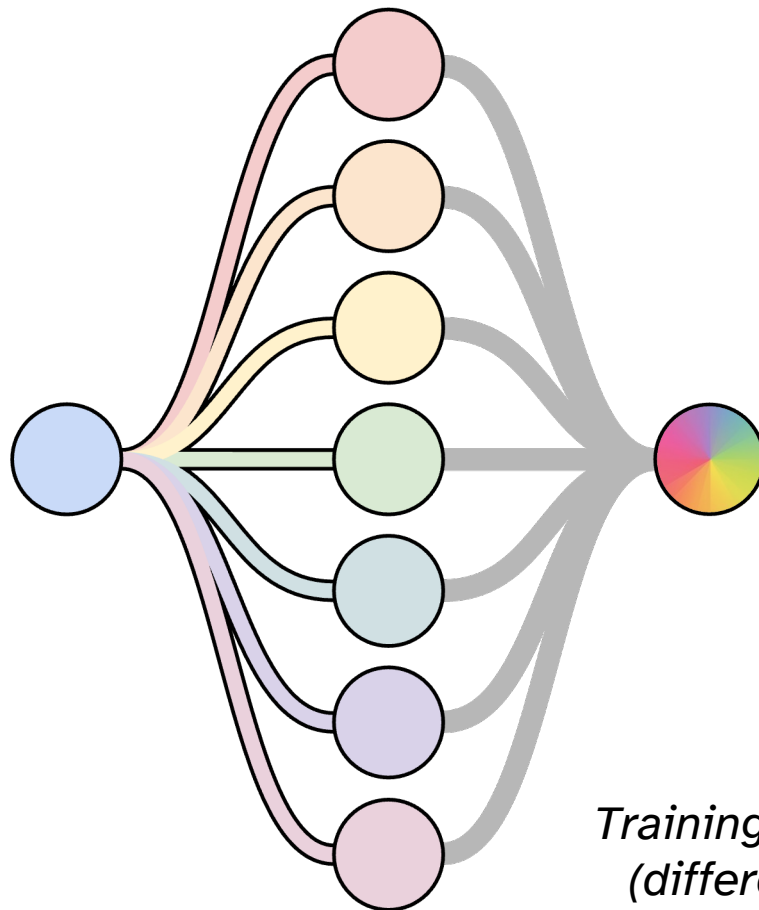
Multitask model



What is Model Merging?

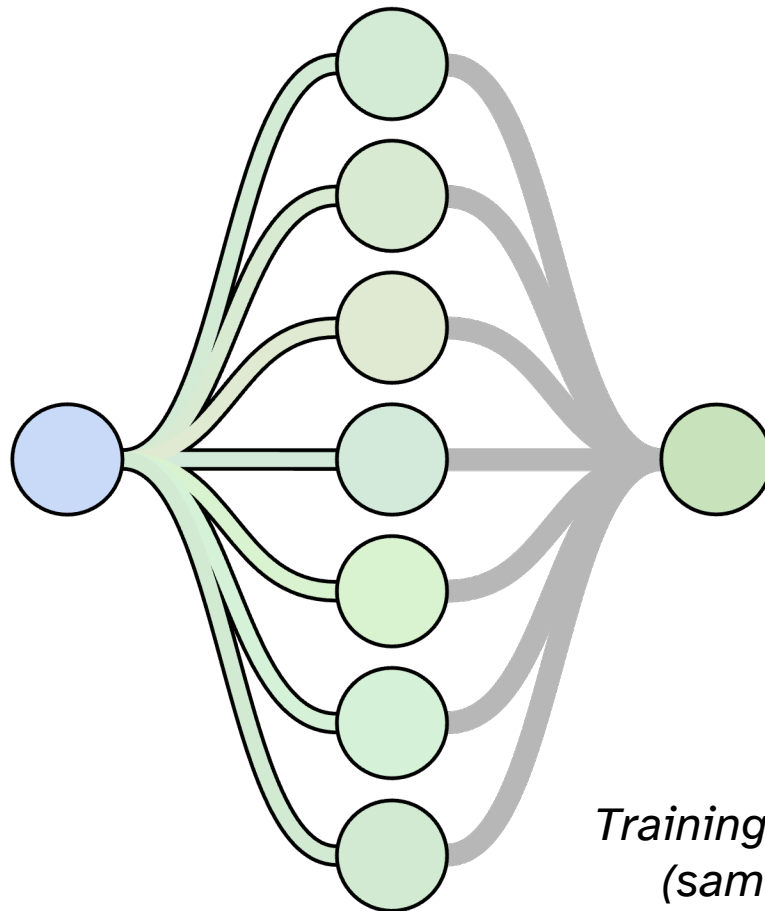


What is Model Merging?



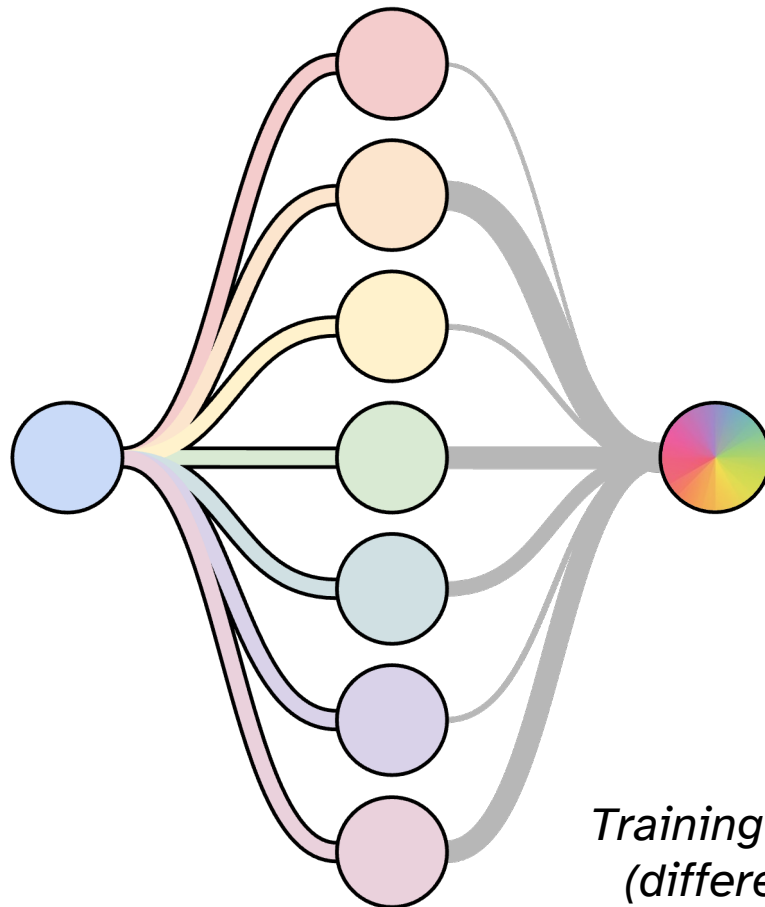
*Training Trajectory
(different data)*

What is Model Merging?



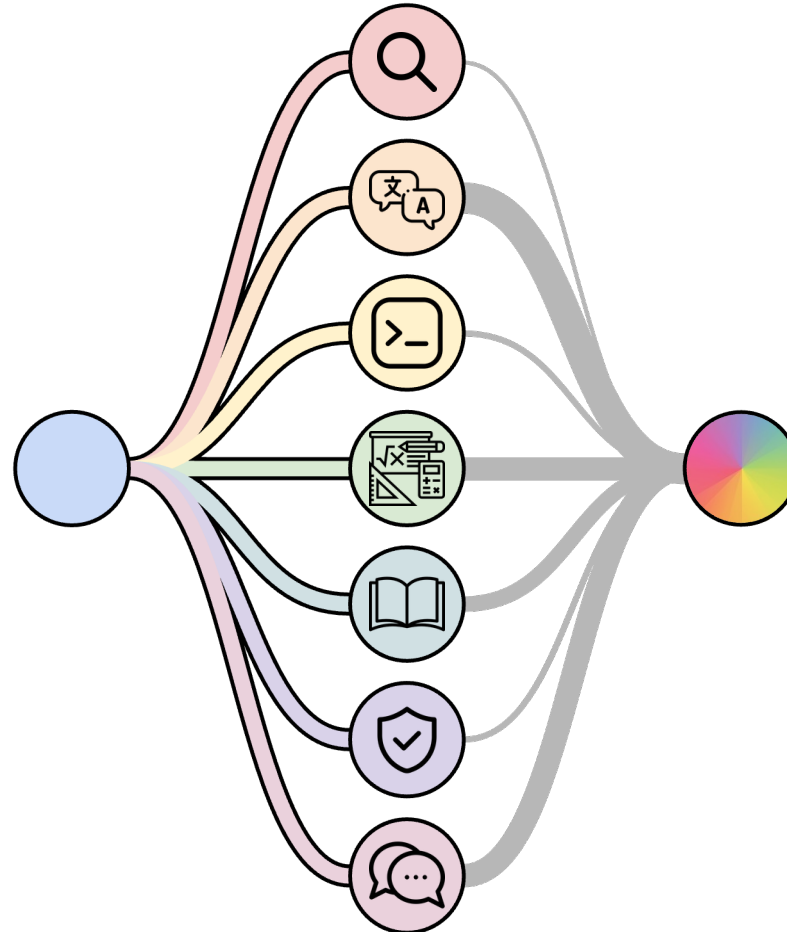
*Training Trajectory
(same data)*

What is Model Merging?

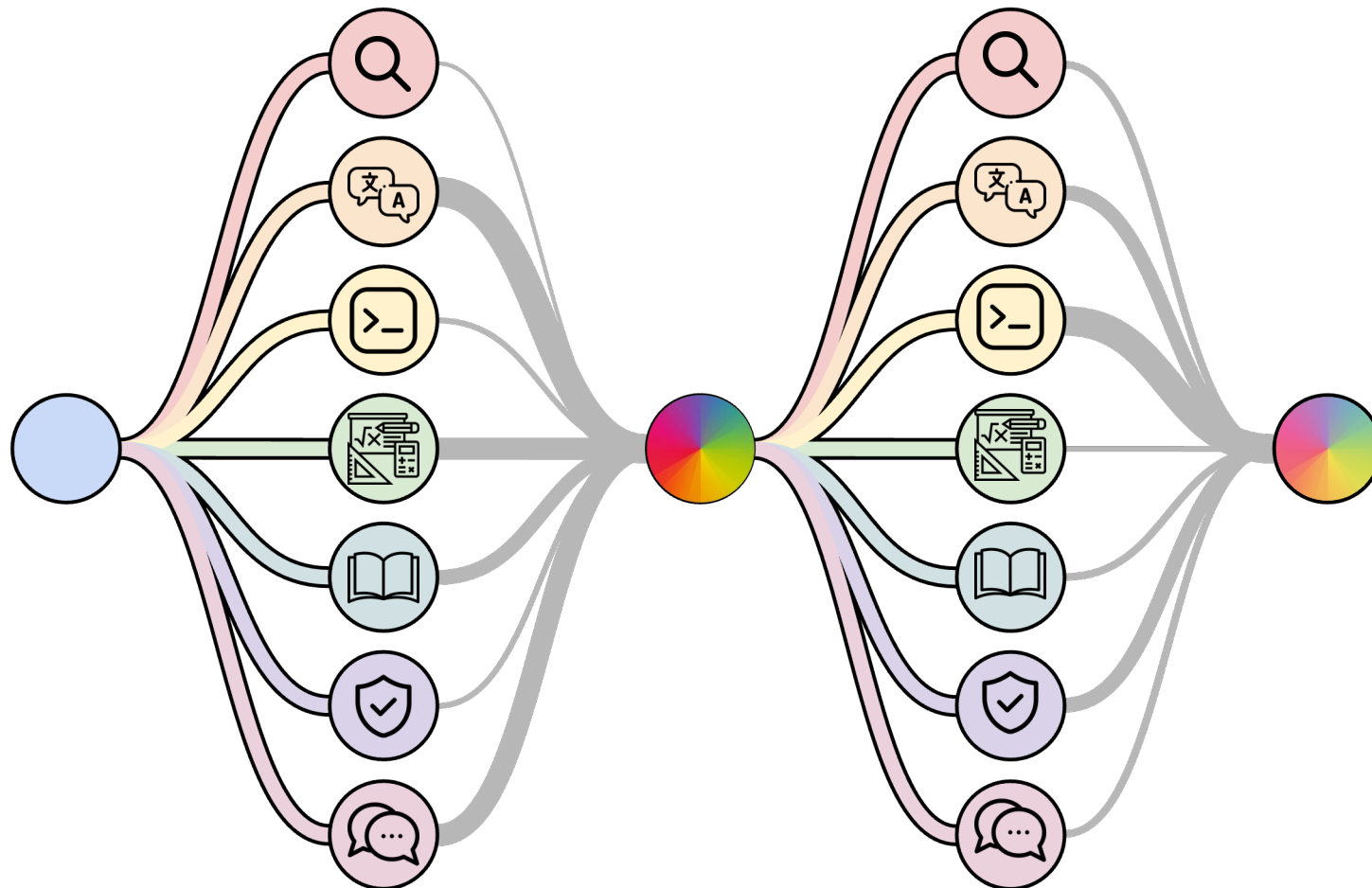


*Training Trajectory
(different data)*

What is Model Merging?



What is Model Merging?





Foundations

History and foundations of parameter averaging
Linear mode connectivity and beyond



Methods

Overview of merging methods
Evaluating model merging



Applications

Fruitful applications of merging in llms
Applications in ai4s



Foundations

Core merging approach: **Parameter Averaging**
(linear interpolation in the weight space)

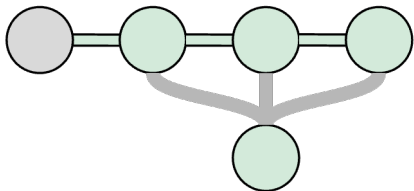
$$\theta_{\text{merged}} = \sum_{i=1}^M \lambda_i \theta_i$$

often, but not always, a convex combination

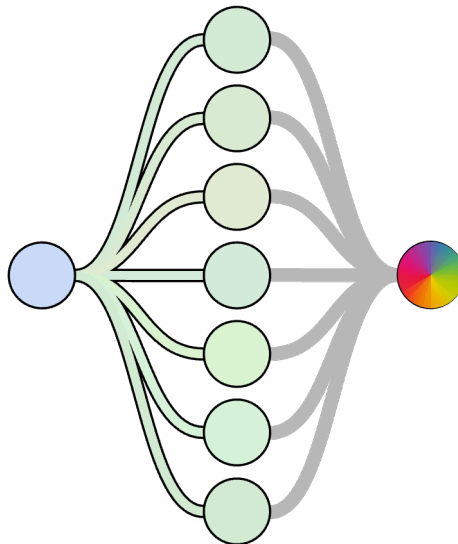
$$\sum_{i=1}^M \lambda_i = 1$$

Our roadmap

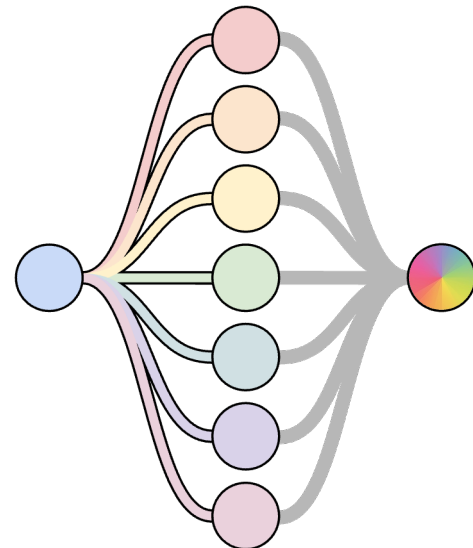
*single training
trajectory*



*multiple training
trajectory (same data)*



*multiple training
trajectory (different data)*



Polyak-Ruppert averaging

uniform average of SGD iterates accelerates stochastic optimization

SGD update

$$\theta_{t+1} = \theta_t - \eta_t \nabla f(\theta_t)$$

Running Average

$$\bar{\theta}_{t+1} = \bar{\theta}_t + \frac{1}{t+1} (\theta_{t+1} - \bar{\theta}_t)$$

From "Efficient estimations from a slowly convergent Robbins-Monro process" by Ruppert  1988

 From "Acceleration of stochastic approximation by averaging" by Polyak & Juditsky  1992



“Tail averaging” is a common “folk-law” for improving performance in deep learning

found to be useful to stabilize the training. Model evaluations are performed using a running **average** of the parameters computed over time.

Szegedy et al 
2015 

produce long term coherent waveforms. The learning rate was held constant at 2×10^{-4} , and Polyak **averaging** (Polyak & Juditsky, 1992) was applied over the parameters. The

van den Oord et al 
2017 

Motivated by this observation, we investigate **averaged** SGD (ASGD) to further improve the training process.

Merity et al 
2017 

For the base models, we used a single model obtained by **averaging** the last 5 checkpoints, which were written at 10-minute intervals. For the big models, we **averaged** the last 20 checkpoints. We

Vaswani et al 
2017 

rates and take two **D** steps per **G** step. For evaluation, we employ moving **averages** of **G**'s weights following Karras et al. (2018); Mescheder et al. (2018); Yazıcı et al. (2018), with a decay of 0.9999.

Brock et al  2018


We examine two different techniques for parameter averaging in GAN training. Moving **Average** (MA) computes the time-**average** of parameters, whereas Exponential Moving **Average** (EMA) computes an exponentially discounted sum.

Yazıcı et al  2018


***Uniform Average** applied only to the last T checkpoints*

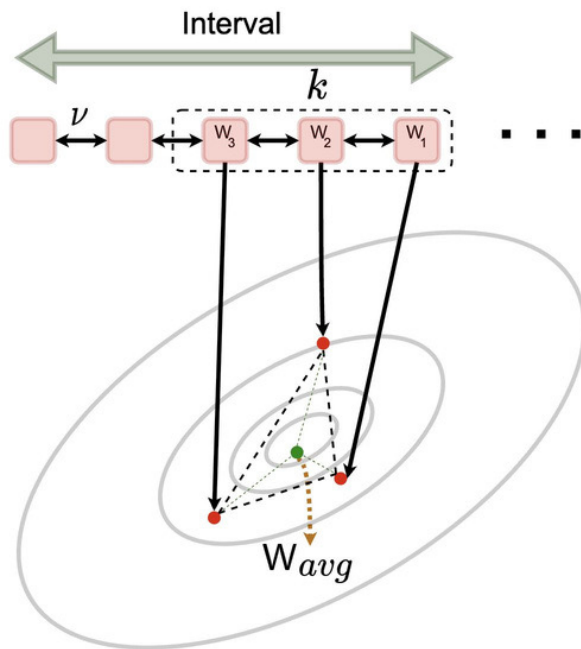
$$\bar{\theta}_T = \frac{1}{T} \sum_{t=1}^T \theta_t$$

***Exponential Moving Average**, more importance to the latest checkpoints*

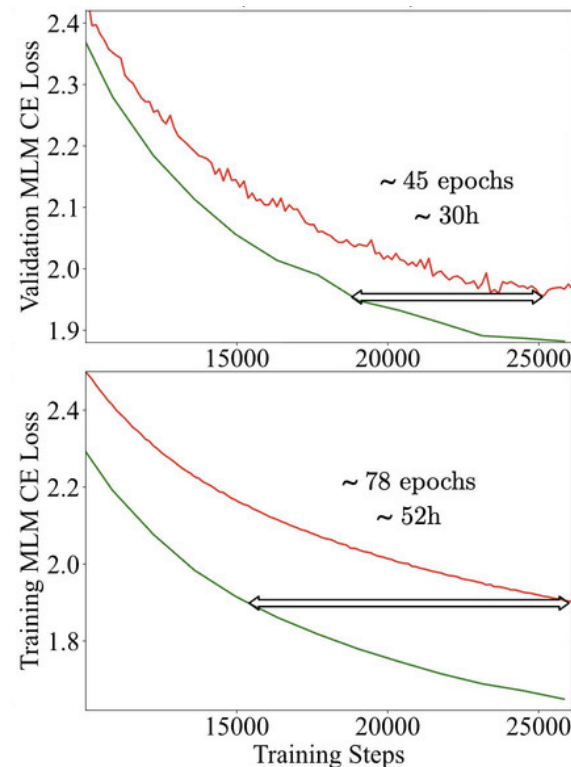
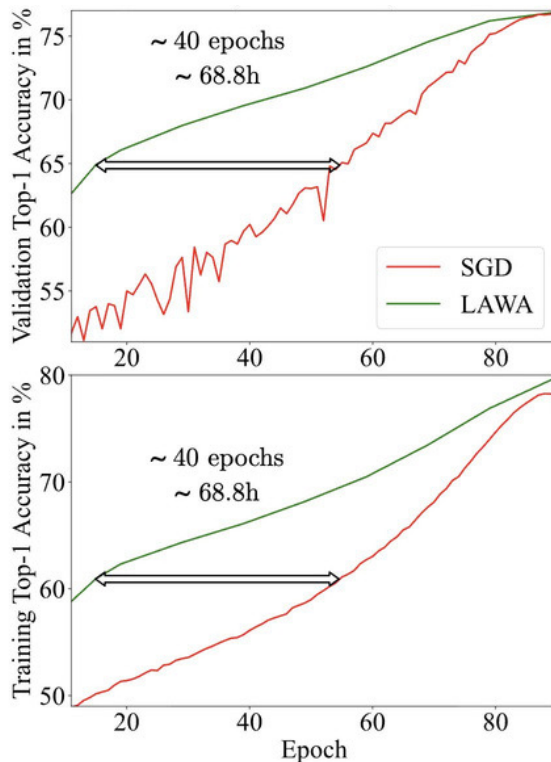
$$\theta_t^{\text{EMA}} = \alpha \theta_{t-1}^{\text{EMA}} + (1 - \alpha) \theta_t$$

Moving Window Average: Latest Weight Averaging (LAWA)

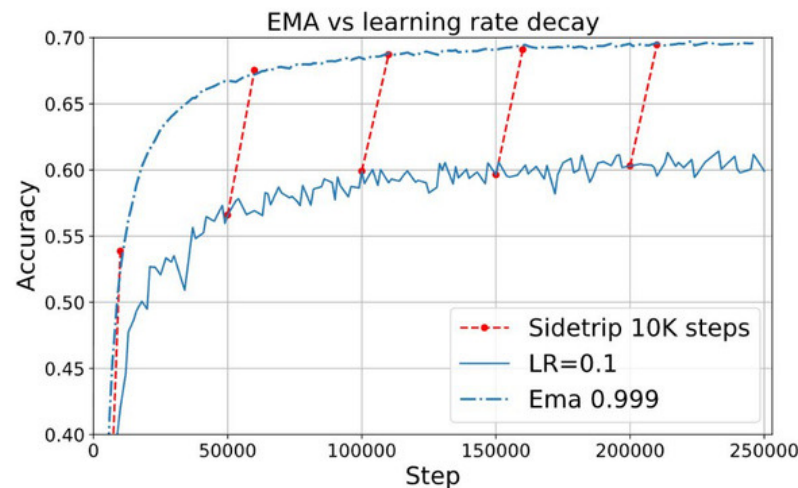
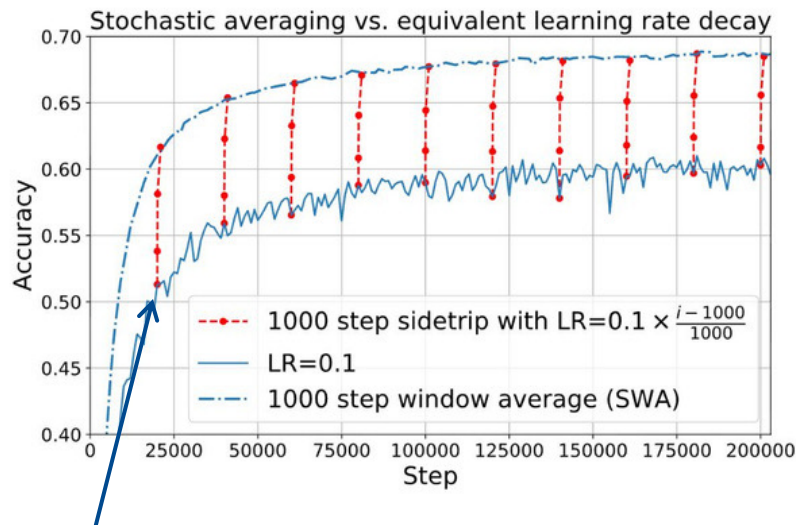
average K periodically sampled checkpoints stored in a FIFO queue



Moving Window Average: Latest Weight Averaging (LAWA) ResNet (ImageNet) WikiText103 (RoBERTa)



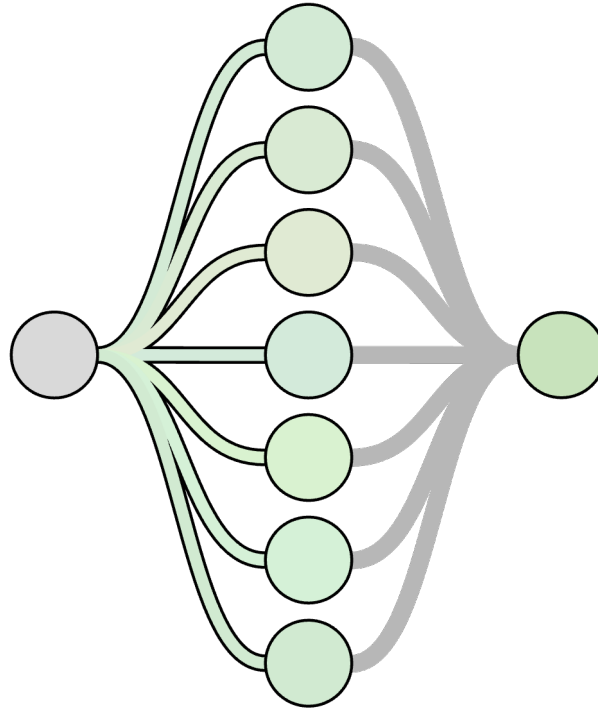
Parameter Averaging can be equivalent to LR decay schedule



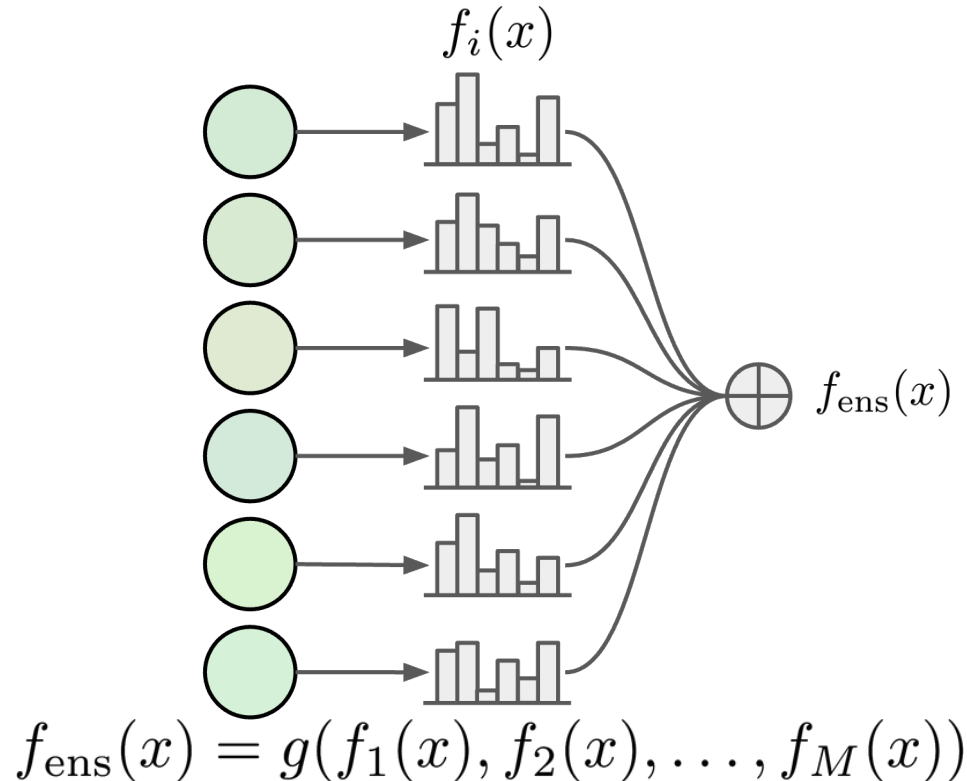
Extra Training
From This ckpt

Averaging method	Equivalent learning rate for last k steps
Stochastic Weight Averaging (Izmailov et al., 2018), window size k , cycle $c = 1$	$\lambda(i) \frac{k+1-i}{k}$
Two point average k step apart	$\lambda(i)/2$
Exponential Moving Average (decay δ)	$\lambda(i)(1 - (1 - \delta)^{k-i})$, where $(k \gg 1/\delta)$

*Multiple training trajectory
(same data)*



Ensembling: *Combining predictions from independent models to improve model robustness and performance*



Ensembling problems

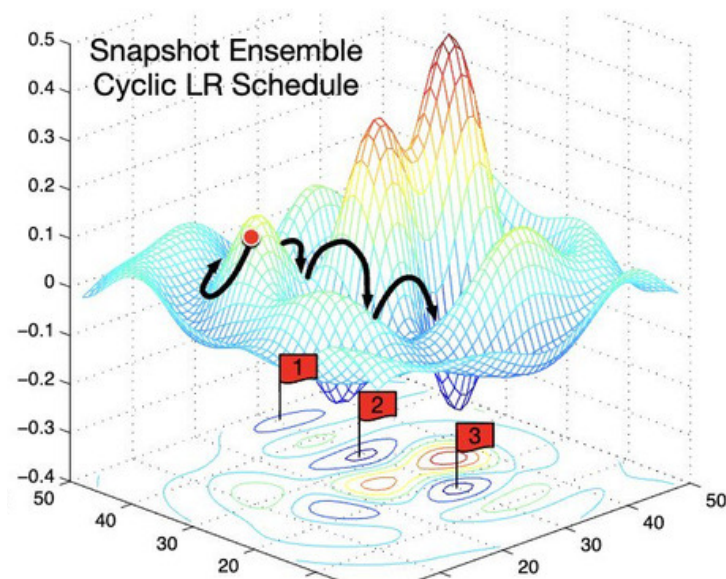
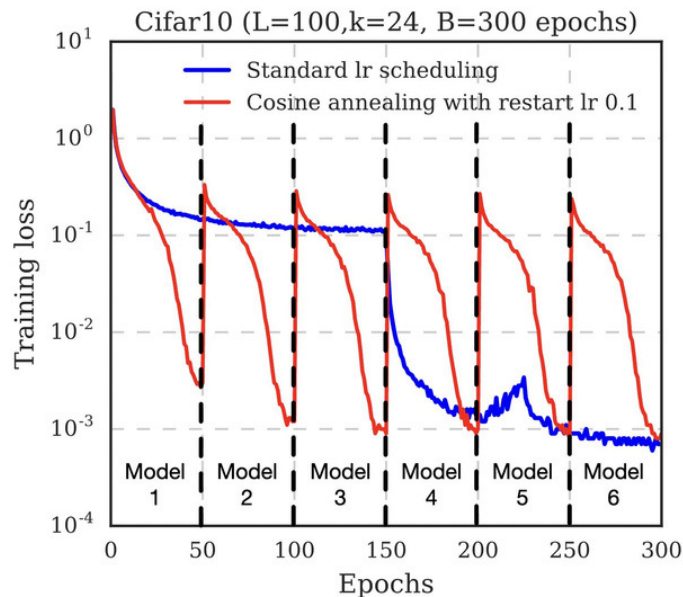
Training cost:
train N models

expensive for Neural Networks
need to balance diversity

Inference cost
produce N predictions

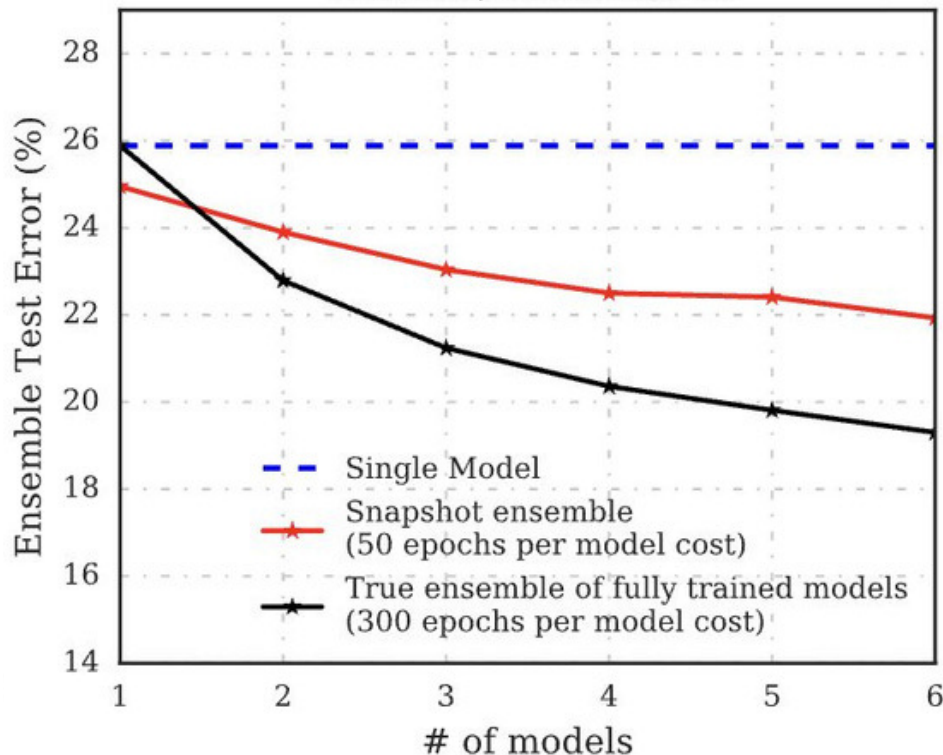
time $O(N)$
memory $O(N)$

Snapshot ensembles train 1, get M for free, by exploring different solutions



Snapshot ensembles

Cifar100, DenseNet-40



方法	合并位置	最终保存几个模型	推理成本
LAWA/SWA	参数空间平均	1个	单模型
Snapshot Ensemble	输出/预测空间平均	M个	约为单模型的 M 倍
Model merging	参数空间合并	1个	单模型
独立深度集成	输出空间平均	M个	约为单模型的 M 倍

推理成本不变

*But **when** does Weight Averaging work?*

Mode Connectivity

minima are connected by paths with constantly low loss

设两个训练完成的模型参数为:

$$\theta_A, \quad \theta_B.$$

Mode Connectivity关心能否找到一条参数路径:

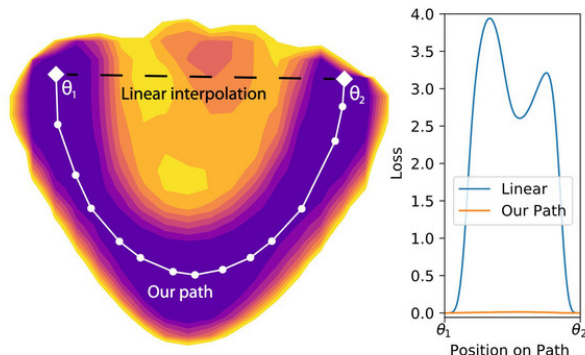
$$\gamma(t), \quad t \in [0, 1],$$

满足:

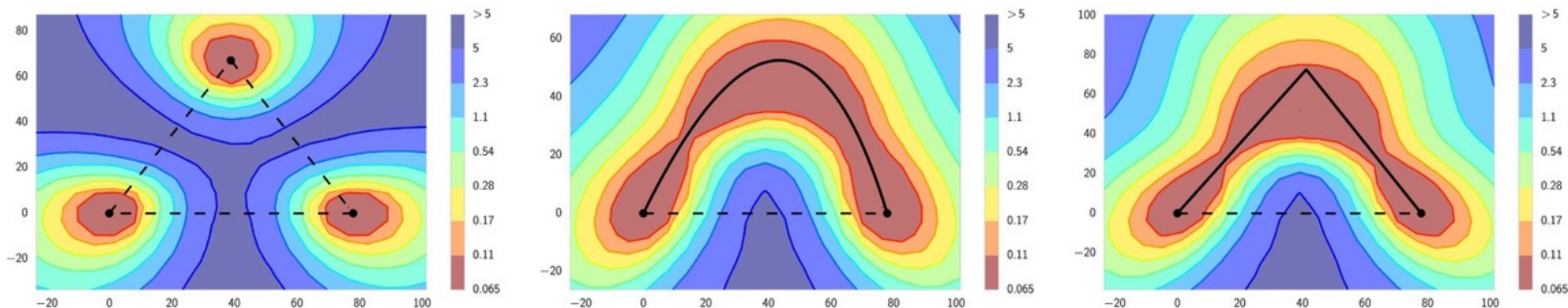
$$\gamma(0) = \theta_A, \quad \gamma(1) = \theta_B,$$

并且路径上的损失始终较低:

$$L(\gamma(t)) \approx \text{low}, \quad \forall t \in [0, 1].$$

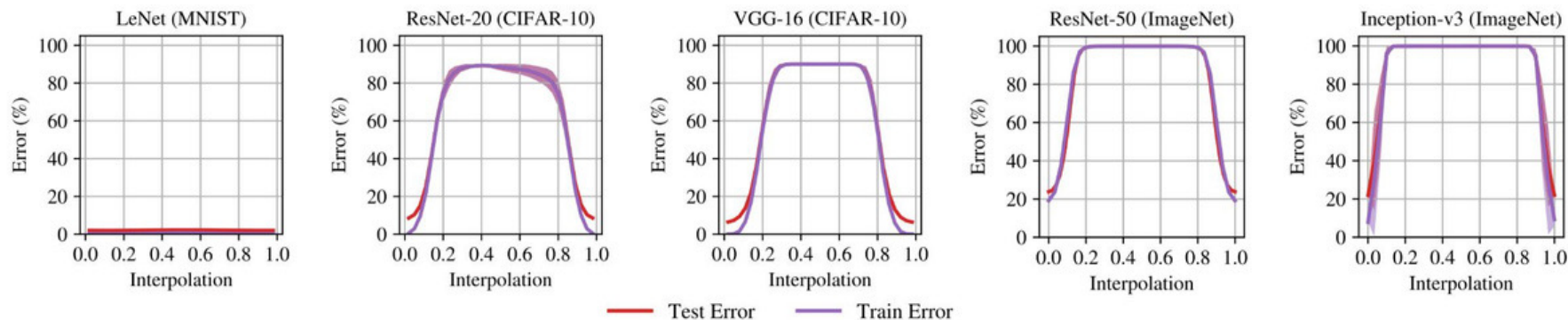


From "Essentially No Barriers in Neural Network Energy Landscape" by Draxler et al.  2018 



From "Loss Surfaces, Mode Connectivity, and Fast Ensembling of DNNs" by Garipov et al.  2018 

*Independently trained models are **not linearly connected***



*Neural Networks have **permutation symmetries***

The same function can be represented by different parameterizations

$$f(x) = \mathbf{W}_2 \sigma(\mathbf{W}_1 x + \mathbf{b}_1) + \mathbf{b}_2$$

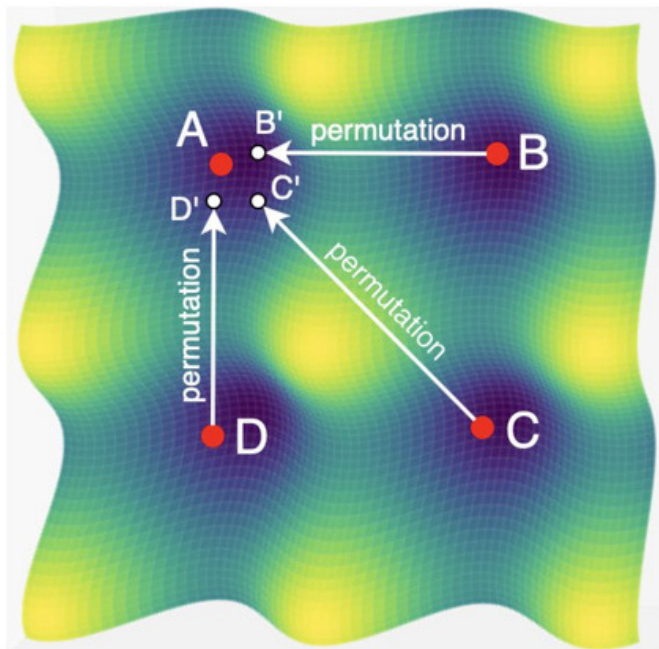
$$\mathbf{W}'_1 = \mathbf{P} \mathbf{W}_1 \quad \text{permute rows}$$

$$\mathbf{b}'_1 = \mathbf{P} \mathbf{b}_1, \quad \text{permute rows} \quad \textcolor{red}{P^T P = I}$$

$$\mathbf{W}'_2 = \mathbf{W}_2 \mathbf{P}^T \quad \text{permute columns}$$

$$f(x) = f'(x)$$

Conjecture: *SGD solutions could be linearly connected modulo permutation symmetries*



SGD训练得到的不同解，在消除置换对称性后，可能具有线性模态连通性。

Finding explicit permutations makes model linearly connected

1. Activation Matching: 激活匹配

给两个模型输入同一批样本，比较隐藏神经元的激活：

$$h_i^A(x), \quad h_j^B(x).$$

计算不同神经元之间的相似度，例如相关系数，然后寻找最佳——匹配：

$$i \leftrightarrow j.$$

该问题可转化为**线性分配问题**，使用匈牙利算法求解。

直观上就是：

模型A中负责相似特征的神经元，应当与模型B中具有相似激活模式的神经元对齐。

它需要一批输入数据来计算激活。

2. Weight Matching: 权重匹配

不比较激活，而是直接比较权重，希望找到置换 P ，使两个模型的参数尽可能接近：

$$\min_P \|\theta_A - P(\theta_B)\|^2.$$

由于一层的置换会影响前后相邻层，因此总体上是耦合的二次分配问题。Git Re-Basin采用逐层或坐标下降方式近似求解。

它的重要特点是：

通常只需要模型参数，不一定需要训练数据。

3. Learning Permutations: 直接学习置换

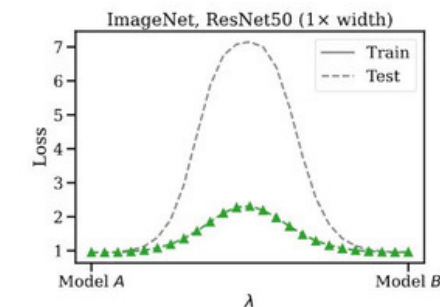
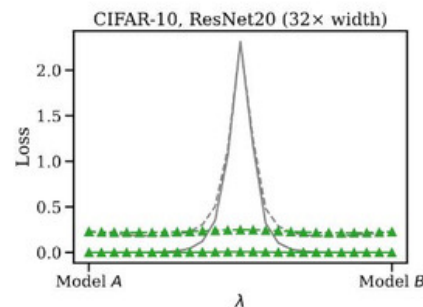
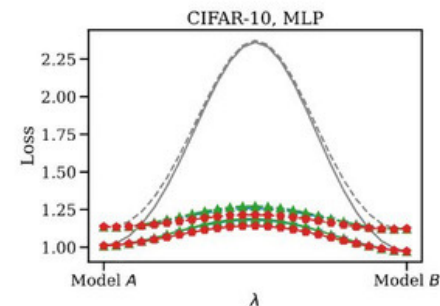
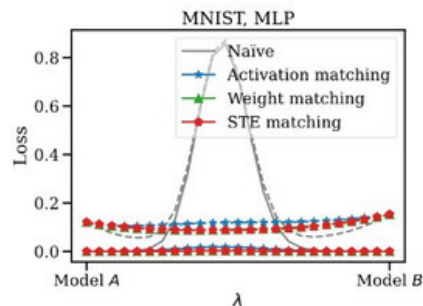
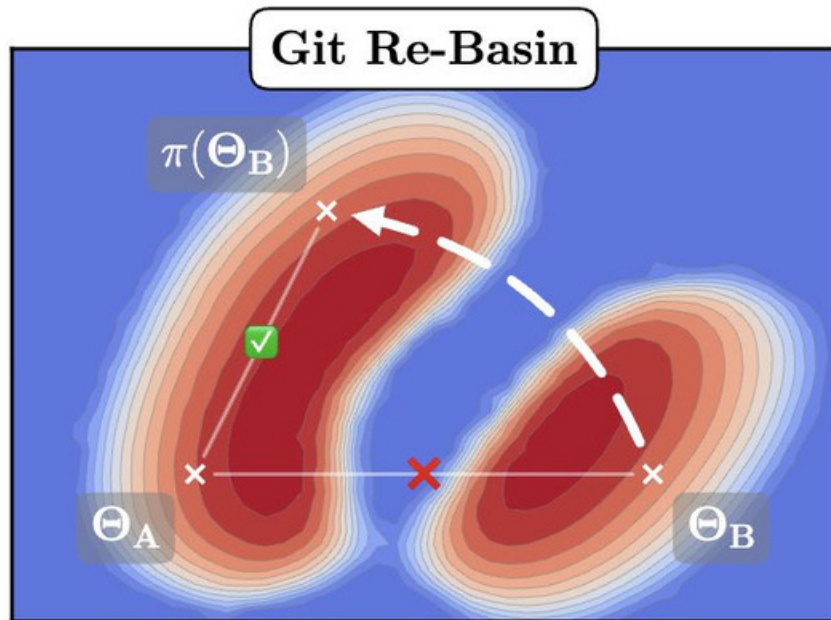
直接把插值路径上的损失障碍作为优化目标：

$$\min_{\pi} \max_{\lambda \in [0,1]} L((1-\lambda)\theta_A + \lambda\pi(\theta_B)).$$

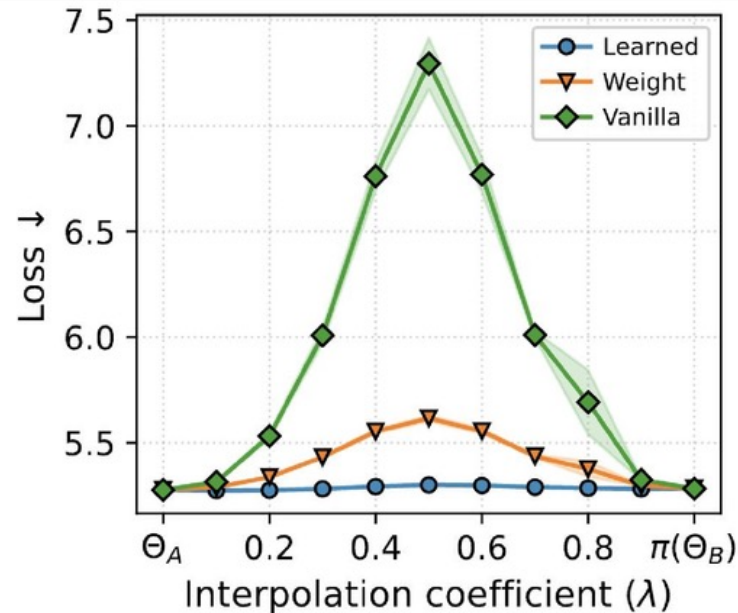
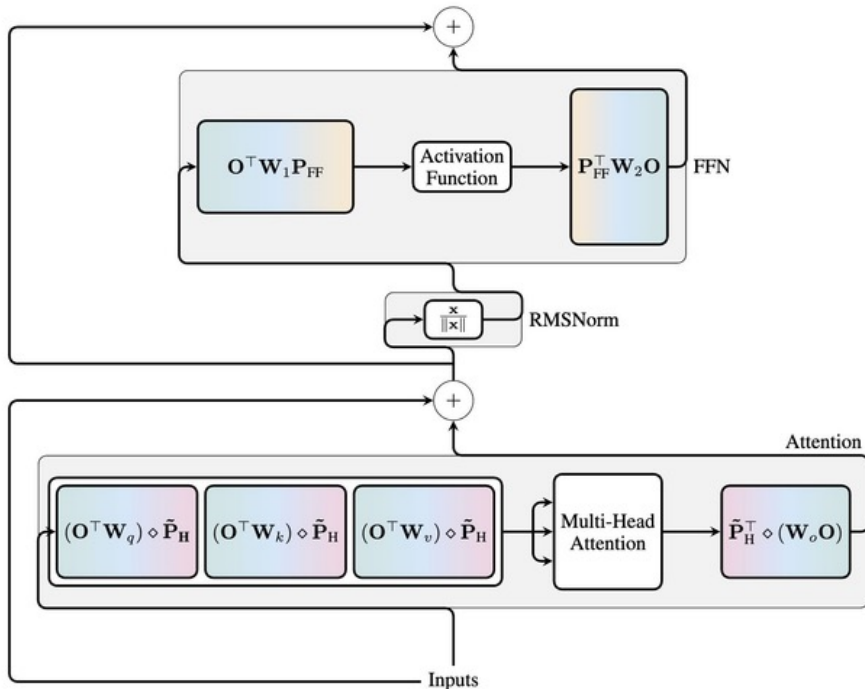
也就是直接寻找一个置换，使两个模型之间的线性路径损失尽可能低。

在Git Re-Basin中，这类方法通常通过可微松弛或直通估计等方式优化，因此第54页图例中会出现 **STE matching**。

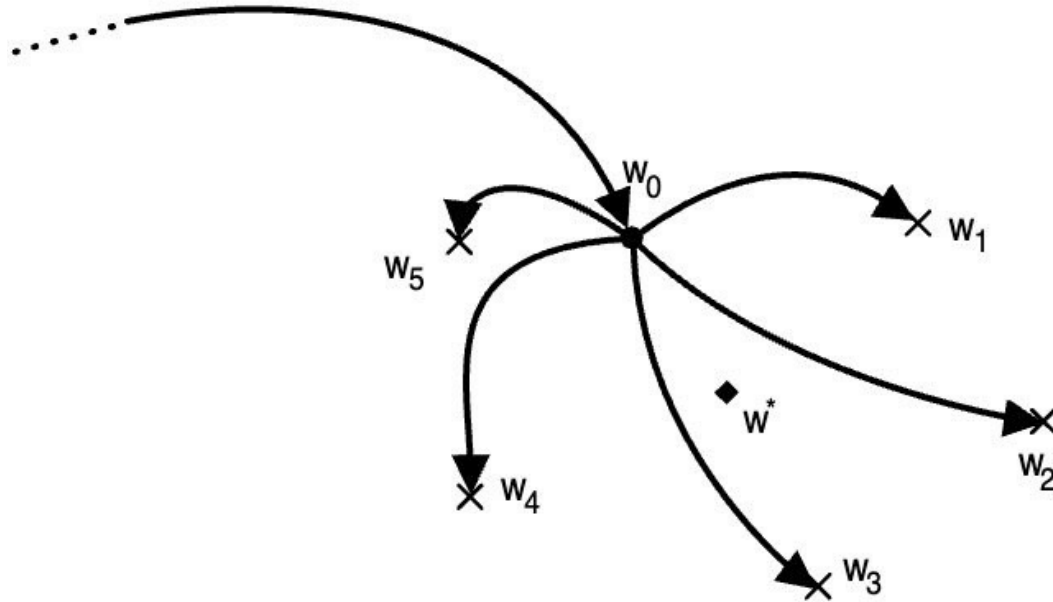
Finding explicit permutations makes model linearly connected



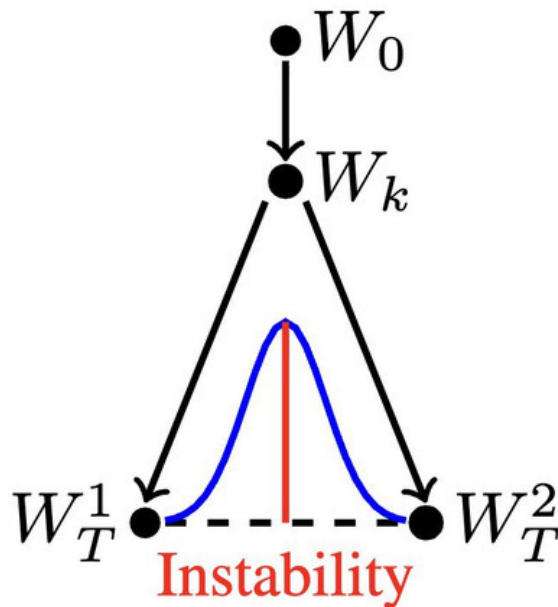
In the Transformer we have even richer continuous symmetries



Theoretical Foundation: *Sharing part of the same trajectory enables LMC*



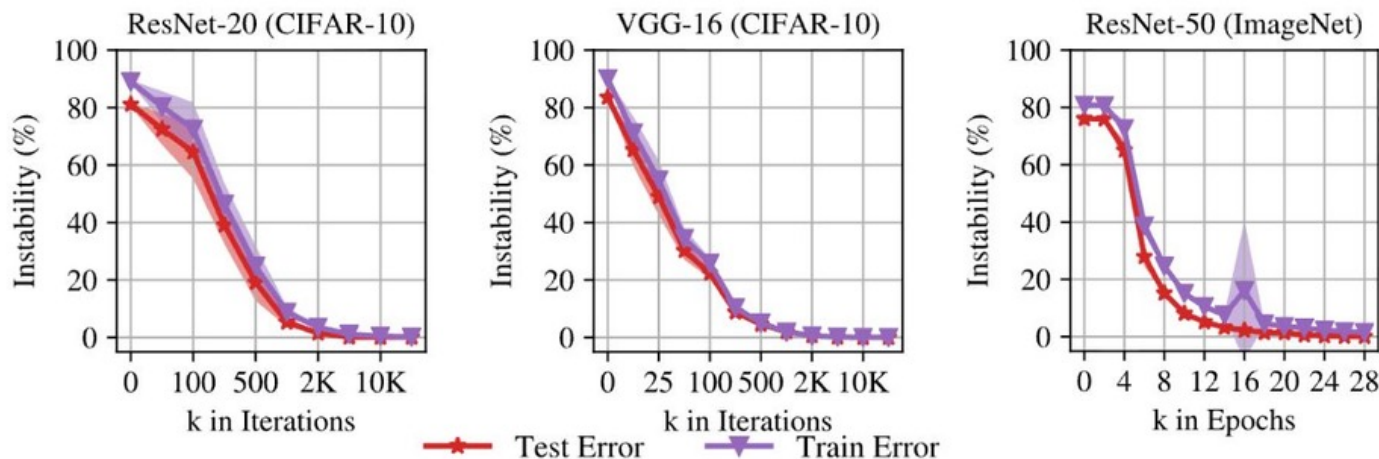
Spawning Experiment: after k iterations, add different SGD noise



$$\mathcal{E}_{\text{sup}}(W_1, W_2) - \text{mean}(\mathcal{E}(W_1), \mathcal{E}(W_2))$$

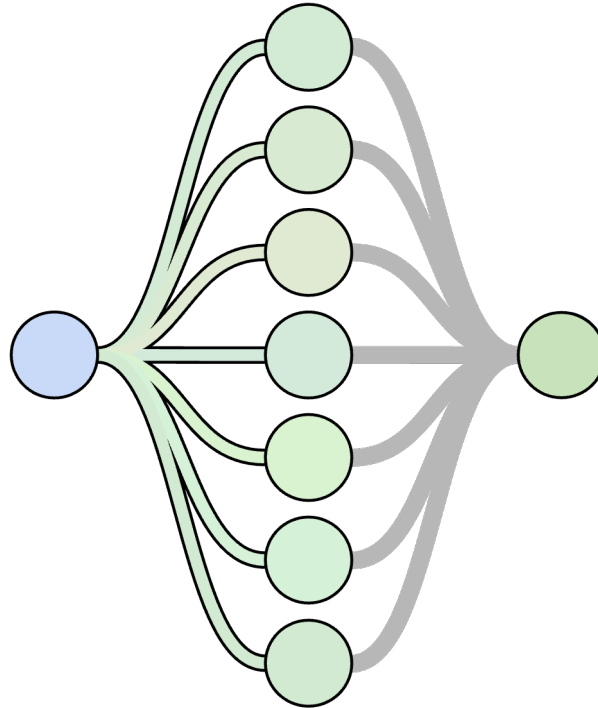
***Linear** Mode Connectivity (LMC) emerges through training*

*after shared training, **independent** optimization trajectories
settle into the same **linearly connected basin***

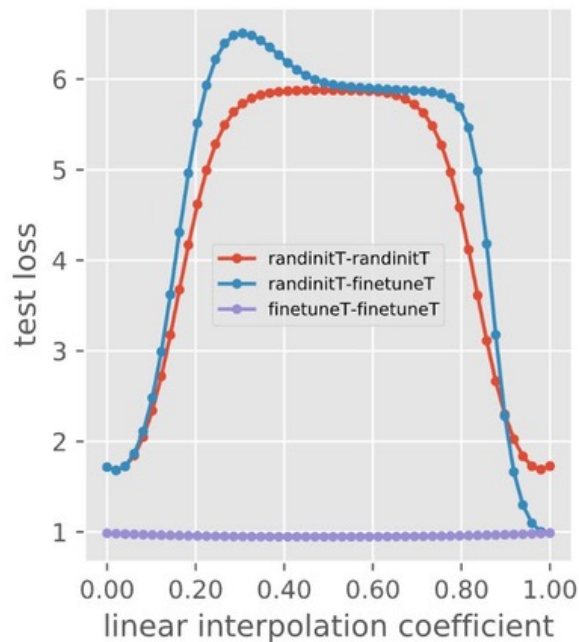


From "Linear Mode Connectivity and the Lottery Ticket Hypothesis" by Frankle et al.  2019

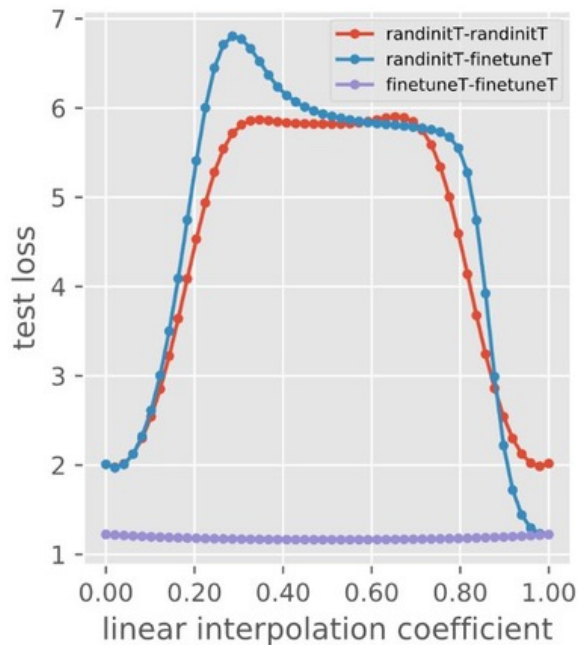
*Multiple training trajectories
(**fine-tuning** with new data)*



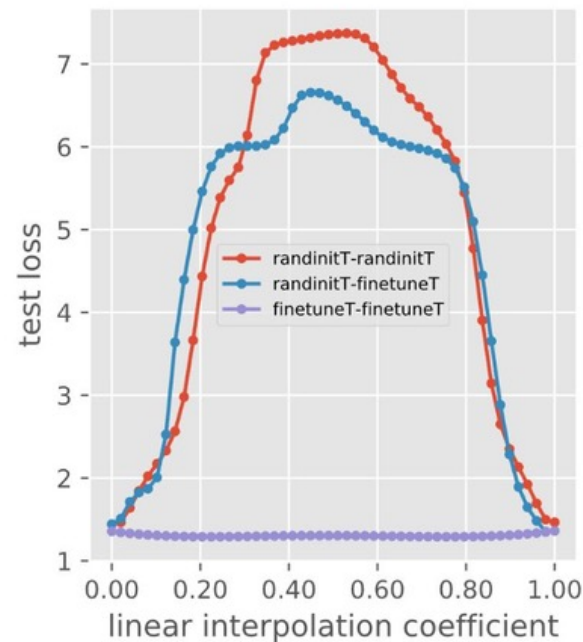
Pretraining (ImageNet) consistently
removes barriers during Fine-tuning (DomainNet)



real



clipart



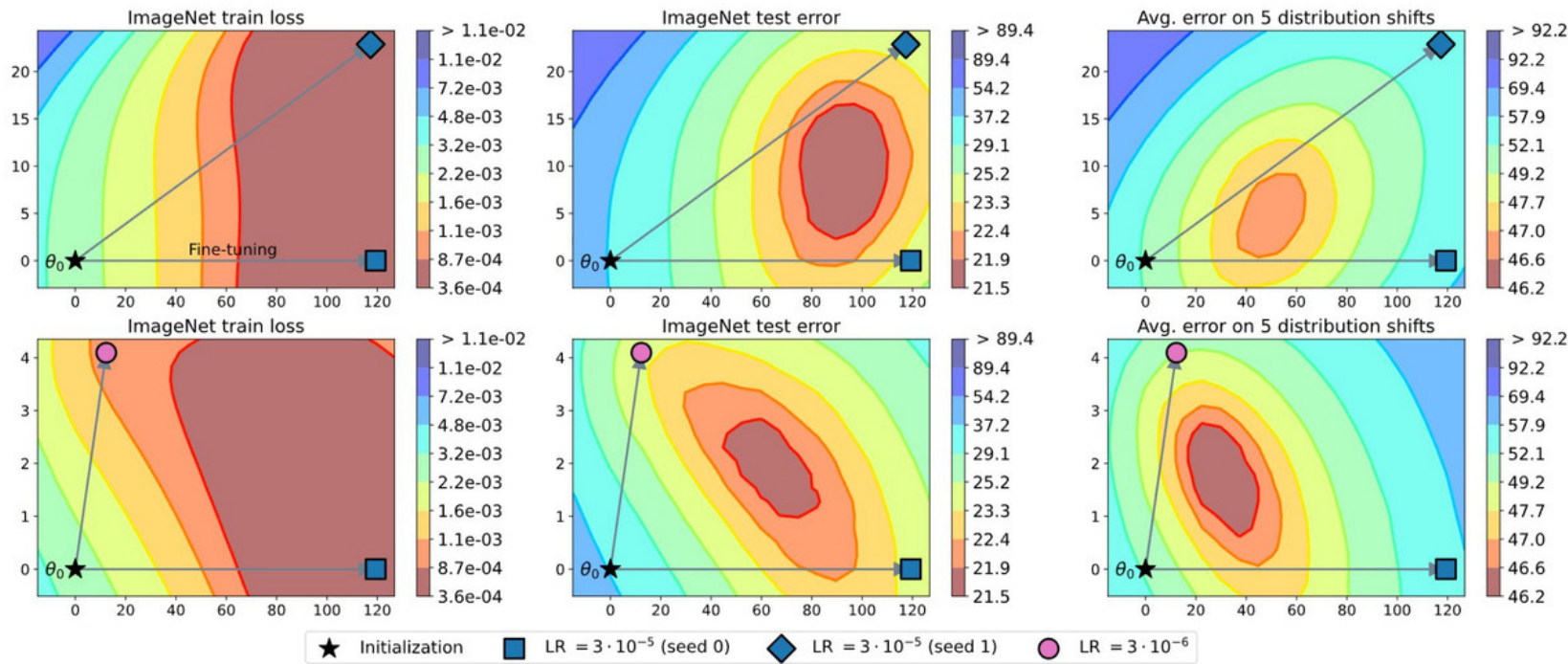
quickdraw

From "What is being transferred in transfer learning?" by Neyshabur et al.  2020

Model Soups: The solution with the **highest accuracy** is often not a fine-tuned model but **lies between fine-tuned models**

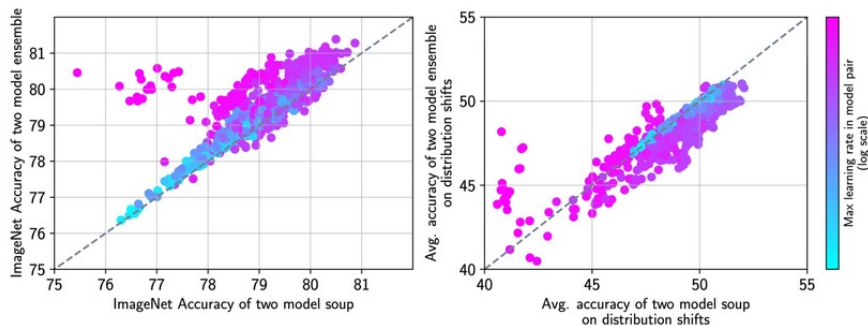
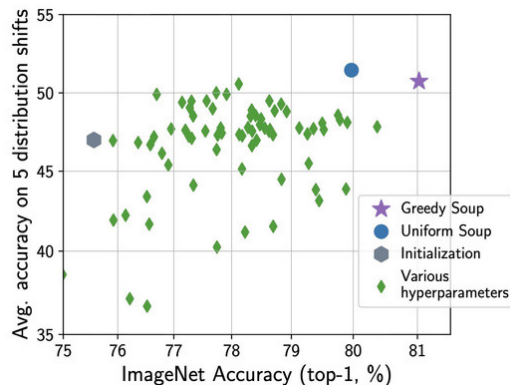
seed

lr



From "Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time" by Wortsman et al. 2022

Ensemble performance correlates with model soup performance



Uniform Soup

对所有候选微调模型等权平均:

$$\theta_{\text{uniform}} = \frac{1}{M} \sum_{i=1}^M \theta_i.$$

Greedy Soup

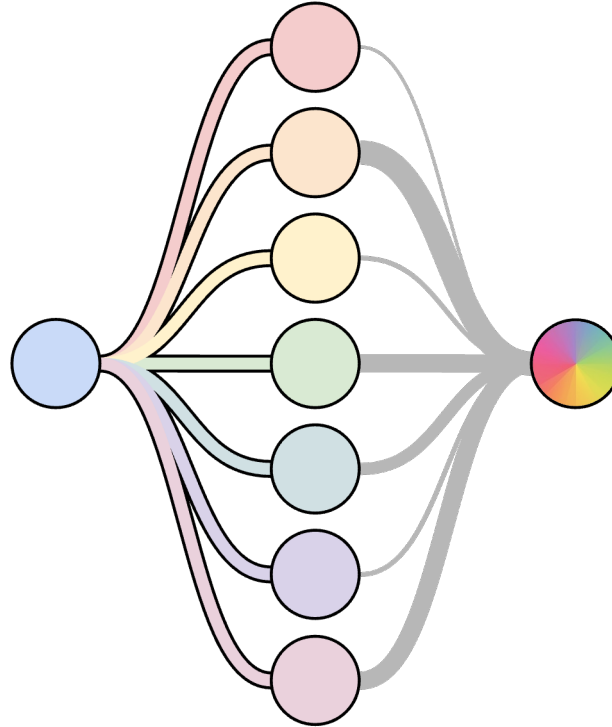
不是把所有模型都加入，而是通常从验证性能最好的模型开始，逐个尝试加入其他模型；只有加入后验证性能提高，才保留该模型。

因此，Greedy Soup属于基于验证性能的模型选择与参数平均。

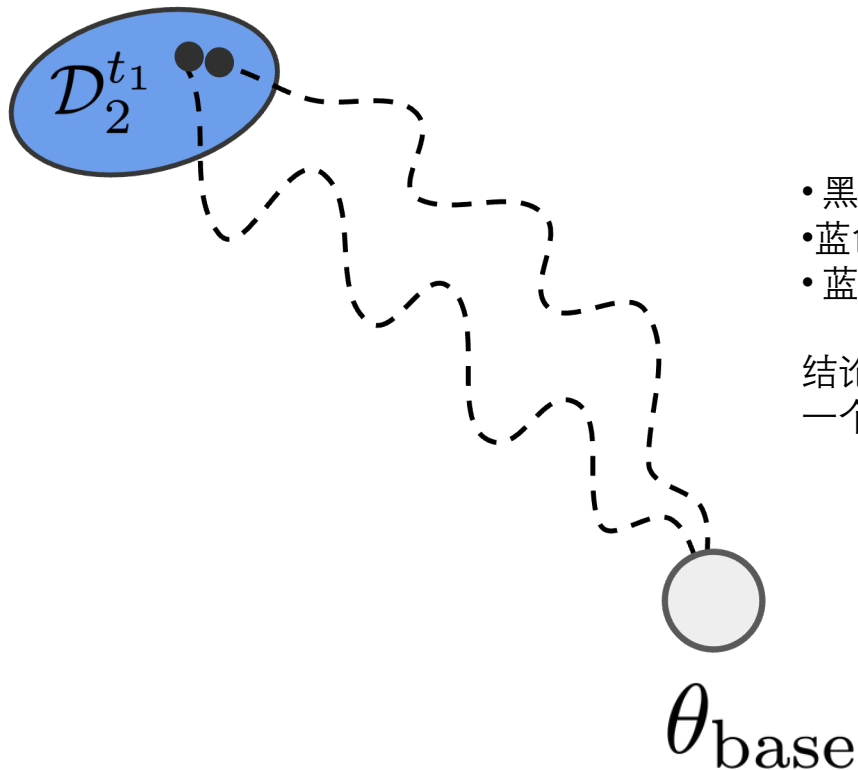
目标：模型具有集成互补性 $\Leftrightarrow$ 参数平均往往也更有效。

From “Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time” by Wortsman et al.  2022 

Multiple training trajectories
(***fine-tuning*** with ***different*** data)



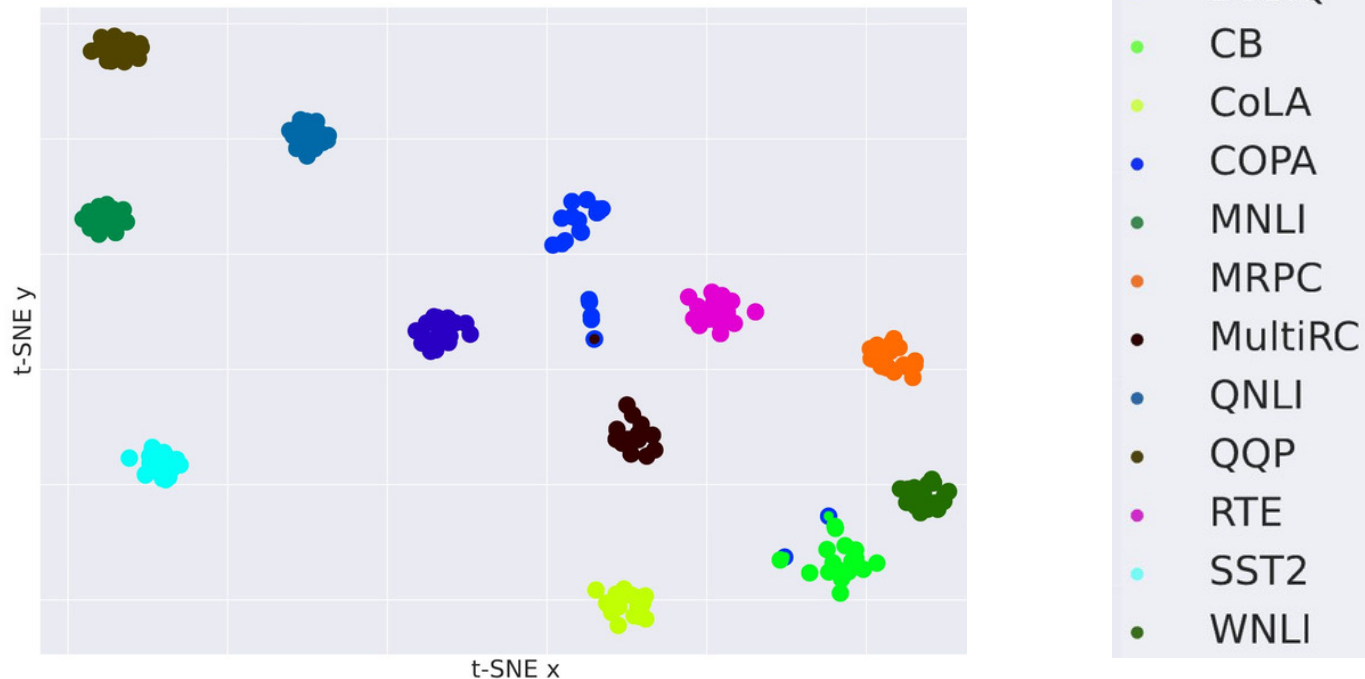
Models fine-tuned on similar data fall into the same region
(Same **dataset**)



- 黑色虚线：不同随机种子或训练设置产生的微调轨迹；
- 蓝色区域：针对某一数据集微调后形成的参数区域；
- 蓝色区域中的黑点：多次独立微调得到的模型。

结论：虽然由不同训练轨迹获得，但最终集中在参数空间的一个局部区域中。

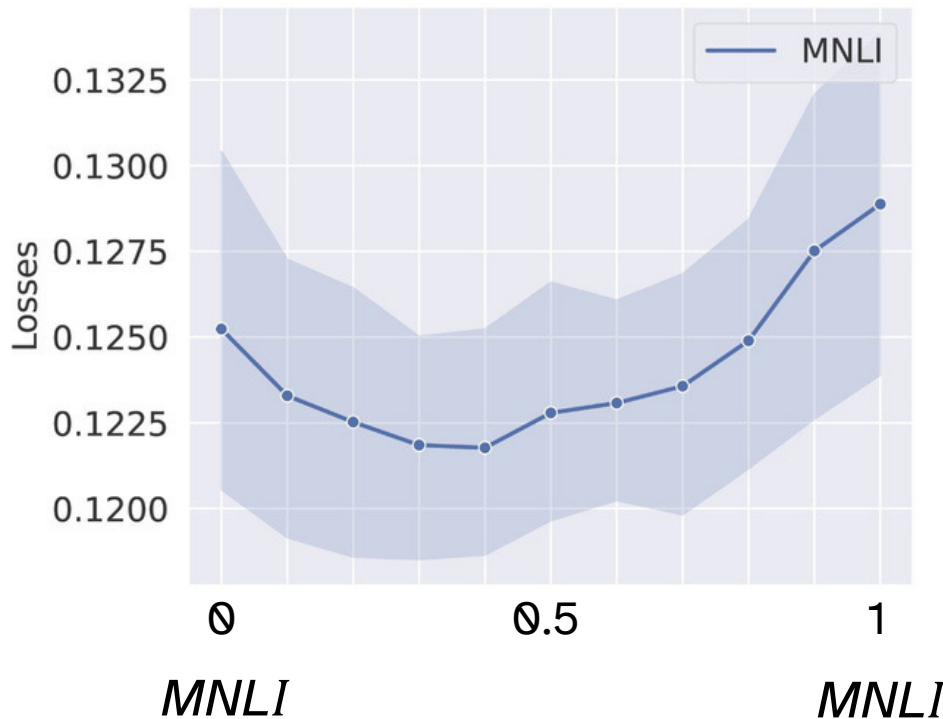
Models fine-tuned on similar data fall into the same region
(Same **dataset**)



每种颜色对应一个数据集，同一颜色的多个点代表在同一数据集上通过不同随机种子或训练运行得到的模型

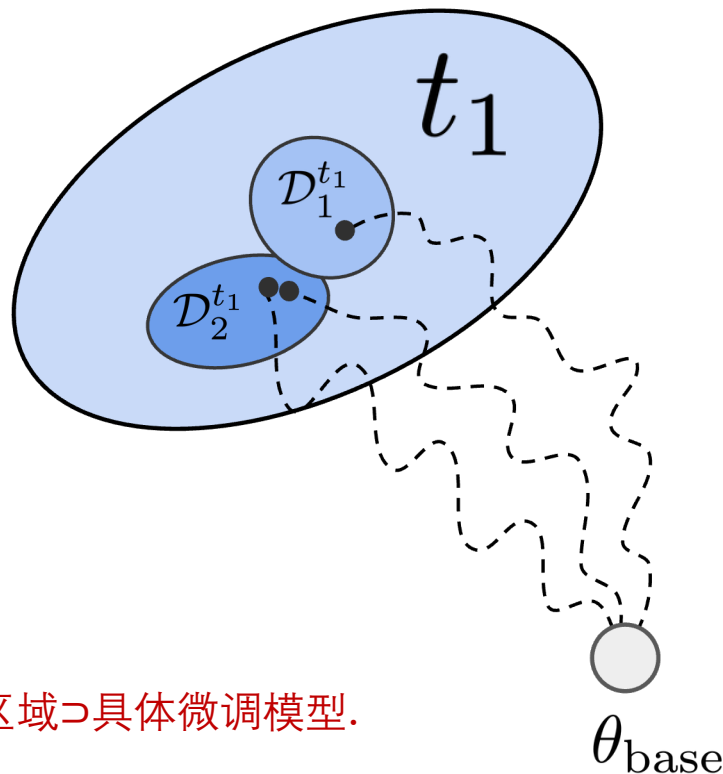
From "Knowledge is a Region in Weight Space for Finetuned Language Models" by Gueta et al. 2023

*Interpolating models fine-tuned on the **same dataset**
(interpolation MNLI models)*



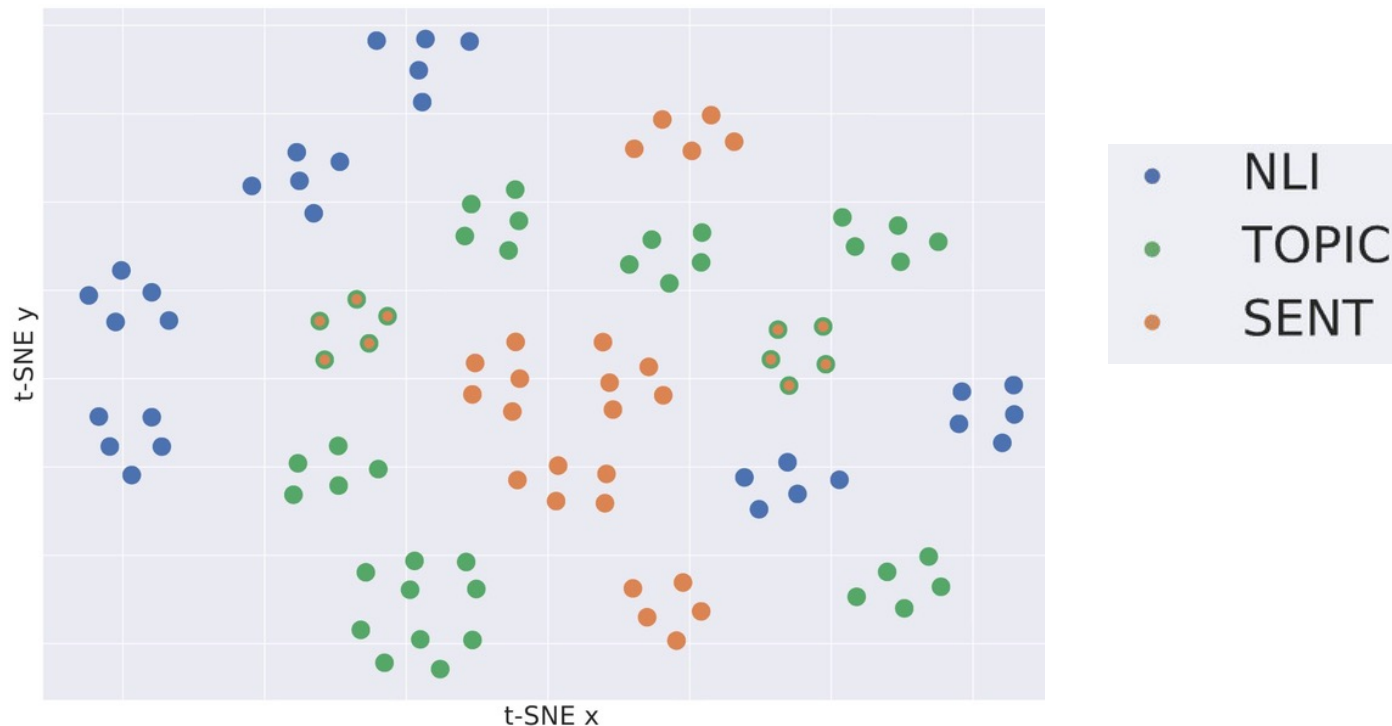
结果说明两个同数据集微调模型之间不仅没有明显的高损失障碍，而且中间参数位置还可能优于两个端点模型

Hypothesis: Models fine-tuned on **different datasets** from the **same task** are also close in the weight space, forming a continuous region



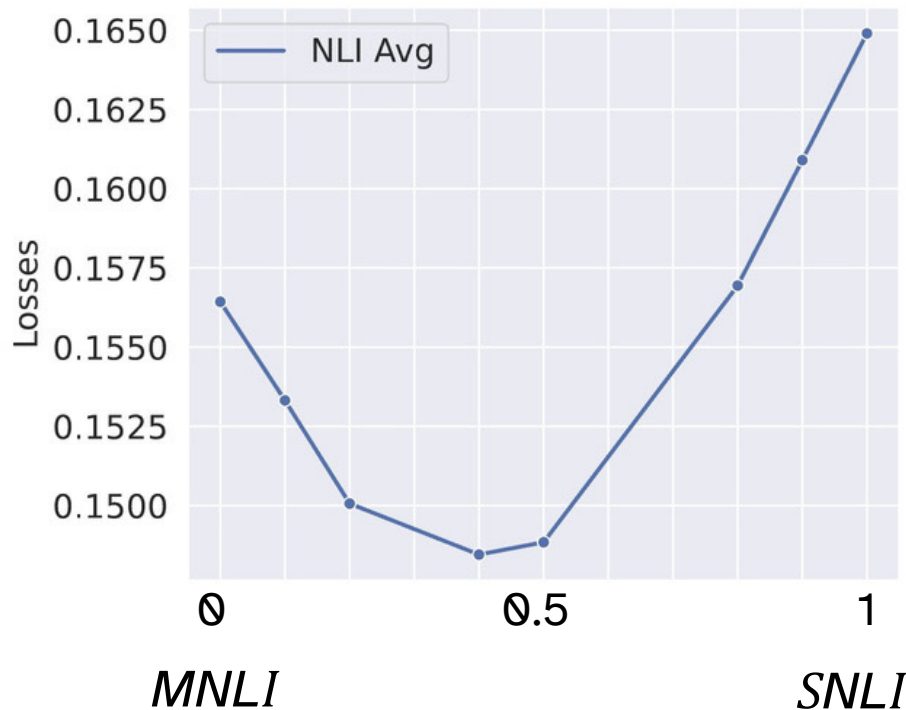
任务区域 $\supset$ 数据集区域 $\supset$ 具体微调模型.
进一步推广

*Models fine-tuned on **different datasets** from the **same task** are also close in the weight space*

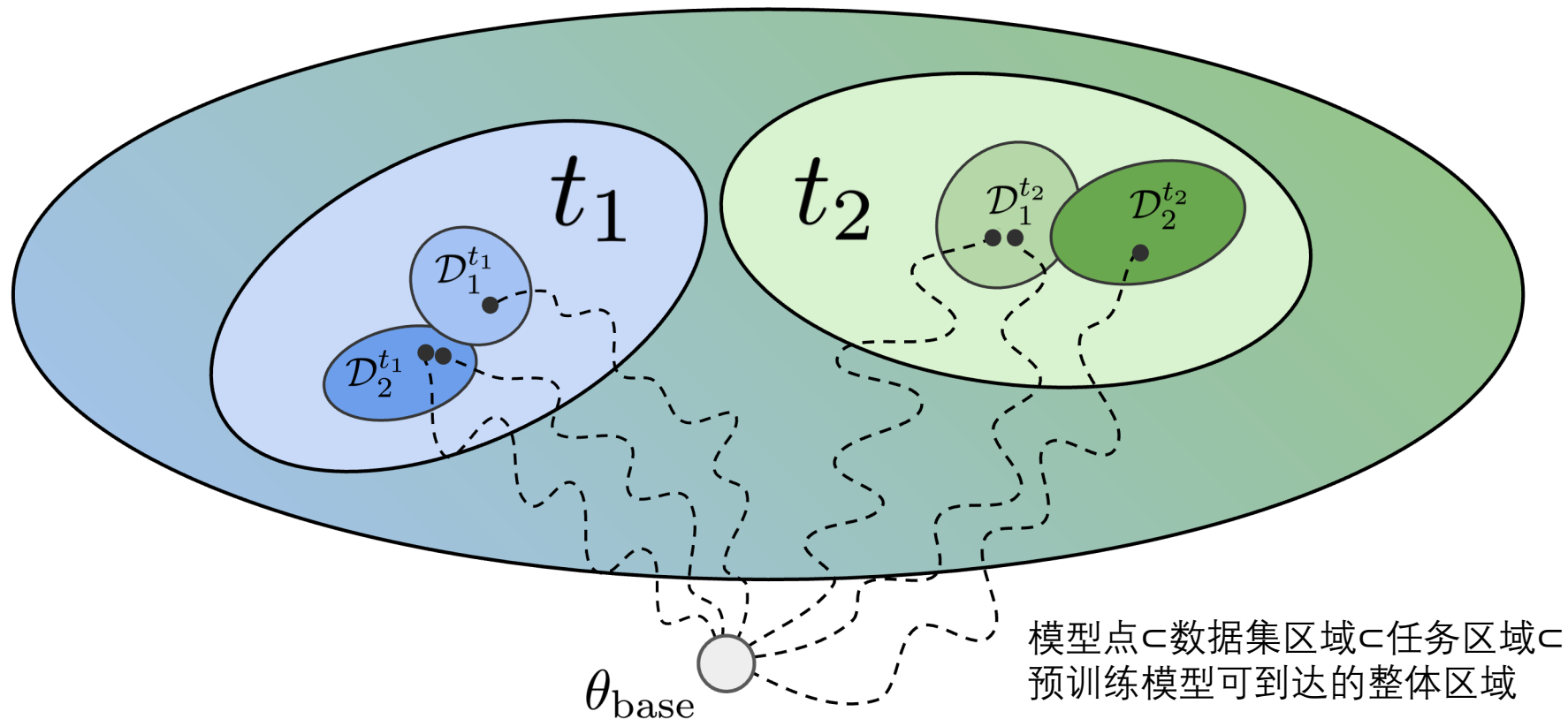


From "Knowledge is a Region in Weight Space for Finetuned Language Models" by Gueta et al.  2023 

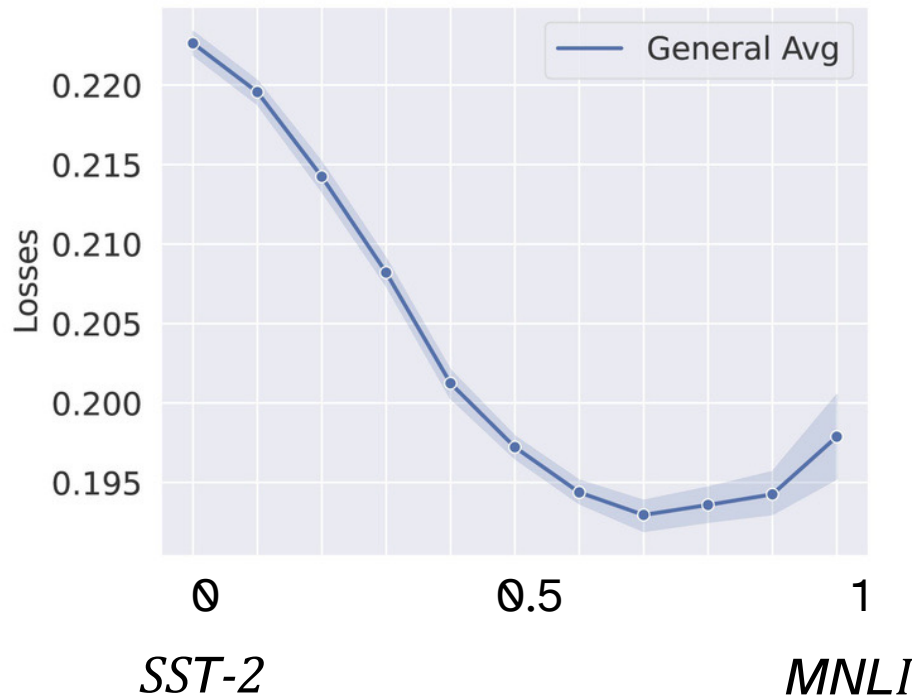
*Interpolating models fine-tuned on **same task (NLI)**, but **different datasets***
avg perf across NLI datasets (MNLI, SNLI, QNLI, ESNL, advNLI)



Knowledge is a Region in Weight Space

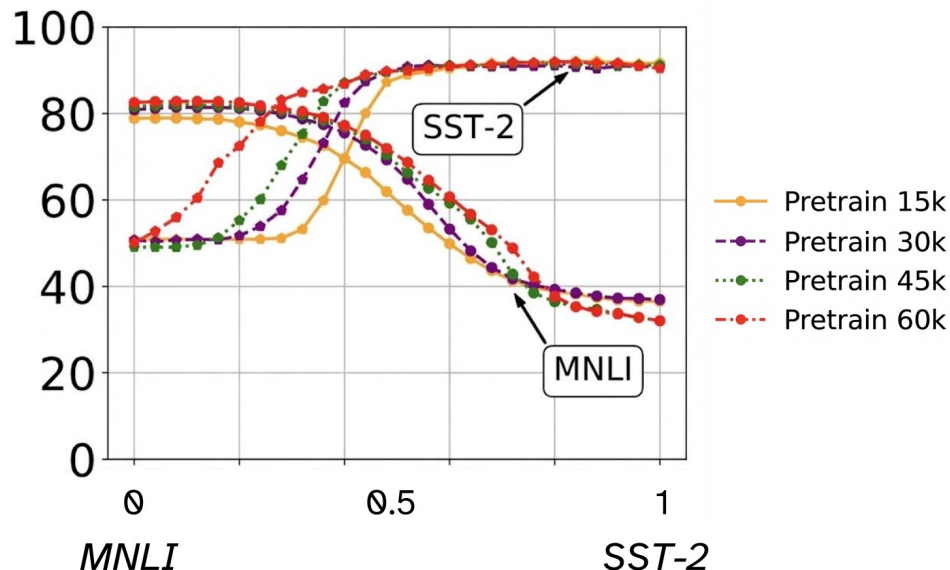


*Interpolating across different tasks/datasets
improve **general** performance (avg across tasks)*



跨任务参数插值也可能得到更好的任务平均性能

Pre-training pulls task boundaries closer

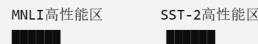


图的结论是：

随着预训练更加充分，两个任务的高性能区域变得更接近，甚至产生更大的重叠区域。

直观理解：

较弱预训练时：



两个区域中间可能存在明显性能下降。

较强预训练时：



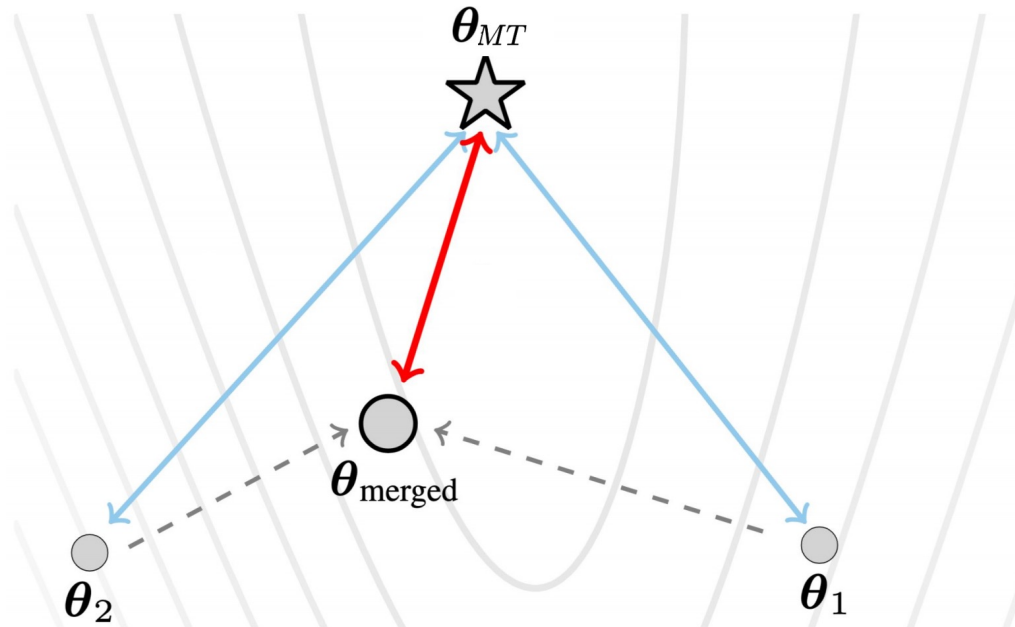
两个任务的高性能区有更多重叠，因此某个中间参数模型可能同时保留两种任务能力。

这提供了模型合并中一个非常重要的经验规律：

共享且充分的预训练 $\Rightarrow$ 不同任务模型更容易在权重空间中兼容

*So, merging always works
when using a pre-trained model?*

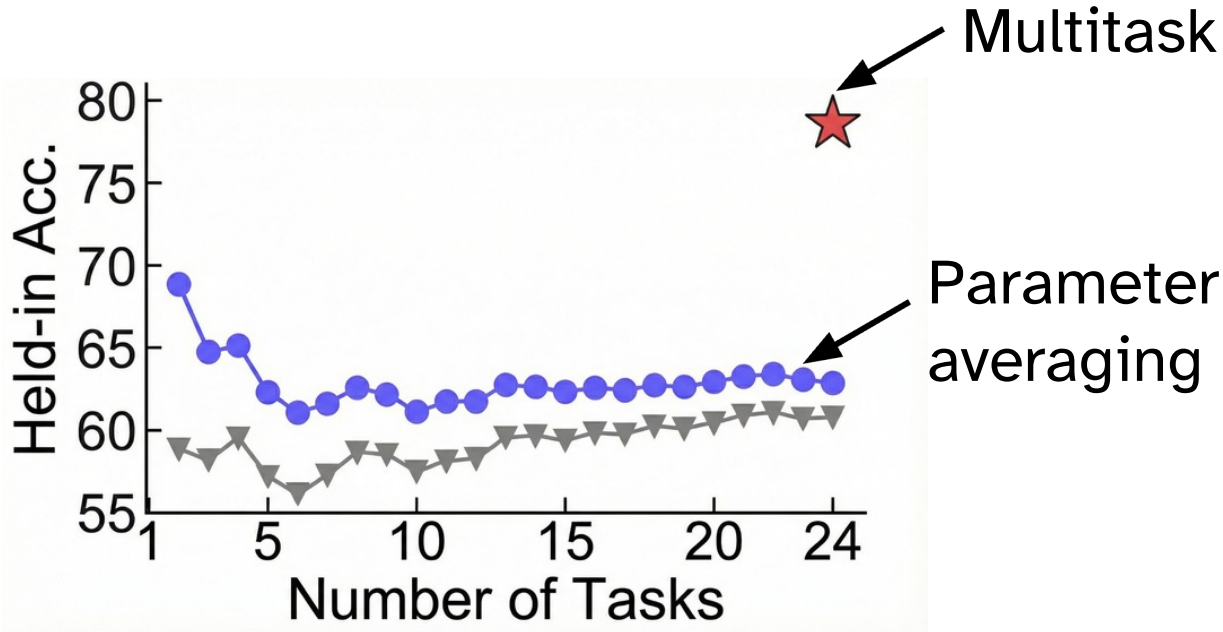
Model Merging vs. Multi-Task Learning



Model Merging performance decreases with more tasks

可能原因:

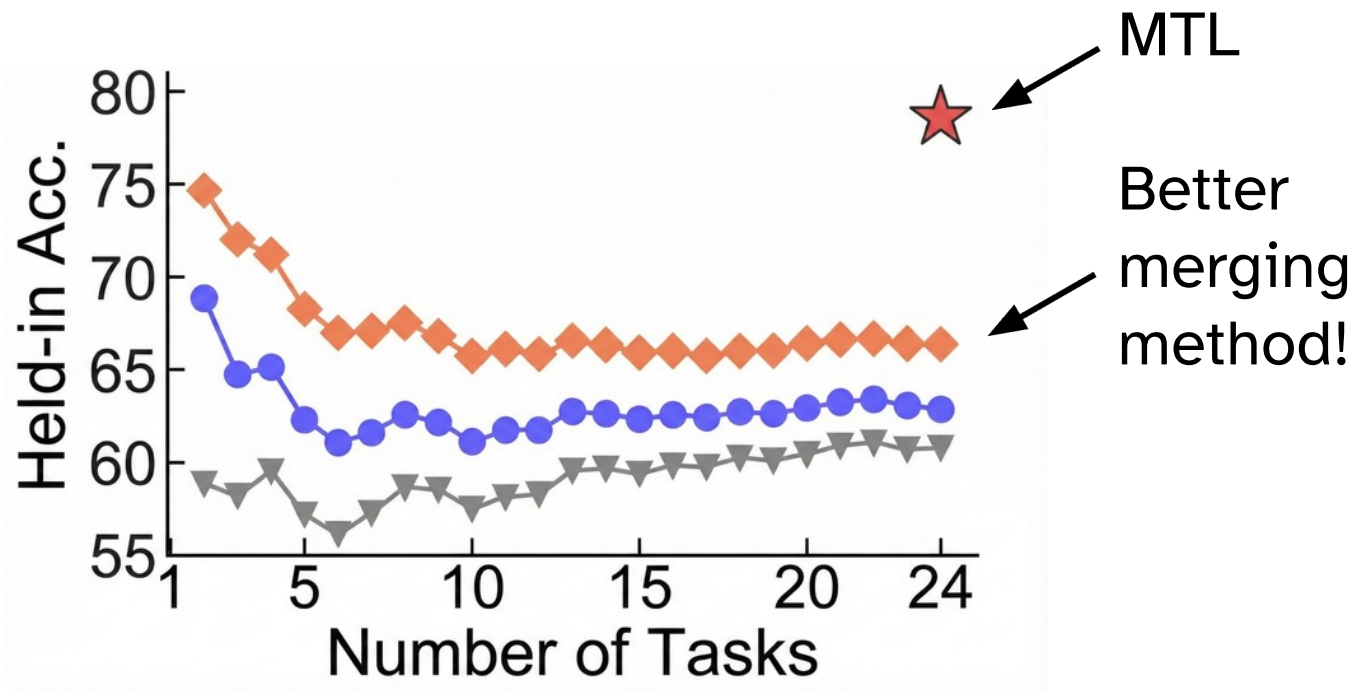
1. 模型更新相互抵消
2. 不相关更新产生噪声:
一个任务的重要参数变化, 对另一个任务可能没有帮助。大量不相关更新叠加后, 会偏离任一任务的理想区域。
3. 等权平均不合理



简单参数平均随任务数量增加而退化

From "Realistic Evaluation of Model Merging for Compositional Generalization" by Tam et al.

Better methods can improve performance - More on practice!



From "Realistic Evaluation of Model Merging for Compositional Generalization" by Tam et al.



Methods

What optimization problem does parameter averaging correspond to?

$$\theta_{\text{merged}}^{(j)} = \frac{\sum_{i=1}^M \lambda_i \theta_i^{(j)}}{\sum_{i=1}^M \lambda_i}$$

↑

*Hyperparameter
controlling the
importance of model i*

Model merging as jointly maximizing model posteriors

$$\theta_{\text{merged}}^{(j)} = \frac{\sum_{i=1}^M \lambda_i \theta_i^{(j)}}{\sum_{i=1}^M \lambda_i}$$

$\uparrow \theta \sim \mathcal{N}(\theta_i, I)$

$$\arg \max_{\theta} \sum_{i=1}^M \lambda_i \underbrace{\log p(\theta | \mathcal{D}_i)}_{\text{Log posterior for model}}$$

如果假设每个模型的后验都是各向同性高斯分布：

$$p(\theta | D_i) \approx \mathcal{N}(\theta_i, I),$$

那么最大化联合后验，恰好会得到普通加权平均。

Using the Laplace approximation produces Fisher merging

$\hat{F}_i^{(j)}$ 大: 第 j 个参数对模型 i 很重要, 稍微改变就会显著影响损失;

$\hat{F}_i^{(j)}$ 小: 该参数在模型 i 周围较不敏感, 可更自由地被其他模型修改。

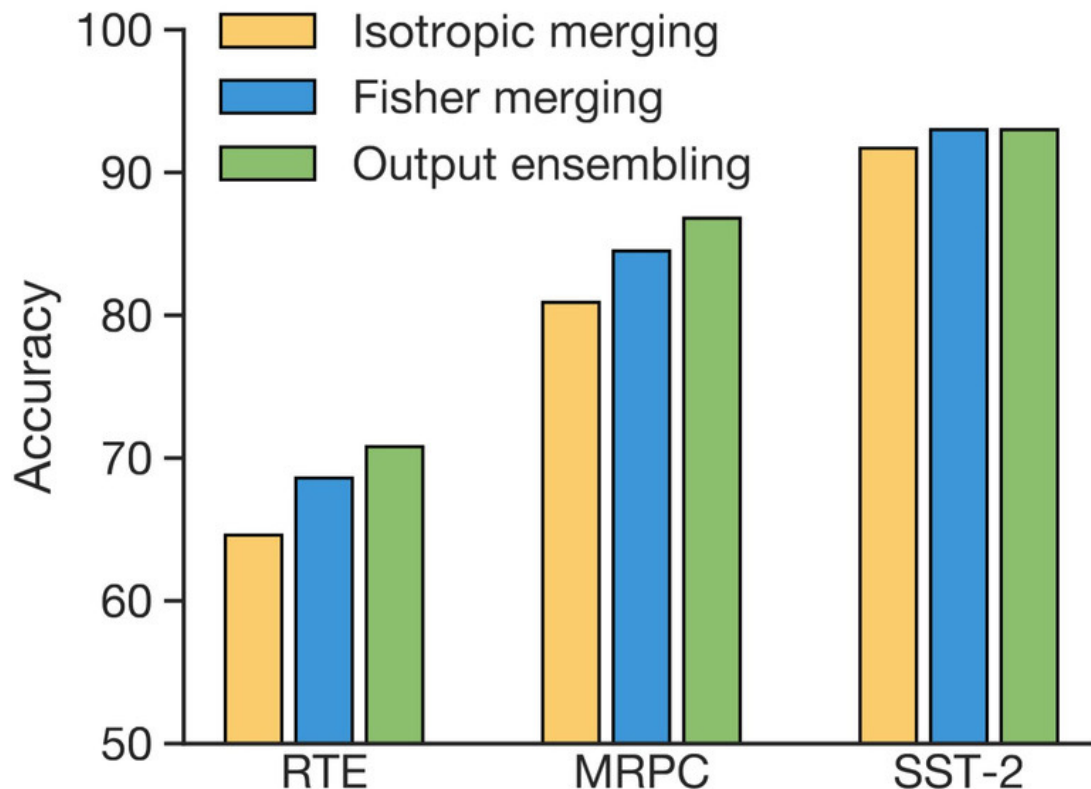
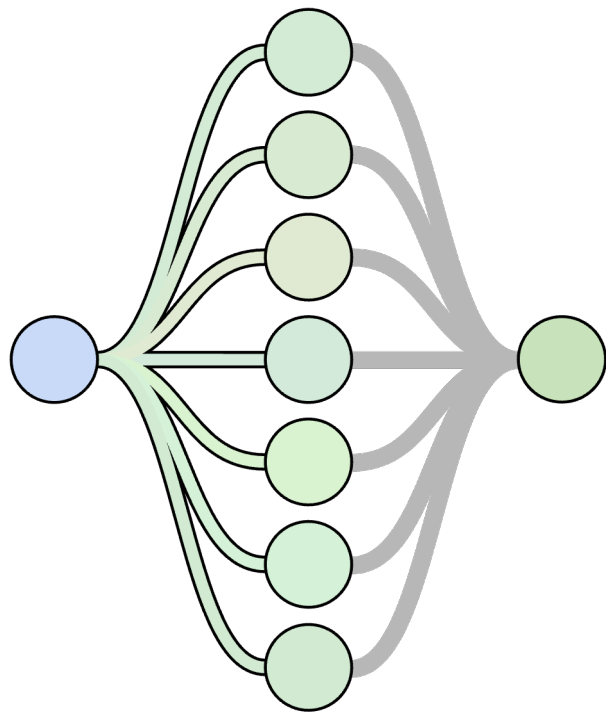
$$\theta_{\text{merged}}^{(j)} = \frac{\sum_{i=1}^M \lambda_i \hat{F}_i^{(j)} \theta_i^{(j)}}{\sum_{i=1}^M \lambda_i \hat{F}_i^{(j)}}$$

$$\uparrow \theta \sim \mathcal{N}(\theta_i, \hat{F}_i^{-1})$$

Fisher信息矩阵或其近似(对角矩阵)

$$\arg \max_{\theta} \sum_{i=1}^M \lambda_i \log p(\theta | \mathcal{D}_i)$$

Fisher merging outperforms "isotropic merging" (parameter averaging)



*RegMean: Merging by **maximizing activation agreement***

$$\min_W \underbrace{\|W^T \overbrace{X_1 - W_1^T X_1}^{\text{Model 1 weights and data}}\|^2 + \|W^T \overbrace{X_2 - W_2^T X_2}^{\text{Model 2 weights and data}}\|^2}_{\text{Merged weights}}$$

Merged weights via solving the linear regression problem

$$W_{\text{merged}} = \left(\sum_{i=1}^M X_i^{\top} X_i \right)^{-1} \sum_{i=1}^M (X_i^{\top} X_i W_i)$$

↑ 闭式解

$$\min_W \|W^T X_1 - W_1^T X_1\|^2 + \|W^T X_2 - W_2^T X_2\|^2$$

这里的: $X^T X$

是该层输入特征的Gram矩阵或二阶激活统计量。它反映不同输入方向在任务数据中的重要性。

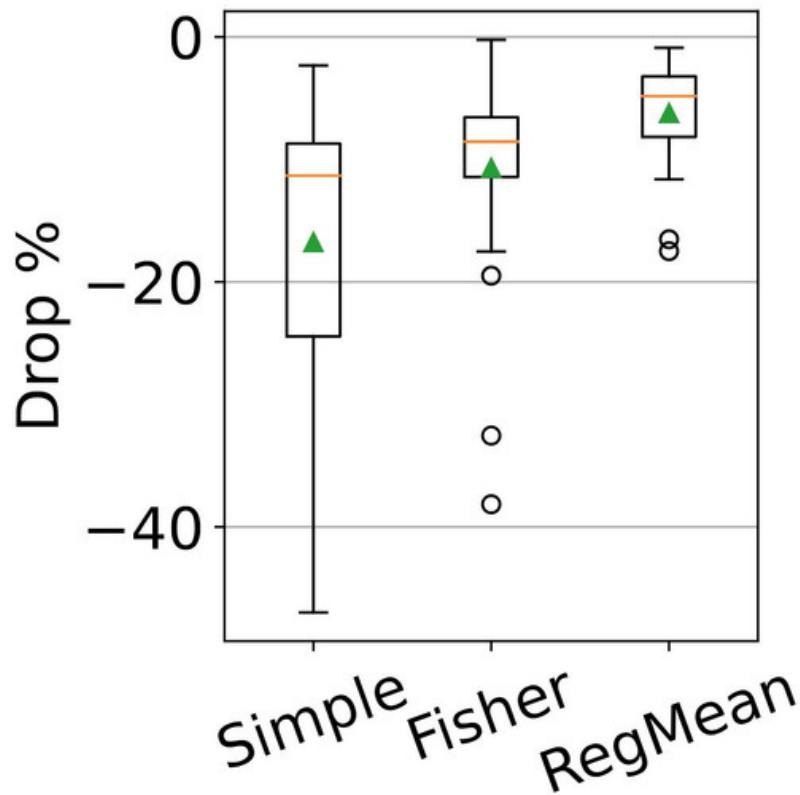
因此, RegMean可以理解为: 根据每个任务的特征协方差, 对权重矩阵进行方向相关的加权

RegMean can outperform Fisher Merging

三种方法保留的信息层次不同：

- Simple averaging: 只知道模型参数；
- Fisher merging: 知道单个参数的重要性；
- RegMean: 知道不同输入特征方向如何激活该层。

RegMean不是逐元素独立处理参数，而是利用矩阵结构描述不同参数方向之间的相关性。



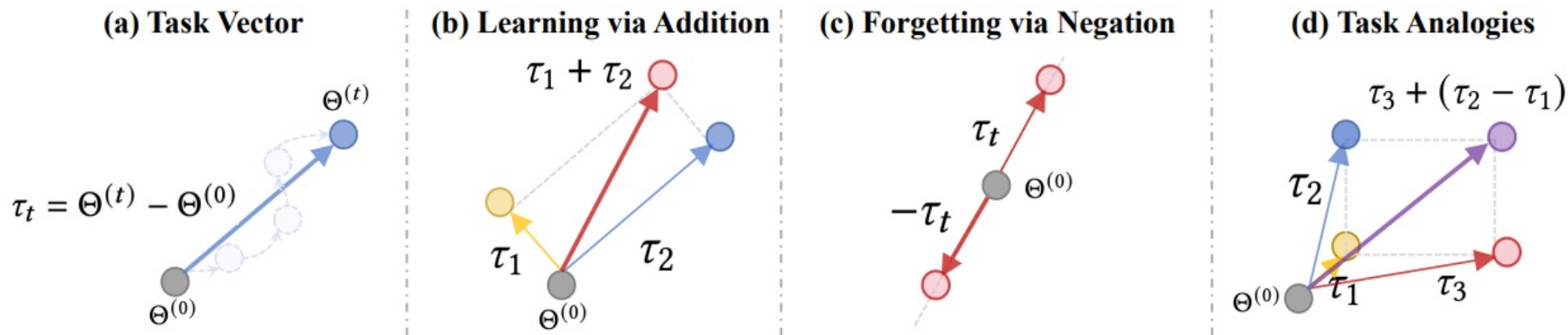
Connecting simple averaging, Fisher merging, and RegMean

Simple averaging: $\theta_{\text{merged}} = \left(\sum_{i=1}^M I \right)^{-1} \left(\sum_{i=1}^M I \theta_i \right)$

Fisher merging: $\theta_{\text{merged}} = \left(\sum_{i=1}^M \hat{F}_i \right)^{-1} \left(\sum_{i=1}^M \hat{F}_i \theta_i \right)$

RegMean: $\theta_{\text{merged}} = \left(\sum_{i=1}^M \frac{1}{N_i} Z_i^\top Z_i \right)^{-1} \left(\sum_{i=1}^M \frac{1}{N_i} (Z_i^\top Z_i) \theta_i \right)$

Task vectors provide an alternative way of merging...



希望把某种任务变化
迁移到另一个领域

... that is, in a sense, equivalent to averaging

$$\tau_i = \theta_i - \theta_p$$

Task
vectors:

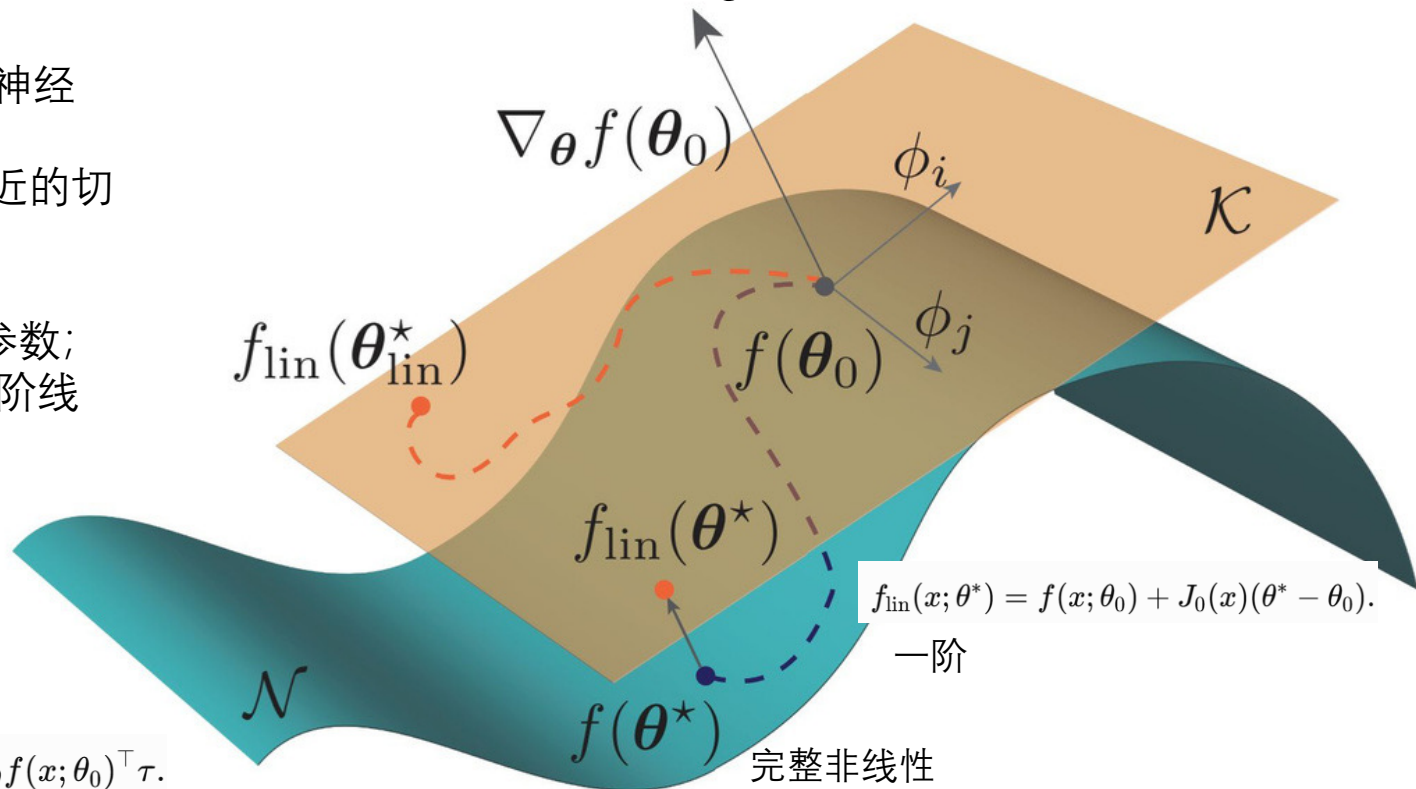
$$\theta_{\text{merged}} = \theta_p + \lambda \sum_{i=1}^M \tau_i$$

Simple averaging:

$$\begin{aligned} \theta_{\text{merged}} &= \frac{1}{\sum_i \lambda_i} \sum_{i=1}^M \lambda_i \theta_i \\ &= \frac{1}{\sum_i \lambda_i} \sum_{i=1}^M \lambda_i (\theta_p + \tau_i) \\ &= \theta_p + \frac{1}{\sum_i \lambda_i} \sum_{i=1}^M \lambda_i \tau_i \end{aligned}$$

Do task vectors work because fine-tuning is inherently linear?

弯曲曲面 $\mathcal{N}$: 非线性神经网络函数形成的流形;
 平面 $\mathcal{K}$: 基础模型附近的切空间;
 θ_0 : 预训练参数;
 θ^* : 非线性微调后的参数;
 f_{lin} : 在 θ_0 附近进行一阶线性化的模型。

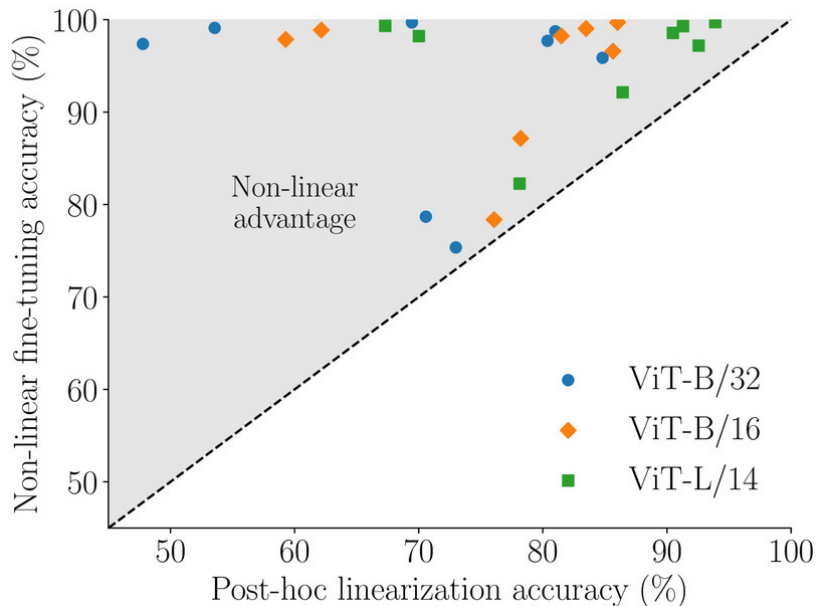


线性化通常写为:

$$f_{\text{lin}}(x; \theta_0 + \tau) = f(x; \theta_0) + \nabla_{\theta} f(x; \theta_0)^{\top} \tau.$$

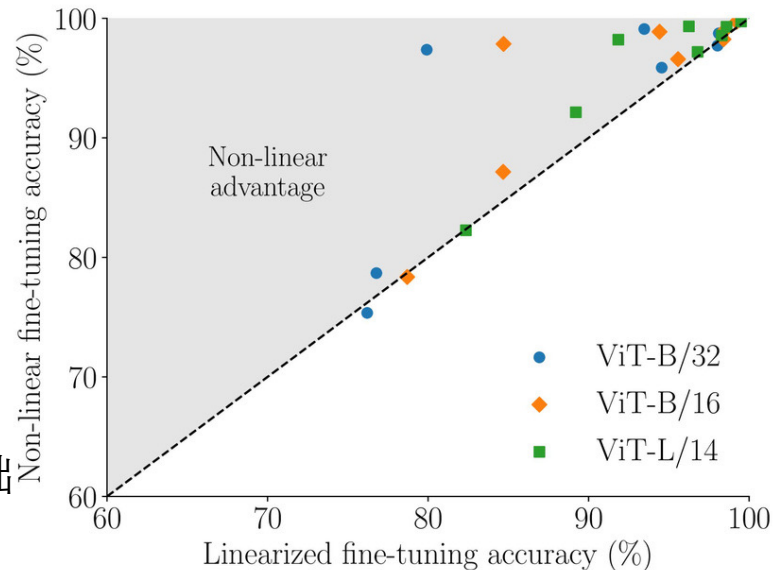
8个任务训练完merge的结果

Method		ViT-B/32	
		Abs. (↑)	Norm. (↑)
Pre-trained	$f(\cdot; \theta_0)$	48.4	—
Non-lin. FT	$f(\cdot; \theta_0 + \tau)$	71.4	76.5
Post-hoc lin.	$f_{\text{lin}}(\cdot; \theta_0 + \tau)$	57.1	81.9



Linearized fine-tuning improves performance

Method		ViT-B/32	
		Abs. ($\uparrow$)	Norm. ($\uparrow$)
Pre-trained	$f(\cdot; \theta_0)$	48.4	—
Non-lin. FT	$f(\cdot; \theta_0 + \tau)$	71.4	76.5
Post-hoc lin.	$f_{\text{lin}}(\cdot; \theta_0 + \tau)$	57.1	81.9
Linear. FT	$f_{\text{lin}}(\cdot; \theta_0 + \tau_{\text{lin}})$	76.5	85.4



虽然“事后线性化”效果不好，但如果从训练开始就固定基础模型的Jacobian，并直接优化切空间中的线性化模型，即：

$$f_{\text{lin}} = f(x; \theta_0) + J_{\theta_0}(x)\tau,$$

非线性微调单任务性能强，但切空间微调可能产生更适合 Task Arithmetic的更新

从训练开始固定基础模型 Jacobian

首先在预训练参数 θ_0 处计算：

$$J_0(x) = \nabla_{\theta} f(x; \theta) \big|_{\theta=\theta_0}.$$

然后用一阶泰勒展开替代原始非线性网络：

$$f_{\text{lin}}(x; \theta_0 + \tau) = f(x; \theta_0) + J_0(x)\tau$$

训练时：

- $f(x; \theta_0)$ 固定；
- $J_0(x)$ 固定；
- 只优化参数增量 τ 。

因此，模型关于 τ 是线性的。

这里的 τ 一般与**整个网络参数维度相同**，可以包含：

- 注意力层参数增量；
- MLP参数增量；
- 卷积层参数增量；
- 归一化层参数增量；
- 输出层参数增量。

所以它不是只训练最后一层。

8. 为什么固定Jacobian有利于Task Arithmetic?

对任务 A 和任务 B ，分别在线性化模型中训练得到：

$$\tau_A, \quad \tau_B.$$

由于模型关于参数增量是严格线性的：

$$f_{\text{lin}}(x; \theta_0 + \tau_A + \tau_B) = f(x; \theta_0) + J_0(x)\tau_A + J_0(x)\tau_B.$$

也就是：

$$\Delta f_{A+B} = \Delta f_A + \Delta f_B$$

参数向量相加，会直接对应函数变化相加。

而在原始非线性网络中：

$$f(\theta_0 + \tau_A + \tau_B)$$

除了线性项外，还包含二阶交叉项，例如：

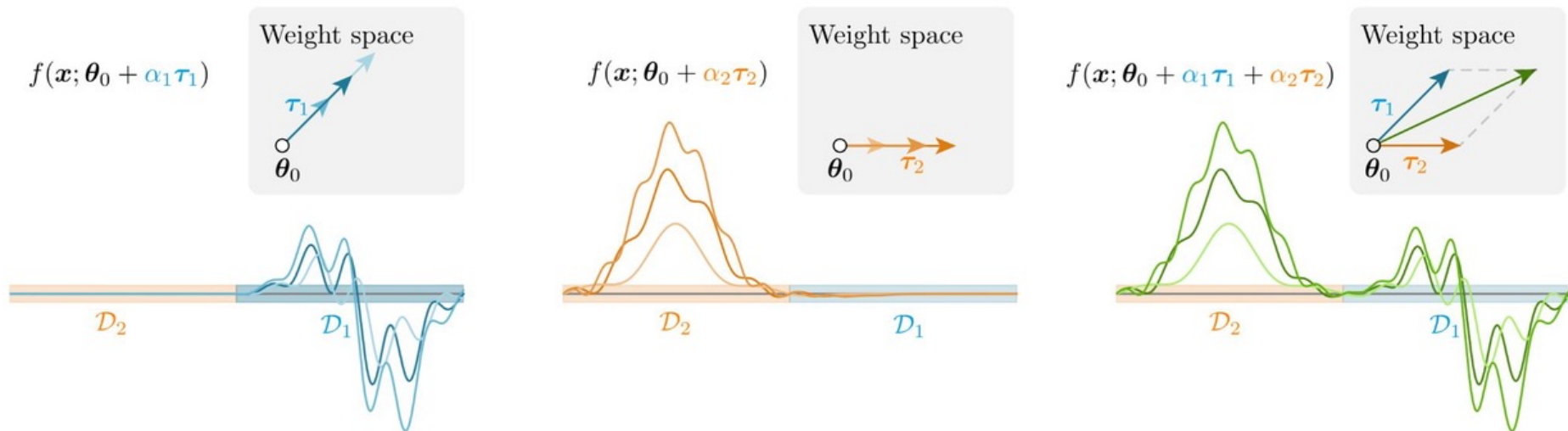
$$\tau_A^{\top} H_0 \tau_B.$$

这些交叉项会导致：

- 任务相互干扰；
- Task Vector相加不再对应能力简单叠加；
- 缩放系数变得敏感；
- 合并结果偏离预期。

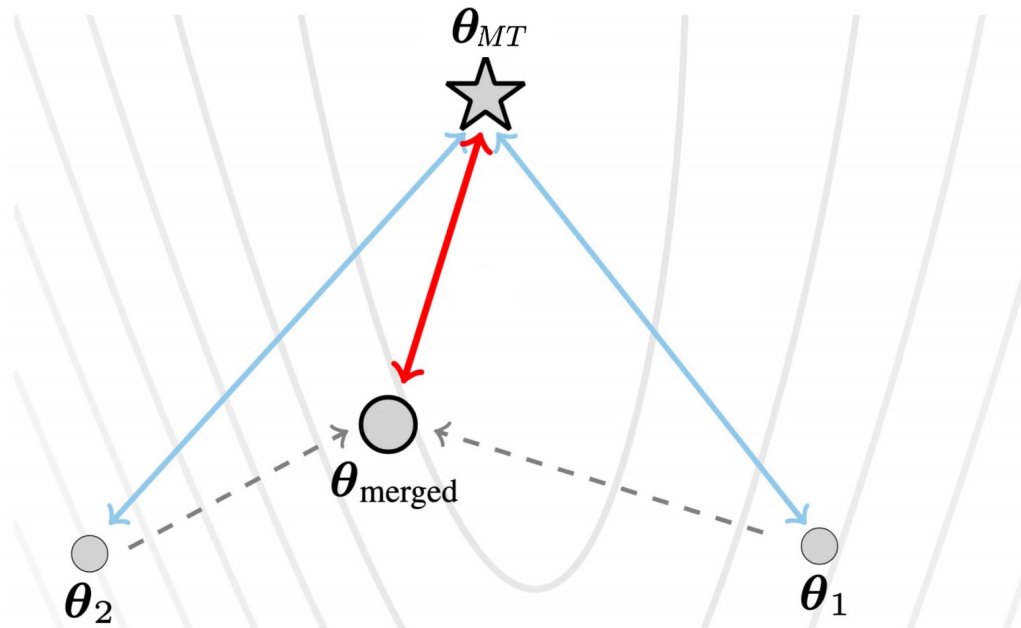
固定Jacobian相当于人为去掉这些高阶交互，使Task Arithmetic具有更明确的线性结构。

"Weight disentanglement" provides an alternative view



不同任务向量在函数作用上相互解耦，而不只是参数向量在欧氏空间中方向不同

Characterizing and minimizing the *merging error*



Quantifying merging error via "gradient mismatch"

$$\underbrace{\theta_{MT} - \left(\theta_p + \lambda \sum_{i=1}^M \tau_i \right)}_{\text{Merging error}} = -\lambda \sum_{i=1}^M \underbrace{\hat{F}_p^{-1}}_{\substack{\text{Preconditioner for} \\ \text{measuring distance from} \\ \text{initialization}}} \overbrace{(\nabla \ell_i(\theta_{MT}) - \nabla \ell_i(\theta_i))}^{\text{"Gradient mismatch"}}$$

Approximating the multitask model gradient

$$\theta_{MT} - \left(\theta_p + \lambda \sum_{i=1}^M \tau_i \right) = -\lambda \sum_{i=1}^M \hat{F}_p^{-1} (\nabla \ell_i(\theta_{MT}) - \nabla \ell_i(\theta_i))$$

If θ_i is indeed close to the local optimal solution of task i , then:

$$\nabla \ell_i(\theta_i) \approx 0.$$

and

$$\nabla \ell_i(\theta_{MT}) \approx \nabla \ell_i(\theta_i) + \hat{F}_i(\theta_{MT} - \theta_i)$$

Merging while minimizing gradient mismatch

$$\theta_{MT} - \left(\theta_p + \lambda \sum_{i=1}^M \tau_i \right) = -\lambda \sum_{i=1}^M \hat{F}_p^{-1} (\nabla \ell_i(\theta_{MT}) - \nabla \ell_i(\theta_i))$$

$$\nabla \ell_m(\theta_{MT}) \approx \nabla \ell_m(\theta_m) + \hat{F}_m(\theta_{MT} - \theta_m)$$

$$\theta_{\text{merged}} = \theta_p + \lambda \left(\hat{F}_p + \lambda \sum_{i=1}^M \hat{F}_i \right)^{-1} \sum_{i=1}^M (\hat{F}_p + \hat{F}_i) \tau_i$$

详细过程（GPT推导）：

第1-3步：前面得到的

$$\theta_{\text{MT}} - \left(\theta_p + \lambda \sum_{i=1}^M \tau_i \right) = -\lambda \sum_{i=1}^M \widehat{F}_p^{-1} [\nabla \ell_i(\theta_{\text{MT}}) - \nabla \ell_i(\theta_i)]. \quad (1)$$

$$\nabla \ell_i(\theta_{\text{MT}}) - \nabla \ell_i(\theta_i) \approx \widehat{F}_i (\theta_{\text{MT}} - \theta_i). \quad (3)$$

将式 (3) 代入式 (1)：

$$\theta_{\text{MT}} - \theta_p - \lambda \sum_i \tau_i \approx -\lambda \widehat{F}_p^{-1} \sum_i \widehat{F}_i (\theta_{\text{MT}} - \theta_i). \quad (4)$$

为简化书写，下面省略求和上下限，默认：

$$\sum_i = \sum_{i=1}^M.$$

第四步：两边左乘 $\widehat{F}_p$

$$\widehat{F}_p \left(\theta_{\text{MT}} - \theta_p - \lambda \sum_i \tau_i \right) = -\lambda \sum_i \widehat{F}_i (\theta_{\text{MT}} - \theta_i). \quad (5)$$

展开两侧：

$$\widehat{F}_p \theta_{\text{MT}} - \widehat{F}_p \theta_p - \lambda \widehat{F}_p \sum_i \tau_i = -\lambda \sum_i \widehat{F}_i \theta_{\text{MT}} + \lambda \sum_i \widehat{F}_i \theta_i. \quad (6)$$

第五步：把包含 θ_{MT} 的项移到左边

$$\left(\widehat{F}_p + \lambda \sum_i \widehat{F}_i \right) \theta_{\text{MT}} = \widehat{F}_p \theta_p + \lambda \widehat{F}_p \sum_i \tau_i + \lambda \sum_i \widehat{F}_i \theta_i. \quad (7)$$

第六步：使用 $\theta_i = \theta_p + \tau_i$

将：

$$\theta_i = \theta_p + \tau_i$$

代入最后一项：

$$\lambda \sum_i \widehat{F}_i \theta_i = \lambda \sum_i \widehat{F}_i (\theta_p + \tau_i).$$

所以式 (7) 的右侧变成：

$$\widehat{F}_p \theta_p + \lambda \widehat{F}_p \sum_i \tau_i + \lambda \sum_i \widehat{F}_i \theta_p + \lambda \sum_i \widehat{F}_i \tau_i.$$

把与 θ_p 相关的项合并：

$$= \left(\widehat{F}_p + \lambda \sum_i \widehat{F}_i \right) \theta_p + \lambda \sum_i \left(\widehat{F}_p + \widehat{F}_i \right) \tau_i. \quad (8)$$

因此：

$$\left(\widehat{F}_p + \lambda \sum_i \widehat{F}_i \right) \theta_{\text{MT}} = \left(\widehat{F}_p + \lambda \sum_i \widehat{F}_i \right) \theta_p + \lambda \sum_i \left(\widehat{F}_p + \widehat{F}_i \right) \tau_i. \quad (9)$$

第七步：求解 θ_{MT}

两边左乘：

$$\left(\widehat{F}_p + \lambda \sum_i \widehat{F}_i \right)^{-1},$$

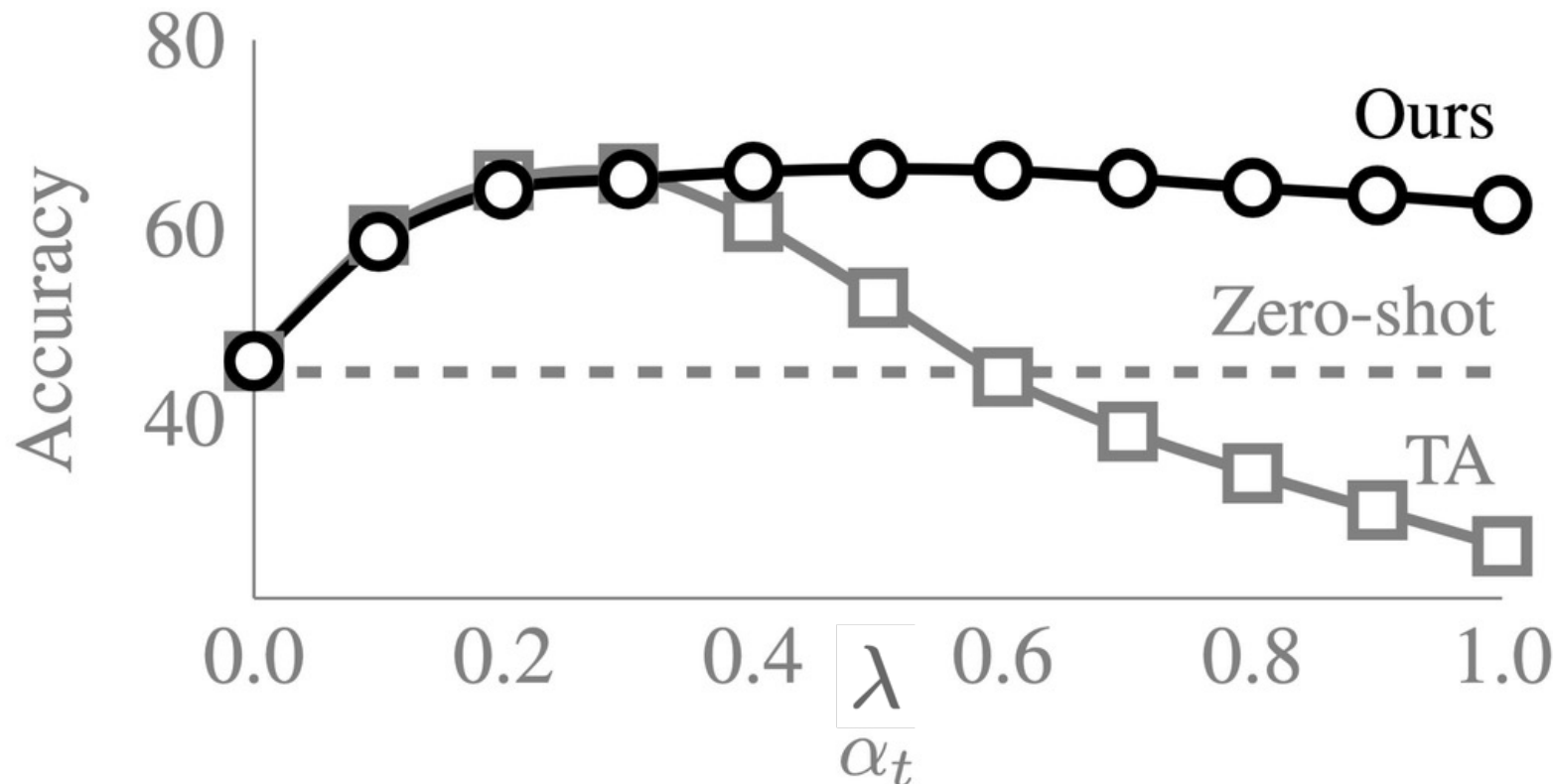
得到：

$$\theta_{\text{MT}} \approx \theta_p + \lambda \left(\widehat{F}_p + \lambda \sum_{i=1}^M \widehat{F}_i \right)^{-1} \sum_{i=1}^M \left(\widehat{F}_p + \widehat{F}_i \right) \tau_i. \quad (10)$$

由于真正的 θ_{MT} 不可获得，作者把右侧定义为合并模型：

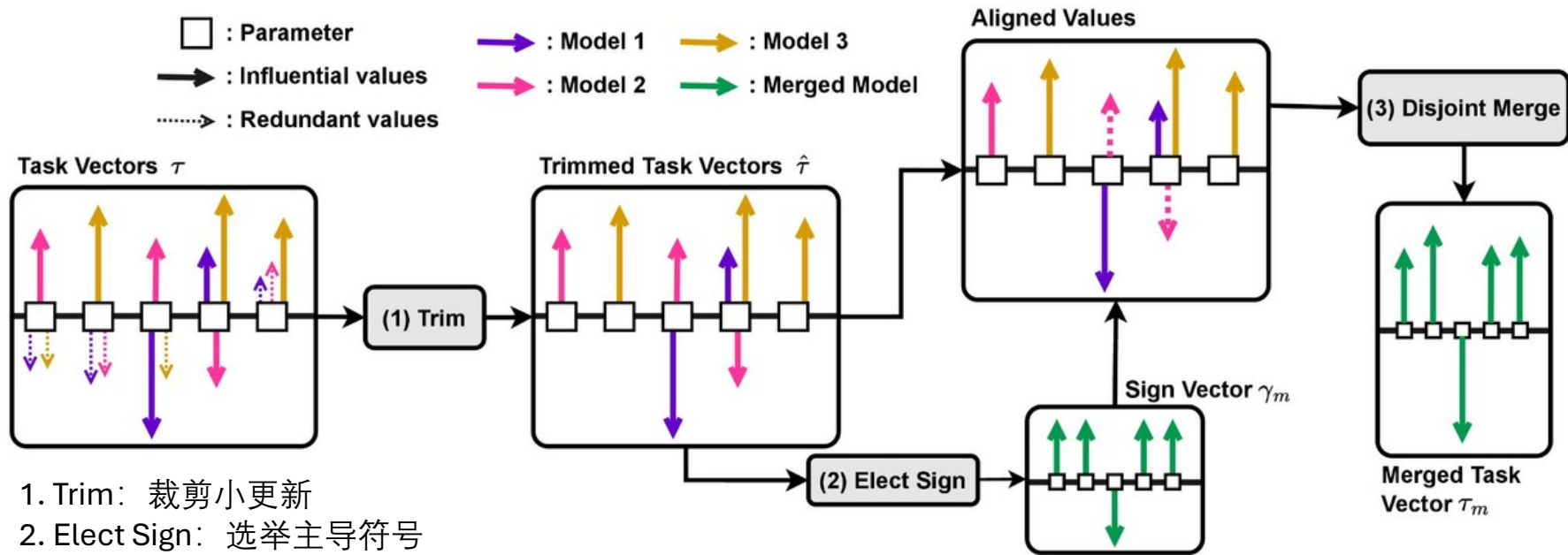
$$\theta_{\text{merged}} = \theta_p + \lambda \left(\widehat{F}_p + \lambda \sum_{i=1}^M \widehat{F}_i \right)^{-1} \sum_{i=1}^M \left(\widehat{F}_p + \widehat{F}_i \right) \tau_i$$

Minimizing gradient mismatch reduces sensitivity of λ



From "Model Merging by Uncertainty-Based Gradient Matching" by Daheim et al.

TIES Merging aims to resolve "interference" between task vectors

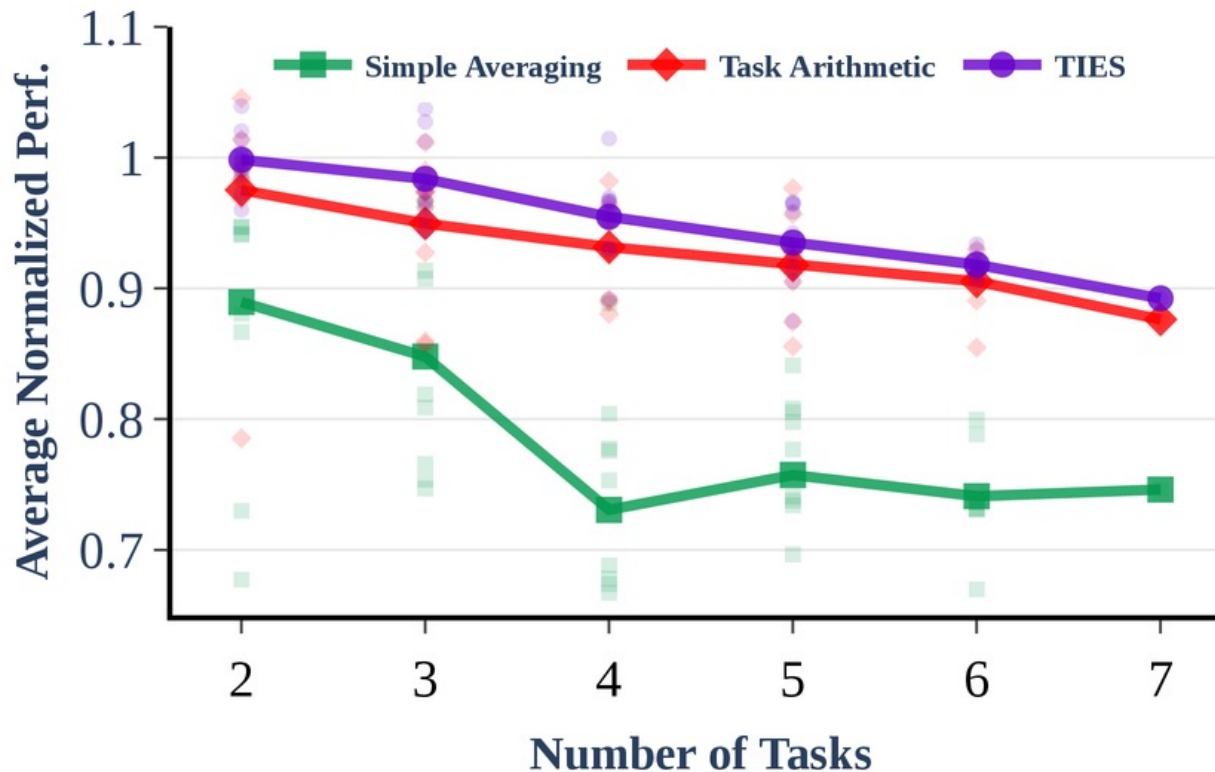


1. Trim: 裁剪小更新

2. Elect Sign: 选举主导符号

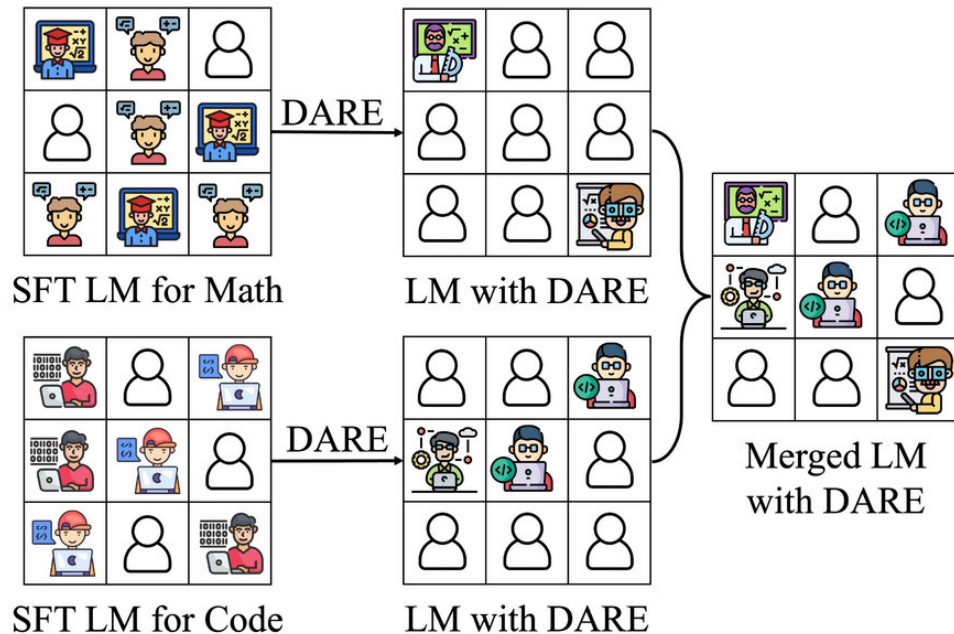
3. Disjoint Merge: 只合并符号一致的更新

TIES helps retain individual model performance



From "Resolving Interference When Merging Models" by Yadav et al.

Dropping parameter updates randomly can also help



该页本身没有展开DARE的完整公式。在常见实现中，它通常对Task Vector使用随机掩码：

$$\tilde{\tau} = \frac{m \odot \tau}{1 - p}, \quad m_j \sim \text{Bernoulli}(1 - p),$$

即：

- 以概率 p 丢弃更新；
- 对剩余更新重新缩放，使其期望幅值基本不变。

直观上，许多微调更新具有冗余性。随机稀疏化可以减少不同任务在同一参数坐标上的碰撞。

TIES与DARE的差别是：

- TIES：根据幅值和符号有结构地筛选；
- DARE：采用随机丢弃和重缩放。

随机逐参数mask模型增量剩下的值乘以 $(1/(1-p))$ ，之后进行merge。

DARE本身主要是合并前的随机稀疏化步骤，而不是完整的合并规则。

Avoiding interference in task singular vectors

$$\begin{array}{c}
 \Delta_i \\
 \left[\begin{array}{ccc} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{array} \right] = \left[\begin{array}{ccc|c} | & | & | & / \\ | & | & | & / \\ | & | & | & / \\ | & | & | & / \end{array} \right] \left[\begin{array}{ccc|c} \cdot & & & / \\ & \cdot & & / \\ & & \cdot & / \\ & & & / \end{array} \right] \left[\begin{array}{ccc|c} \hline \hline \hline \hline \end{array} \right] \\
 \\
 \Delta_j \\
 \left[\begin{array}{ccc} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{array} \right] = \left[\begin{array}{ccc|c} | & | & | & / \\ | & | & | & / \\ | & | & | & / \\ | & | & | & / \end{array} \right] \left[\begin{array}{ccc|c} \cdot & & & / \\ & \cdot & & / \\ & & \cdot & / \\ & & & / \end{array} \right] \left[\begin{array}{ccc|c} \hline \hline \hline \hline \end{array} \right] \\
 U_j \quad \Sigma_j \quad V_j^\top
 \end{array}$$

Diagram illustrating the decomposition of task update matrices Δ_i and Δ_j into singular vectors U_i, U_j , singular values Σ_i, Σ_j , and singular vectors $V_i^\top, V_j^\top$. The matrices Δ_i and Δ_j are shown as 4x3 matrices of red and green dots respectively. The matrices U_i and U_j are shown as 4x4 matrices of red and green vertical lines respectively. The matrices Σ_i and Σ_j are shown as 4x4 matrices of red and green dots respectively. The matrices $V_i^\top$ and $V_j^\top$ are shown as 4x4 matrices of red and green horizontal lines respectively. A dashed line and a perpendicular symbol indicate the orthogonality of the singular vectors U_i and U_j .

对某一层的任务更新矩阵:

$$\Delta_i = W_i - W_0,$$

执行SVD:

$$\Delta_i = U_i \Sigma_i V_i^\top.$$

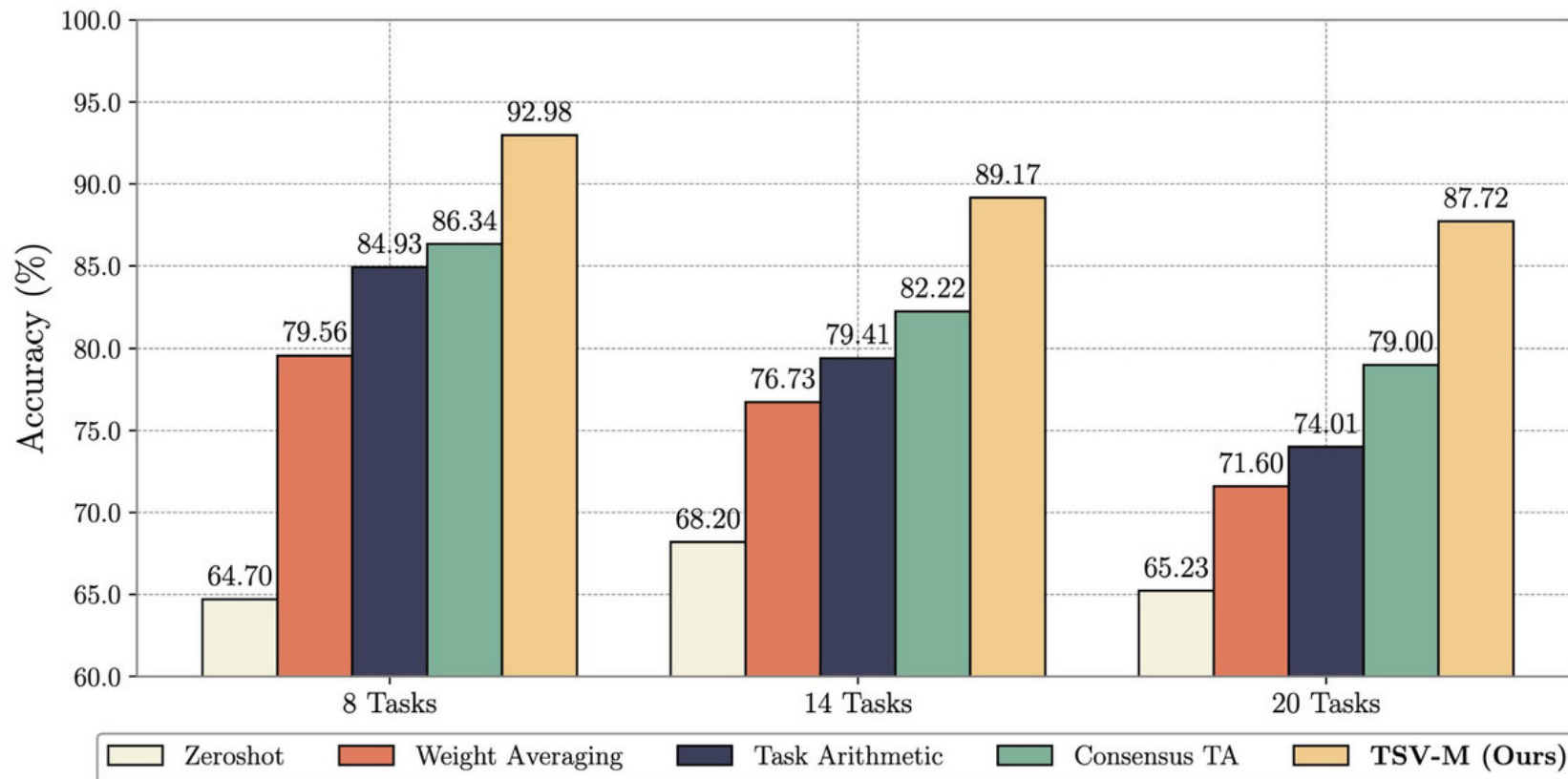
其中:

- U_i : 输出方向的奇异向量;
- V_i : 输入方向的奇异向量;
- Σ_i : 各方向的更新强度。

不同任务的更新矩阵可能在某些奇异方向上重叠或冲突。Task Singular Vector方法的核心是:

不直接逐元素合并 Δ_i , 而是在由奇异向量构成的低秩任务子空间中识别和降低干扰。

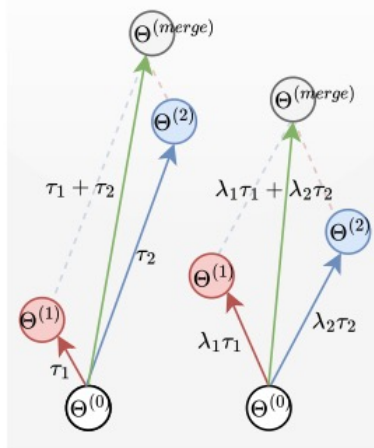
Task singular vectors improve performance



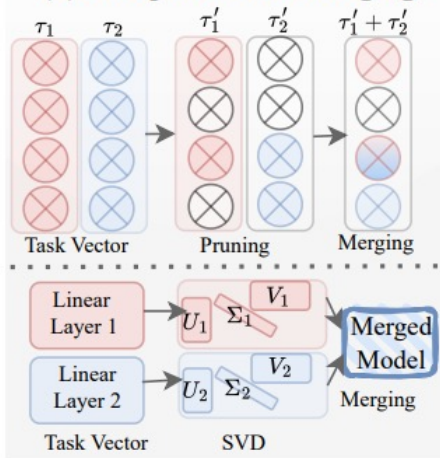
From “Task Singular Vectors: Reducing Task Interference in Model Merging” by Gargiulo et al.

Other ways of merging...

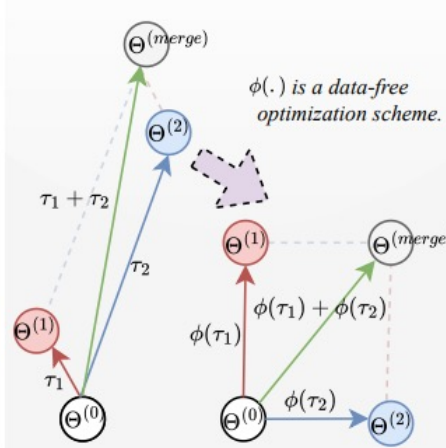
(a) Weighted-based Merging



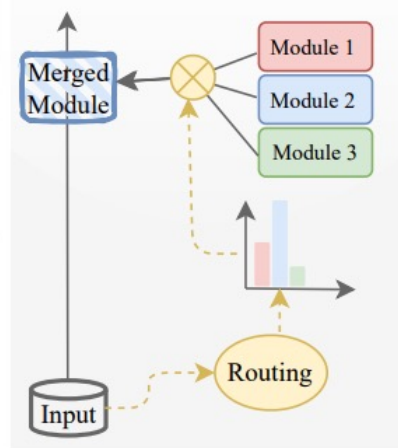
(b) Subspace-based Merging



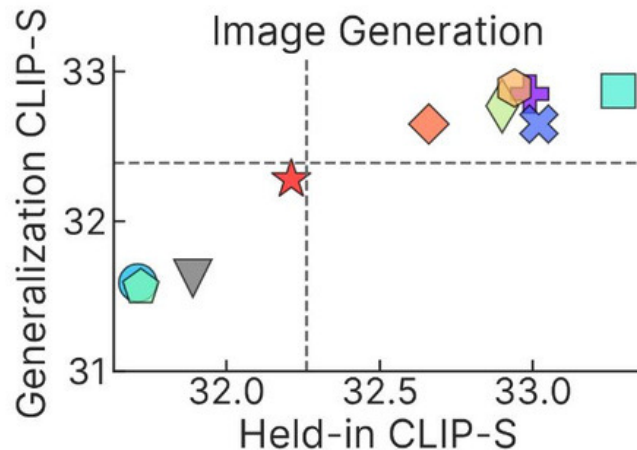
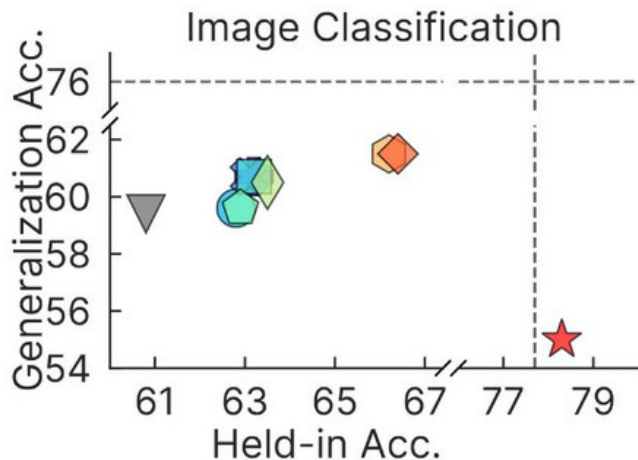
(c) Optimization-based Merging



(d) Routing-based Merging



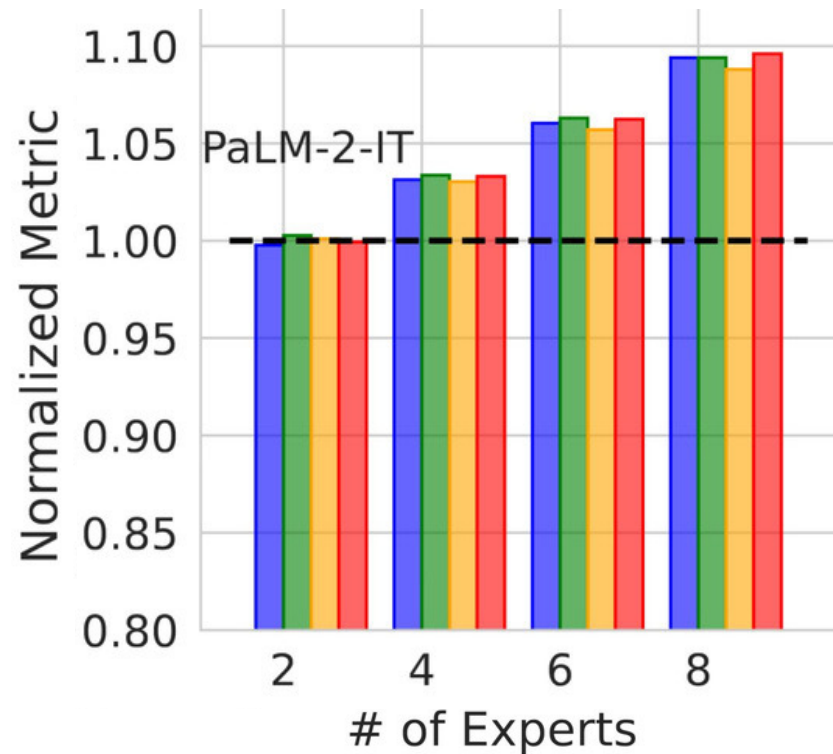
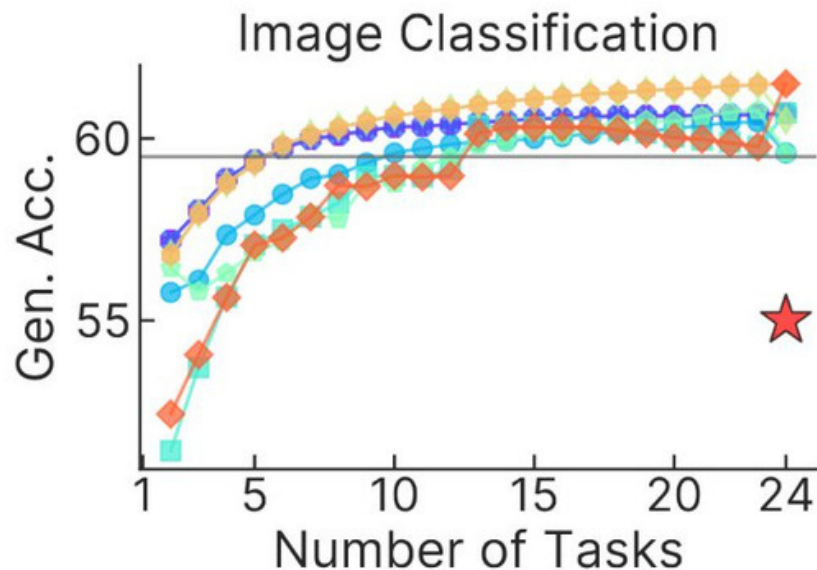
OK, but which merging method should I use?



1. Held-in(在参与合并的那些任务或能力上的性能)性能越高 $\Rightarrow$ 泛化性能通常也越高.

Merging methods have differing costs and requirements

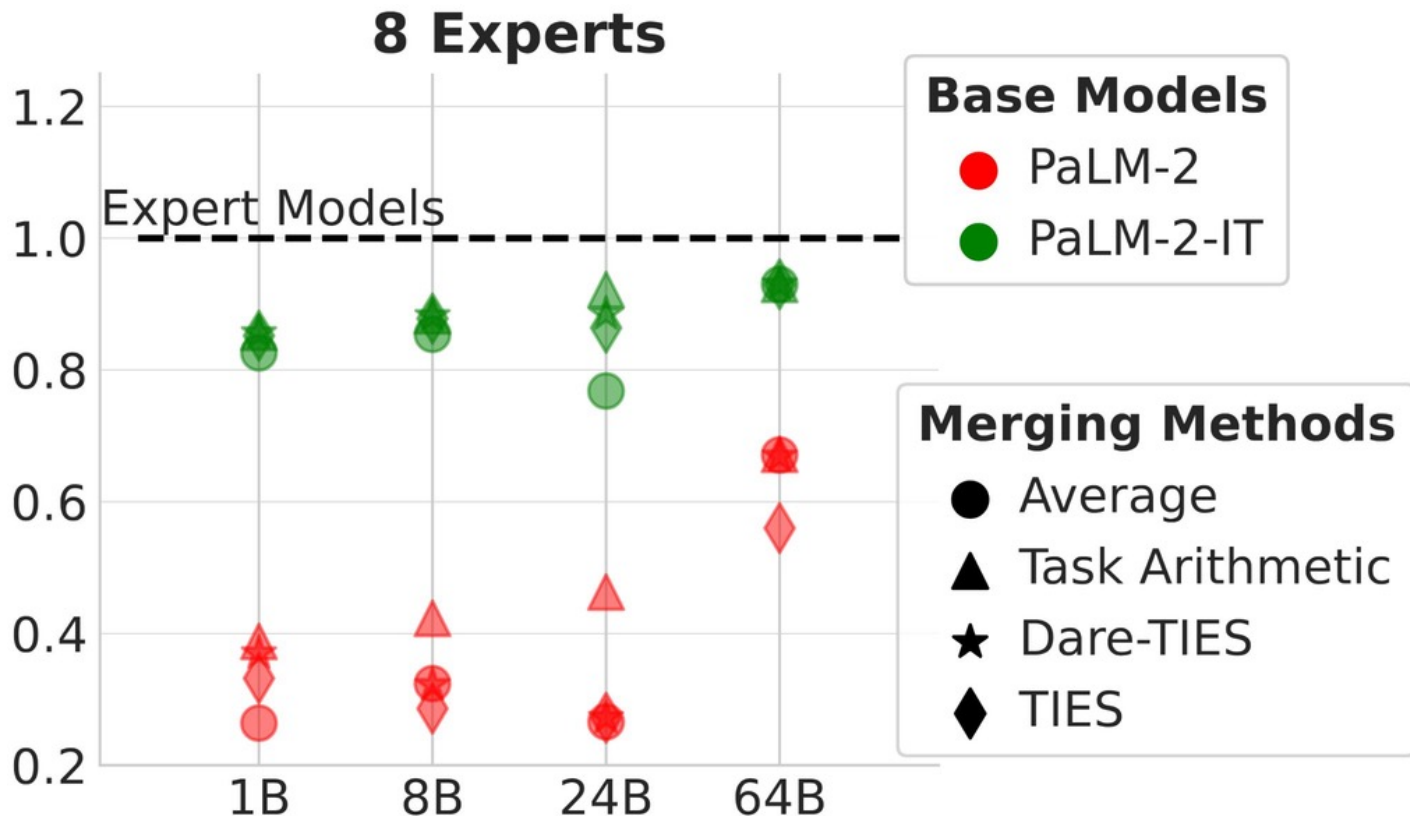
	Prerequisites			Hparams?	Computational cost (FLOPs)	
	θ_p	Stats	Data		Merging	Statistics
Average					Mdk	
SLERP					$\mathcal{O}((5M - 2)dk)$	
Task Arith.	$\times$		$\times$	$\checkmark$	$(2M + 1)dk$	
DARE	$\times$		$\times$	$\checkmark$	$(6M + 1)dk$	
TIES	$\times$		$\times^*$	$\checkmark^*$	$(4M + 1)dk$	$\mathcal{O}(MKdk)$
Fisher		$\times^*$	$\times^*$		$(3M - 1)dk$	$4MTd^2k$
RegMean		$\times^*$	$\times^*$	$\checkmark$	$\mathcal{O}((M + 2)d^2k)$	MTd^2k
MaTS		$\times^*$	$\times^*$	$\checkmark$	$\mathcal{O}((M + N)d^2k)$	$4MTd^2k$



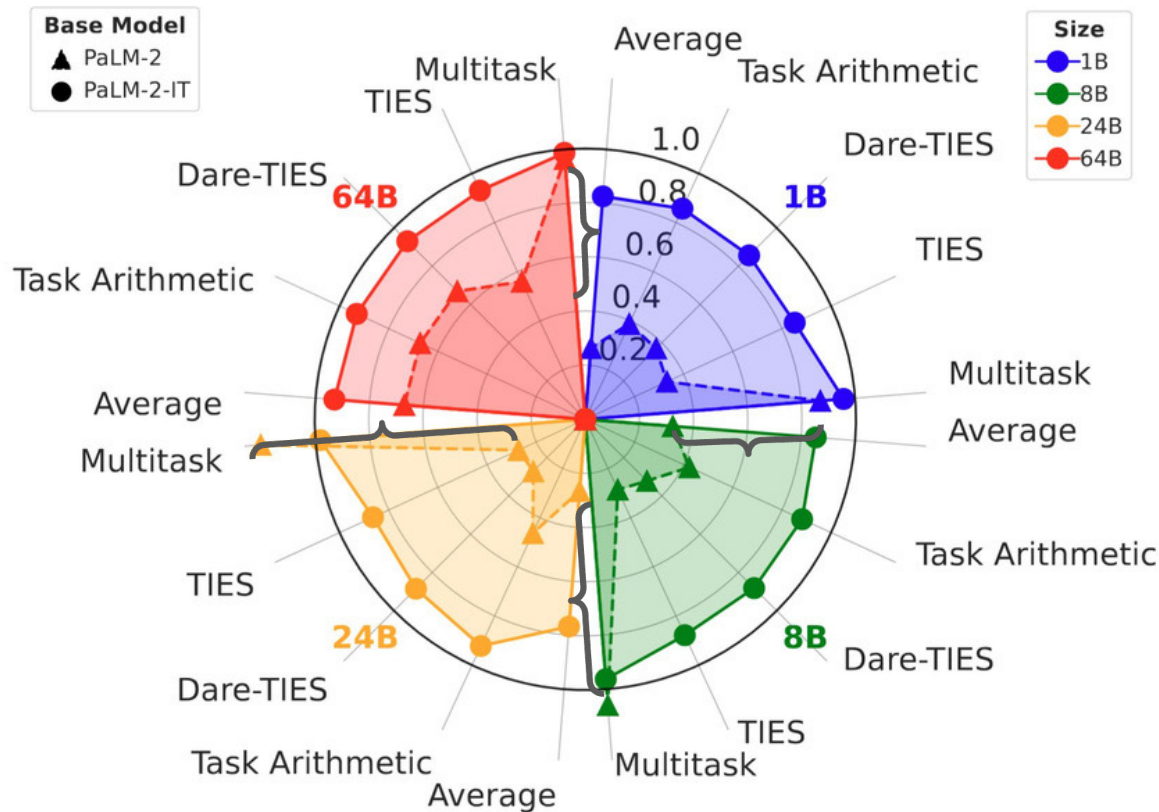
语言模型专家合并任务

2. 合并更多模型，泛化性通常提高，但存在边际收益递减

From "Realistic Evaluation of Model Merging for Compositional Generalization" by Tam et al.
and "What Matters for Model Merging at Scale?" by Yadav et al.



3. 基础模型越强，方法之间的差距越小

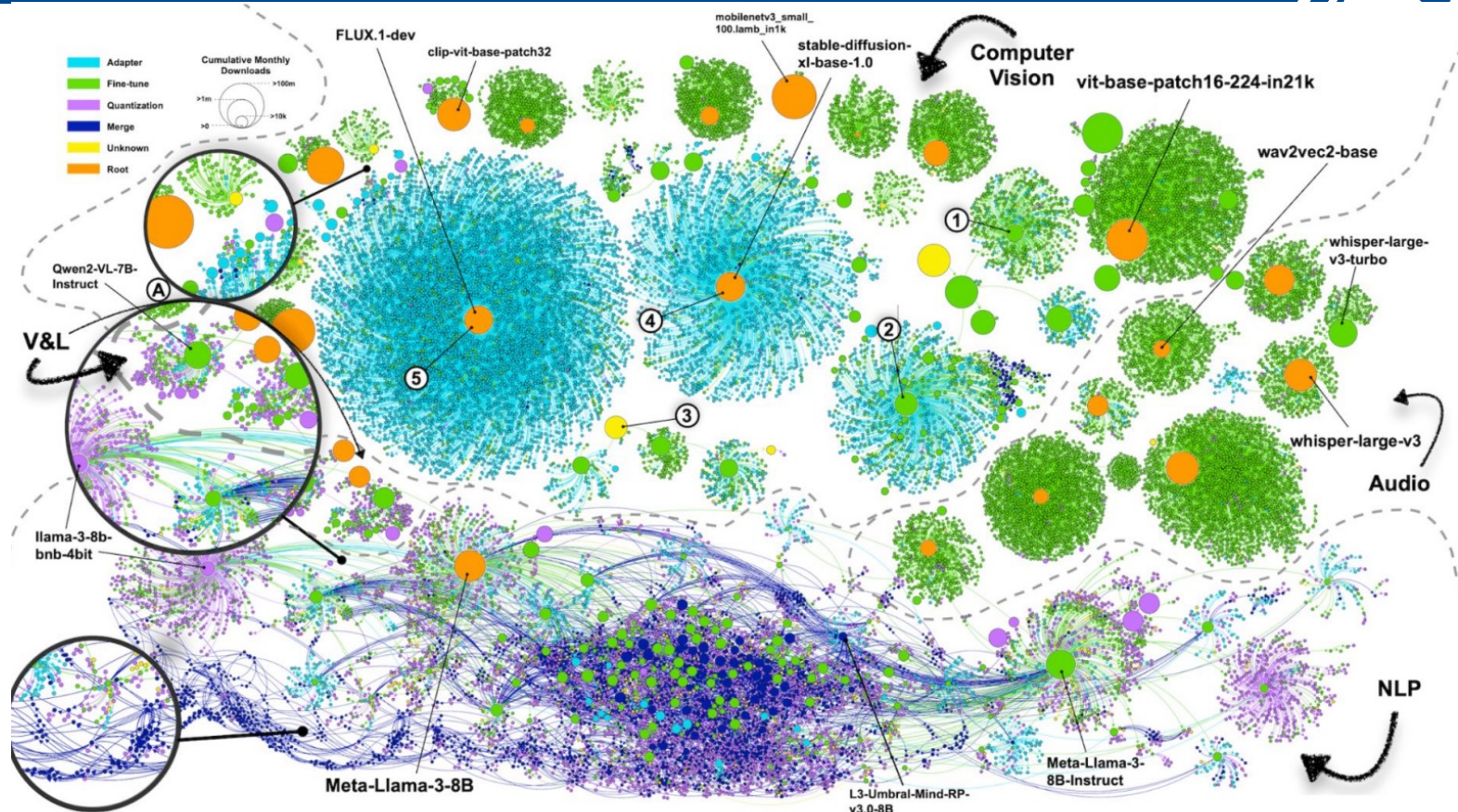


4. 模型越大，合并结果越接近多任务训练

From "What Matters for Model Merging at Scale?" by Yadav et al.



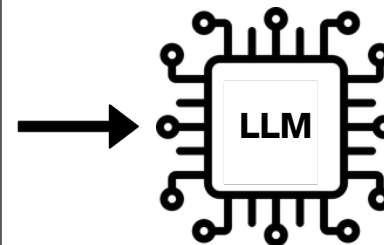
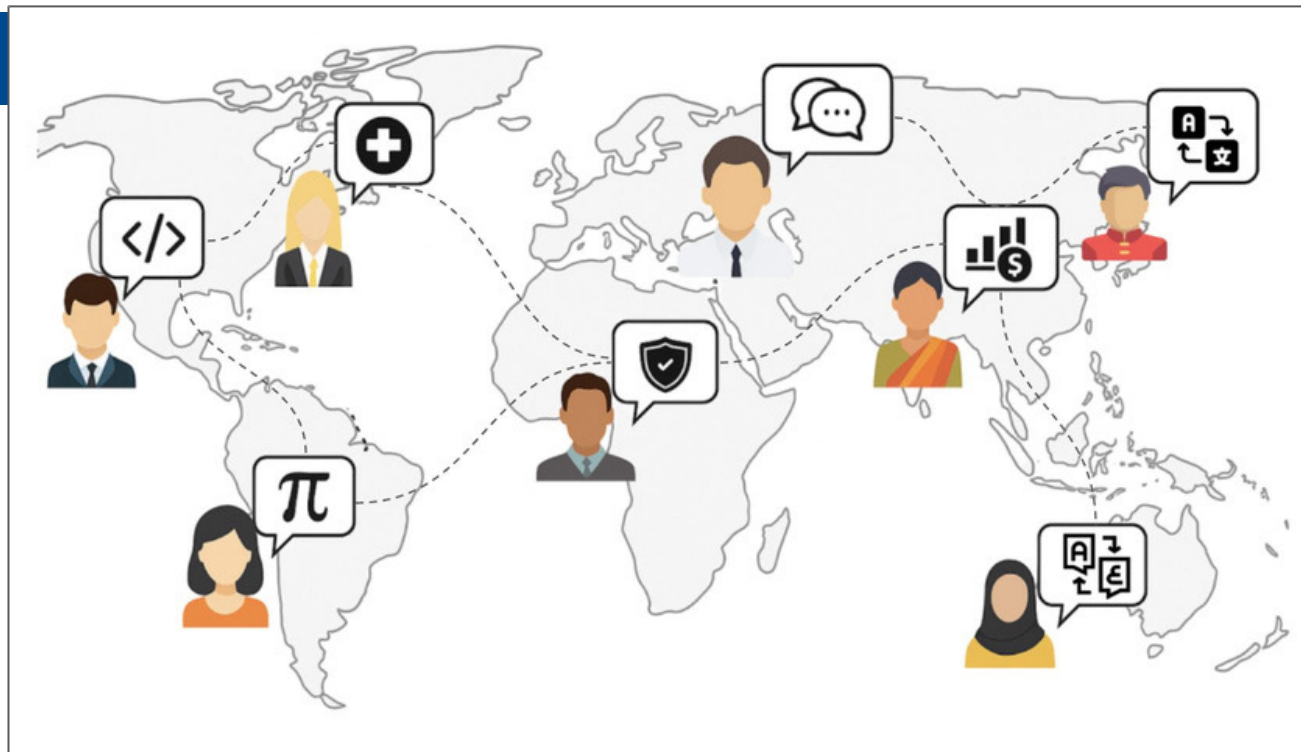
Applications



From "Charting and Navigating Hugging Face's Model Atlas" by Horwitz, Eliahu, et al.

But Why?

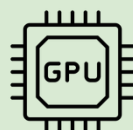
What applications are driving widespread adoption?



How?



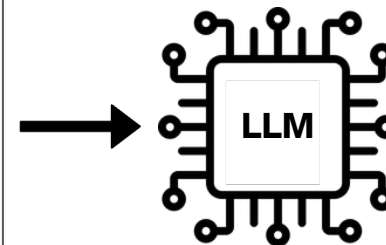
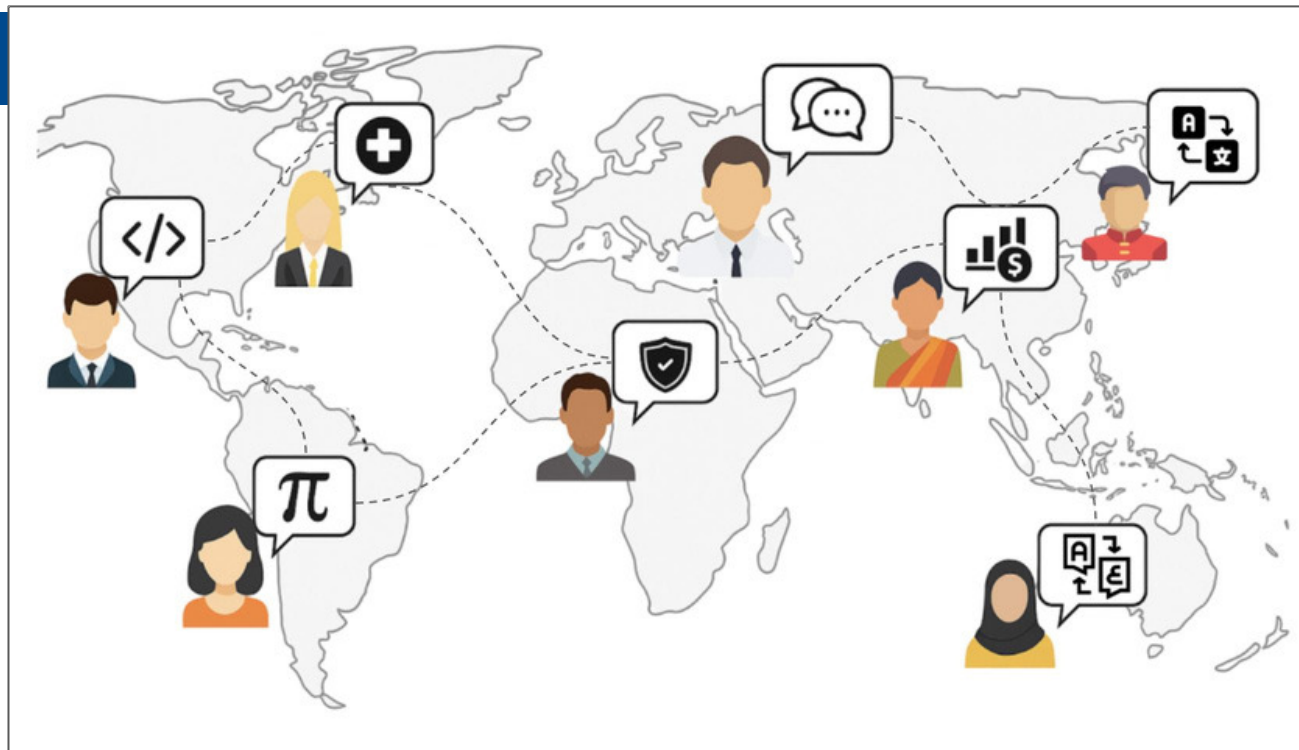
**Diverse
Datasets**



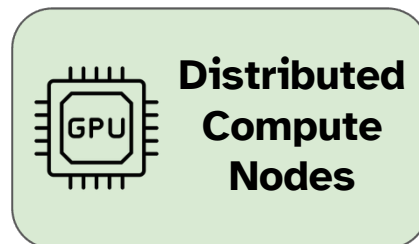
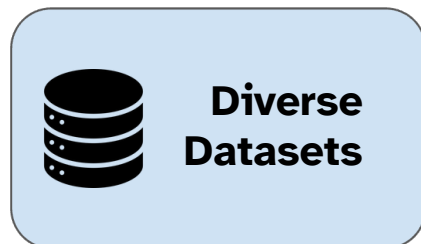
**Distributed
Compute
Nodes**



**Different
Time Zones**



Model Merging!



Model Merging Helps Overcome Core Challenges across the Model development Lifecycle.

Combining Model Capabilities

Multiple tasks, languages, and modalities to integrate

Decentralized Training

Distributed compute and human resources across organizations

Continual Model Development

Continuous training and evolution of models

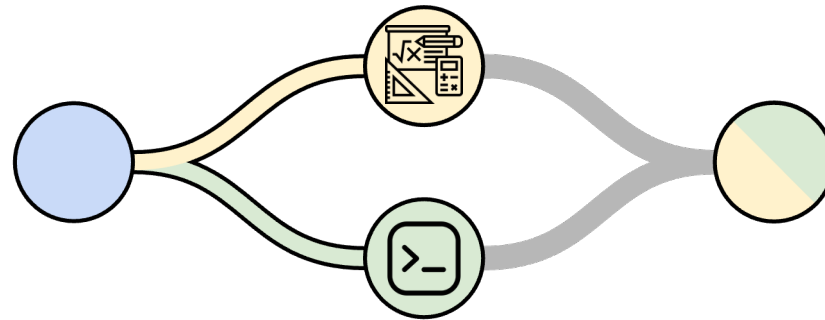
Pluralistic Alignment

Post-hoc multi-objective optimization for diverse stakeholders

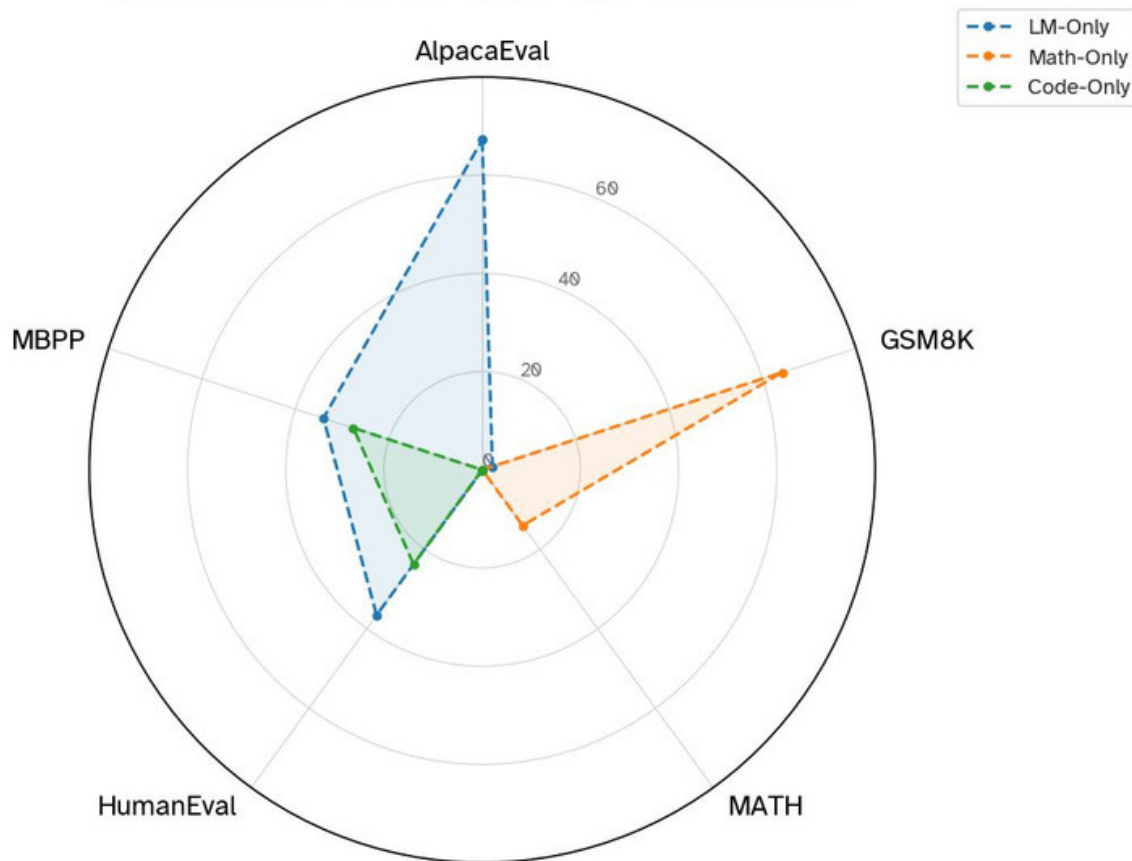
Generalization and Robustness

Ensuring robust, and compositionally generalizable behaviour in the real-world

Combining Model Capabilities

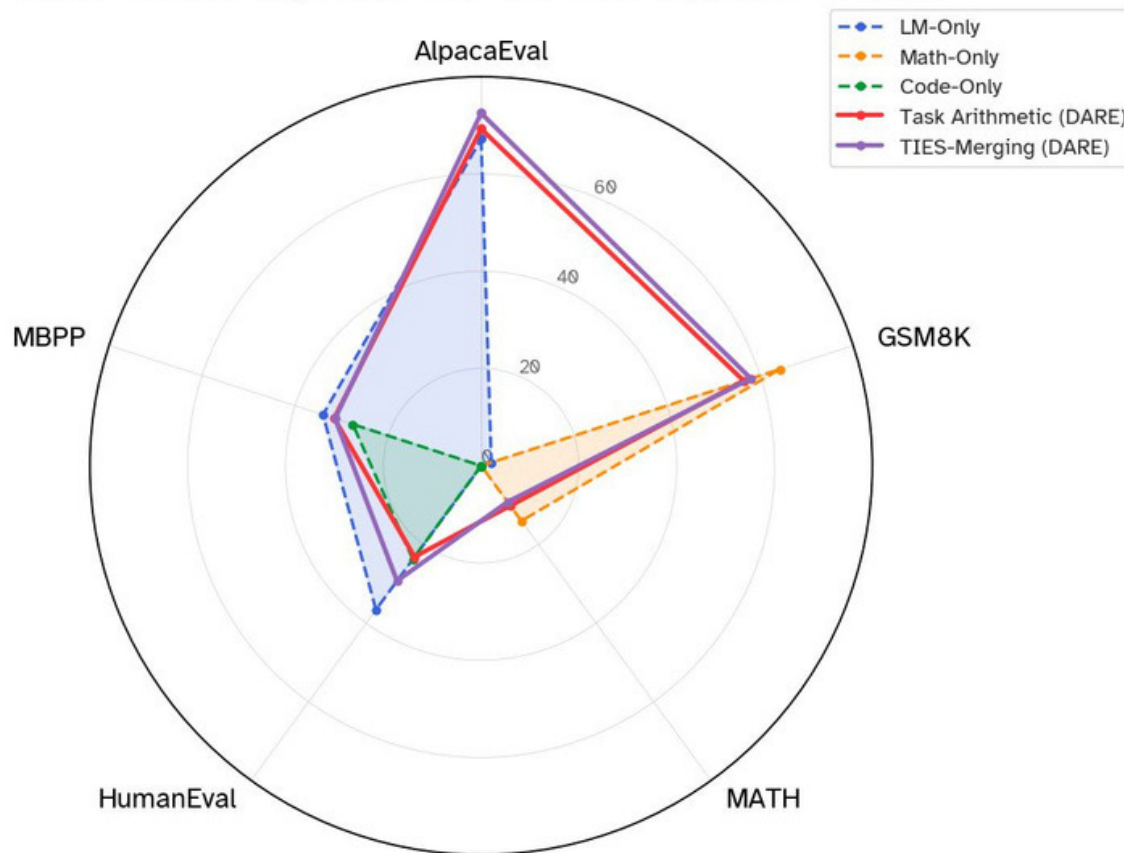


Combining Model Capabilities

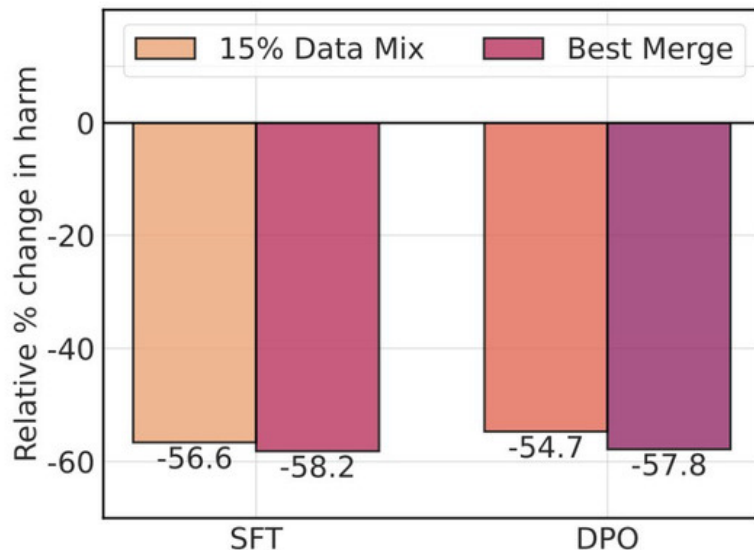


From "Language Models are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch" by Yu, Le, et al.

Combining Model Capabilities



Merging Methods	Models	Use DARE	AlpacaEval	GSM8K	MATH	HumanEval	MBPP
/	LM	No	67.20	2.20	0.04	36.59	34.00
	Math	No	/	64.22	14.02	/	/
	Code	No	/	/	/	23.78	27.60
Task Arithmetic	LM & Math & Code	No	69.03	58.45	9.88	18.29	29.80
	LM & Math & Code	Yes	69.28	56.48	10.16	23.17	31.60
TIES-Merging	LM & Math & Code	No	65.91	62.55	9.54	21.95	30.40
	LM & Math & Code	Yes	72.50	58.00	9.20	29.27	31.40



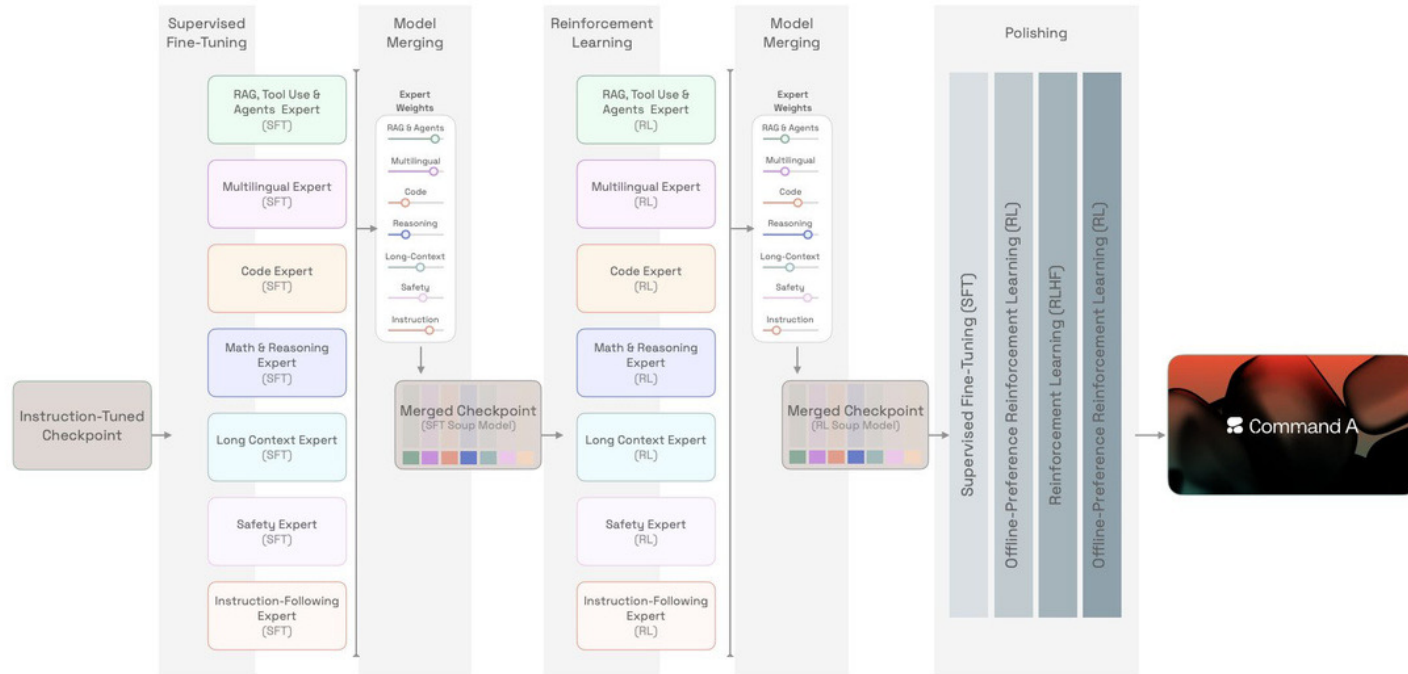
Safety Performance



General Performance

Combining Model Capabilities

Command A is a successful real-world example of multitask and multilingual learning via model merging.

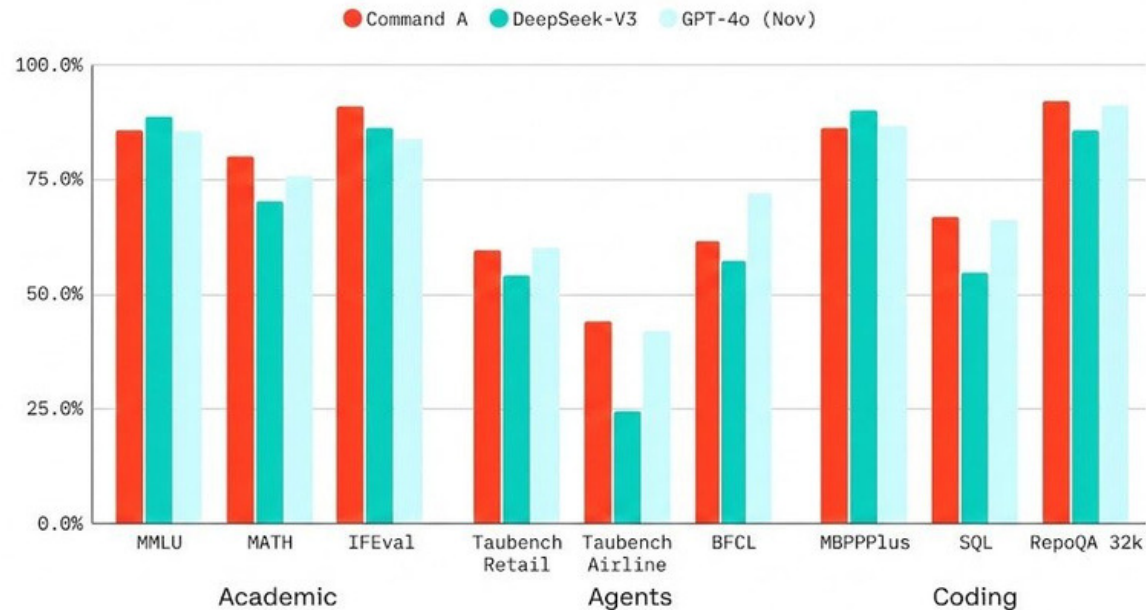


Combining Model Capabilities



Command A is on par or better than GPT-4o and DeepSeek-V3 across agentic enterprise tasks with expanded support for 23 languages

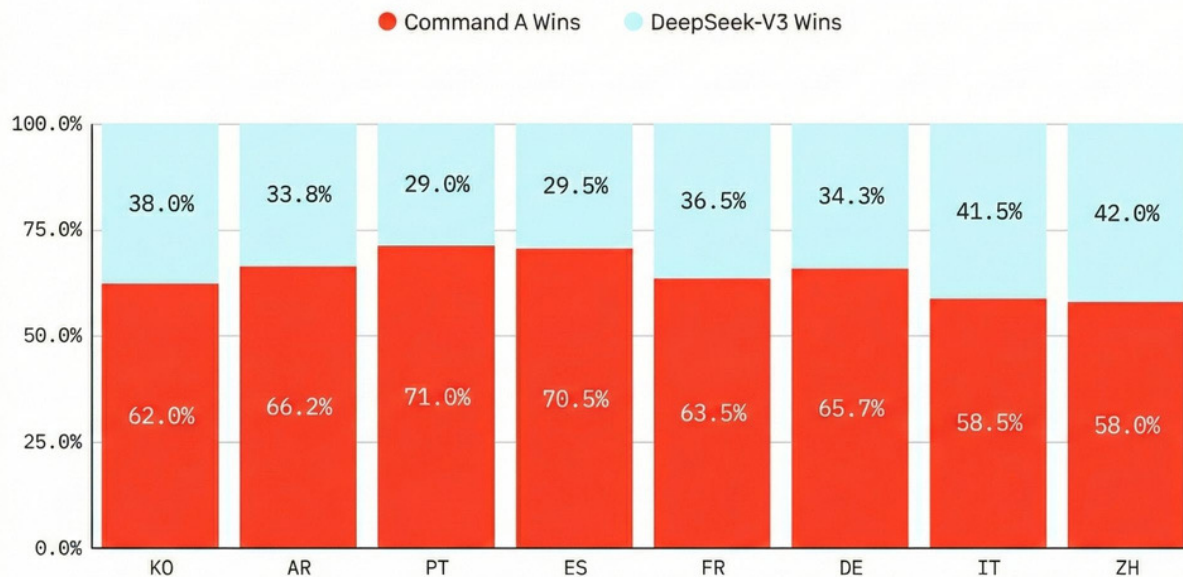
Benchmarks



Combining Model Capabilities

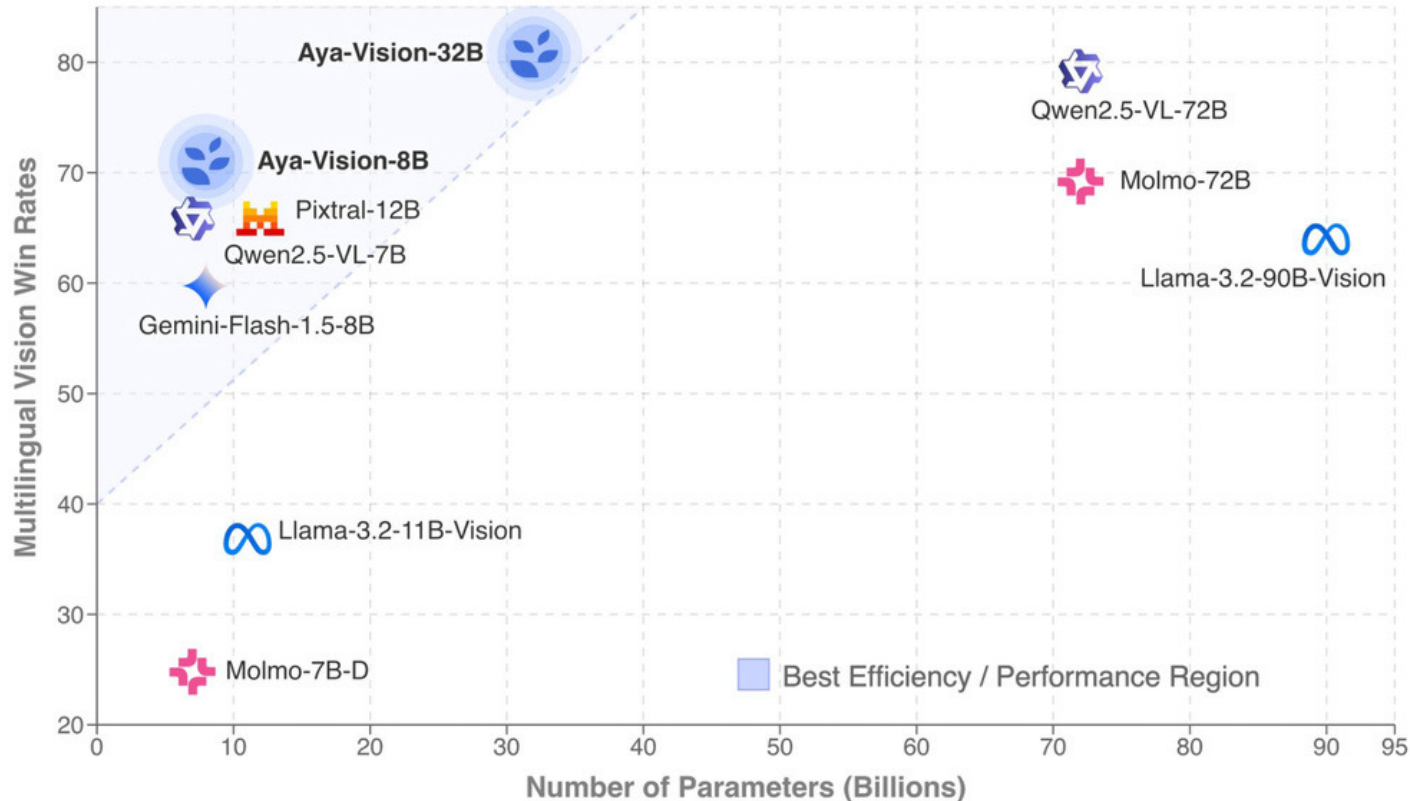
Command A is on par or better than GPT-4o and DeepSeek-V3 across agentic enterprise tasks with expanded support for 23 languages

Multilingual Human Evaluations



Combining Model Capabilities

Aya Vision is a successful example of multimodal learning via model merging



From "Aya Vision: Advancing the Frontier of Multilingual Multimodality" by Dash, Saurabh, et al.

Model Merging Helps Overcome Core Challenges across the Model development Lifecycle.

Combining Model Capabilities

Multiple tasks, languages, and modalities to integrate

Decentralized Training

Distributed compute and human resources across organizations

Continual Model Development

Continuous training training and evolution of models

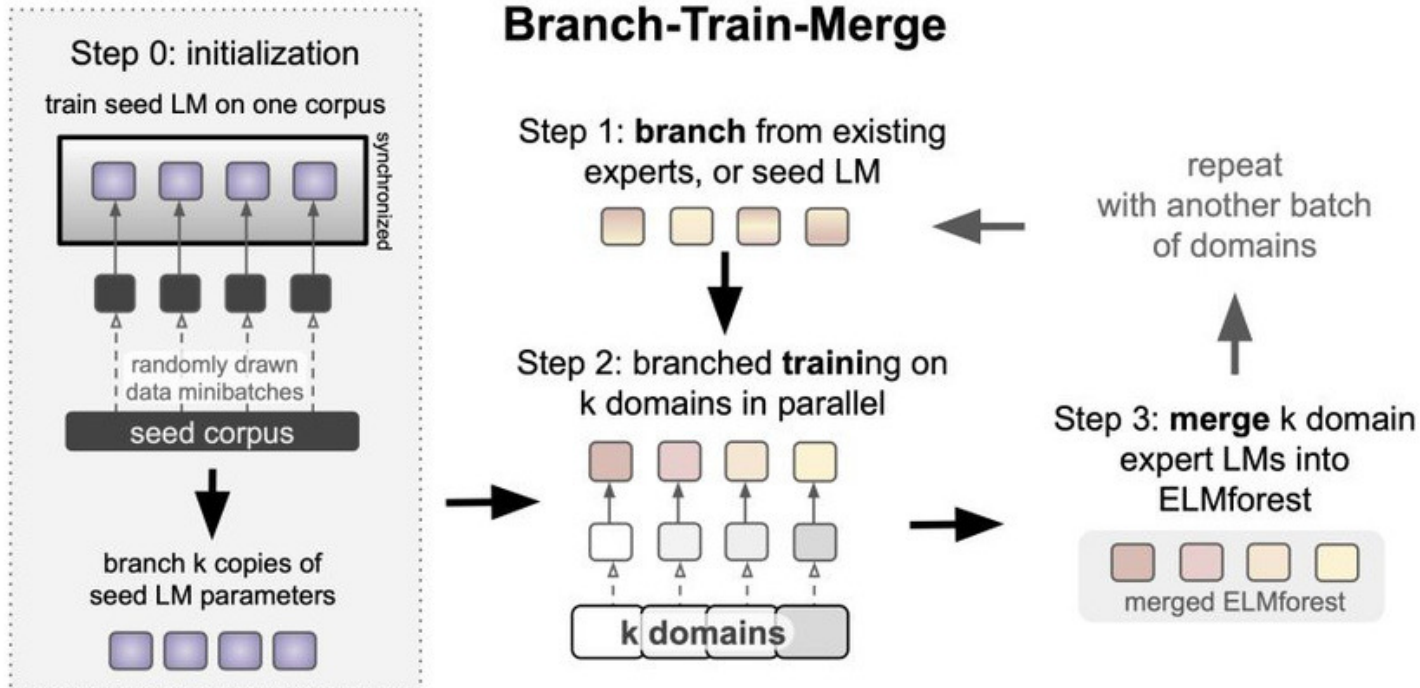
Pluralistic Alignment

Post-hoc multi-objective optimization for diverse stakeholders

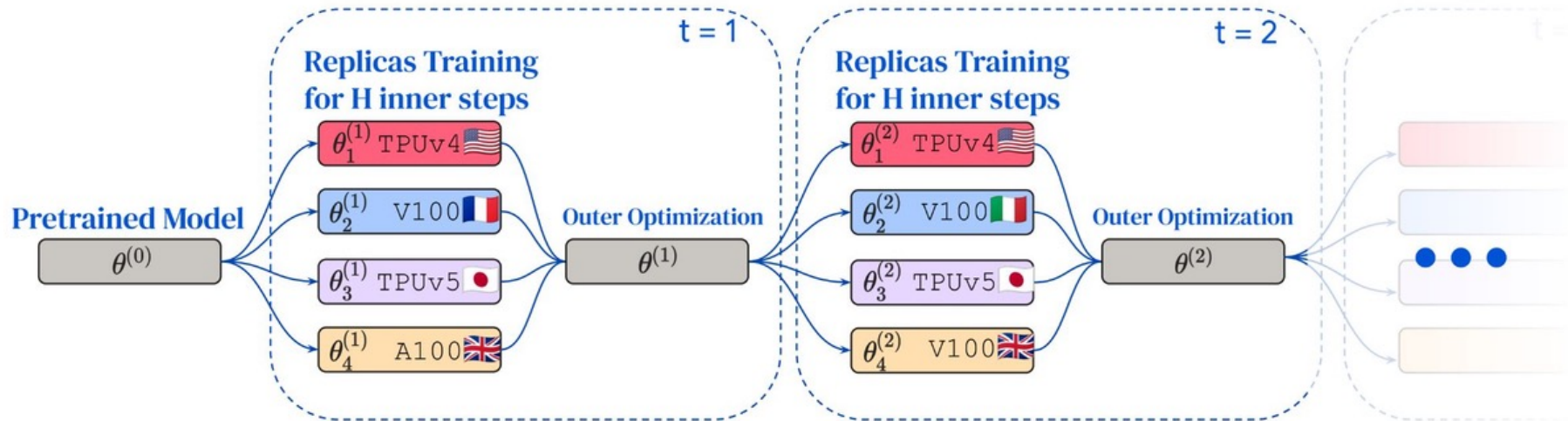
Generalization and Robustness

Ensuring robust, and compositionally generalizable behaviour in the real-world

Merging enables massive parallel training of specialized models



Periodic merging allows decentralized training with low-communication



INTELLECT-1 is a successful real-world example of decentralized trained model using DiLoCo



INTELLECT-1: The First Decentralized Trained 10B Model, trained on 1 trillion tokens using up to 14 concurrent nodes distributed across 3 continents, with contributions from 30 independent compute providers.



Model Merging Helps Overcome Core Challenges across the Model development Lifecycle.

Combining Model Capabilities

Multiple tasks, languages, and modalities to integrate

Decentralized Training

Distributed compute and human resources across organizations

Continual Model Development

Continuous training and evolution of models

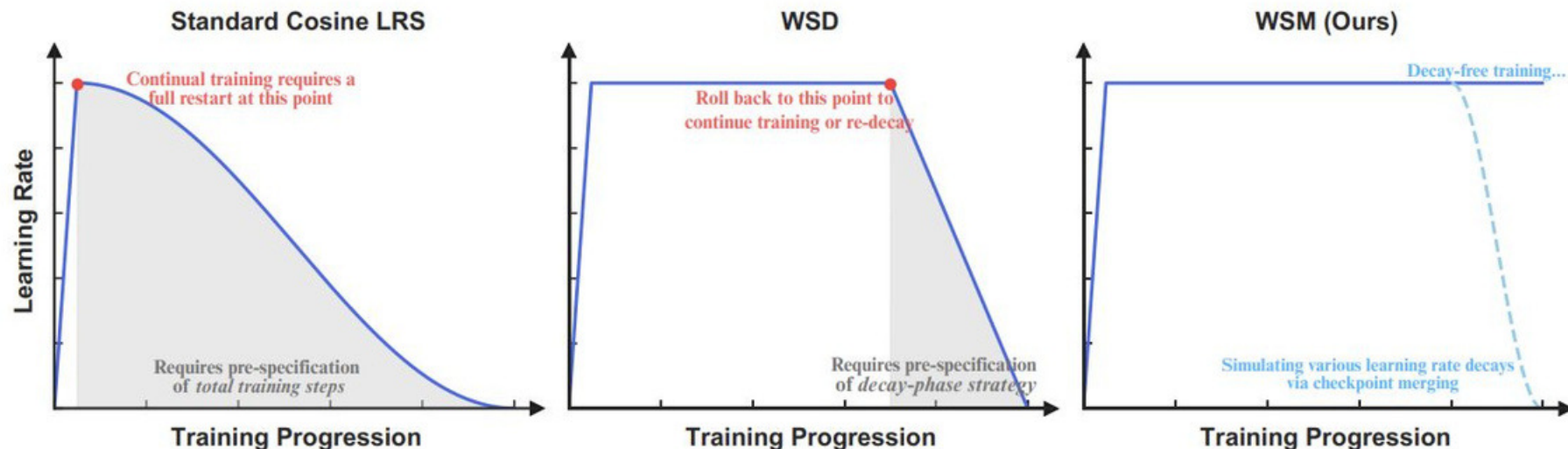
Pluralistic Alignment

Post-hoc multi-objective optimization for diverse stakeholders

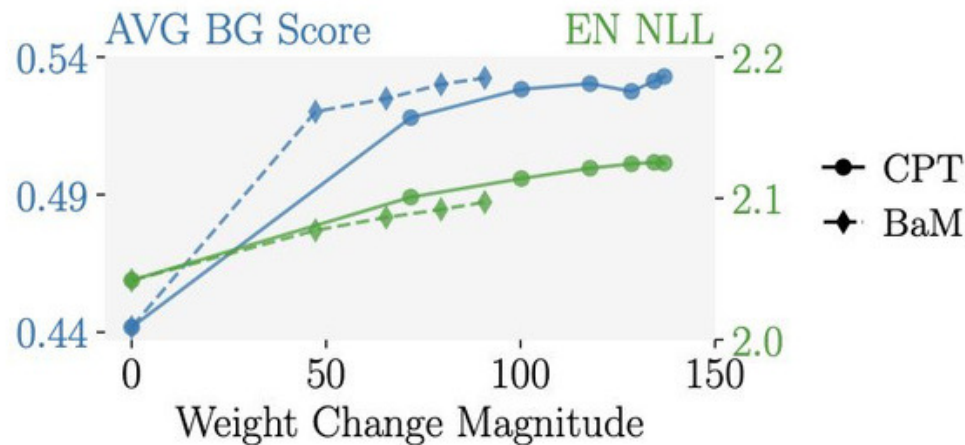
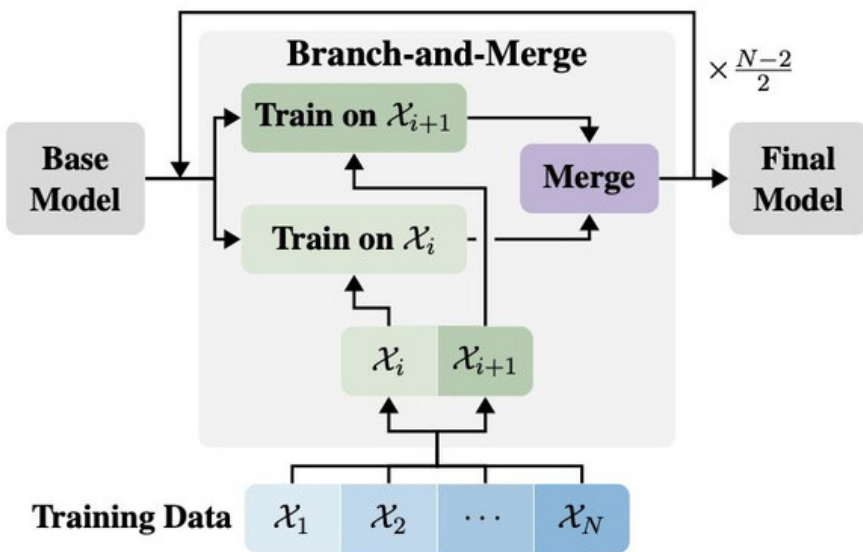
Generalization and Robustness

Ensuring robust, and compositionally generalizable behaviour in the real-world

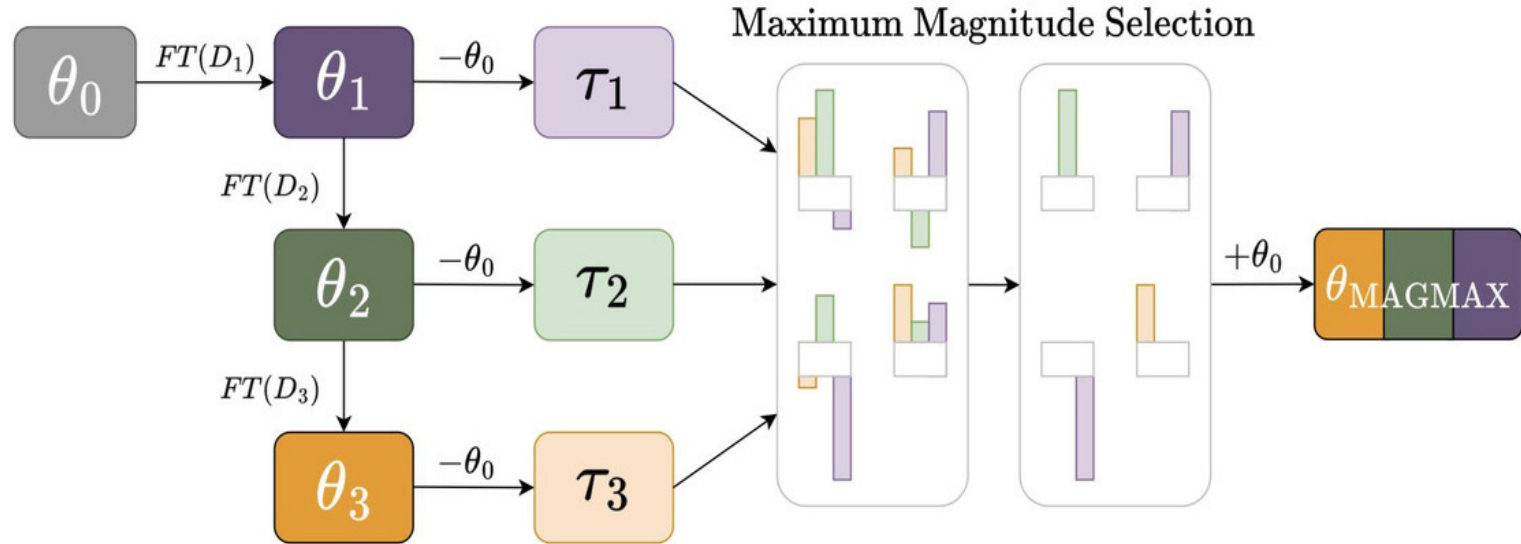
Model merging enables a decay-free LR schedule by checkpoint merging for LLM pretraining



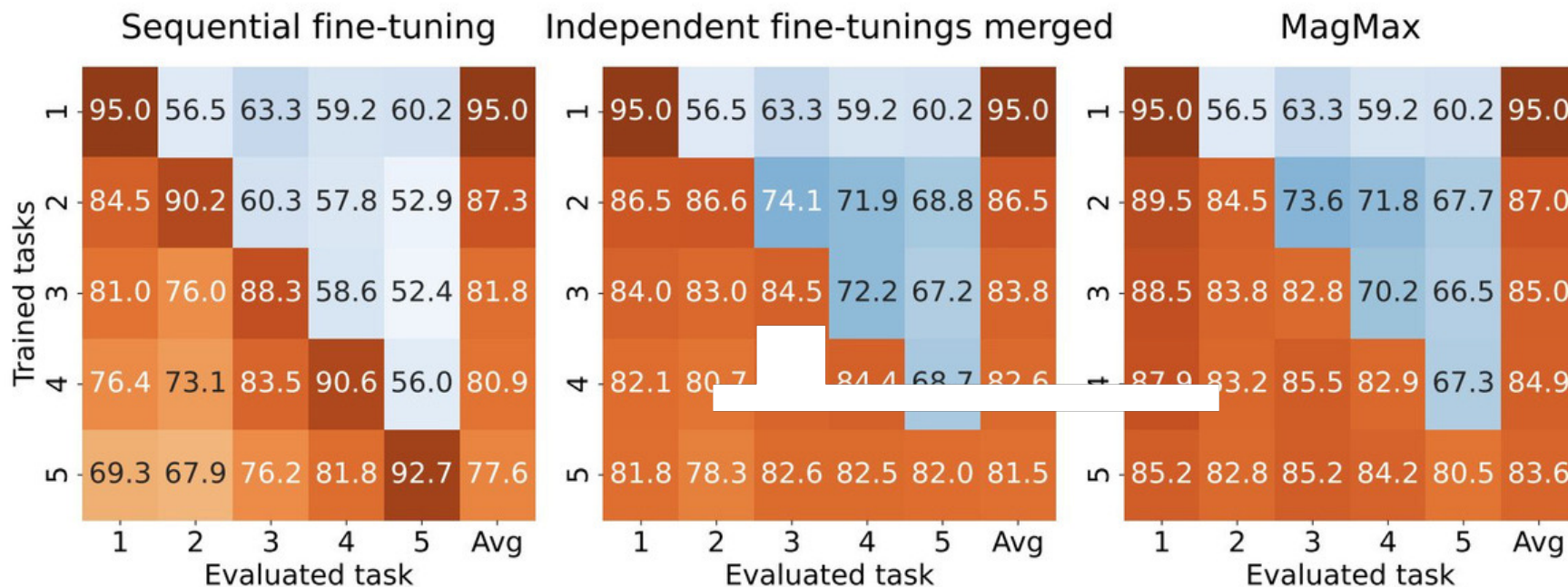
Branch-and-merge helps mitigate catastrophic forgetting in language transfer



Task-vector merging using maximum-magnitude selection helps consolidate knowledge and mitigate forgetting



Task-vector merging using maximum-magnitude selection helps consolidate knowledge and mitigate forgetting



Model Merging Helps Overcome Core Challenges across the Model development Lifecycle.

Combining Model Capabilities

Multiple tasks, languages, and modalities to integrate

Decentralized Training

Distributed compute and human resources across organizations

Continual Model Development

Continuous training and evolution of models

Pluralistic Alignment

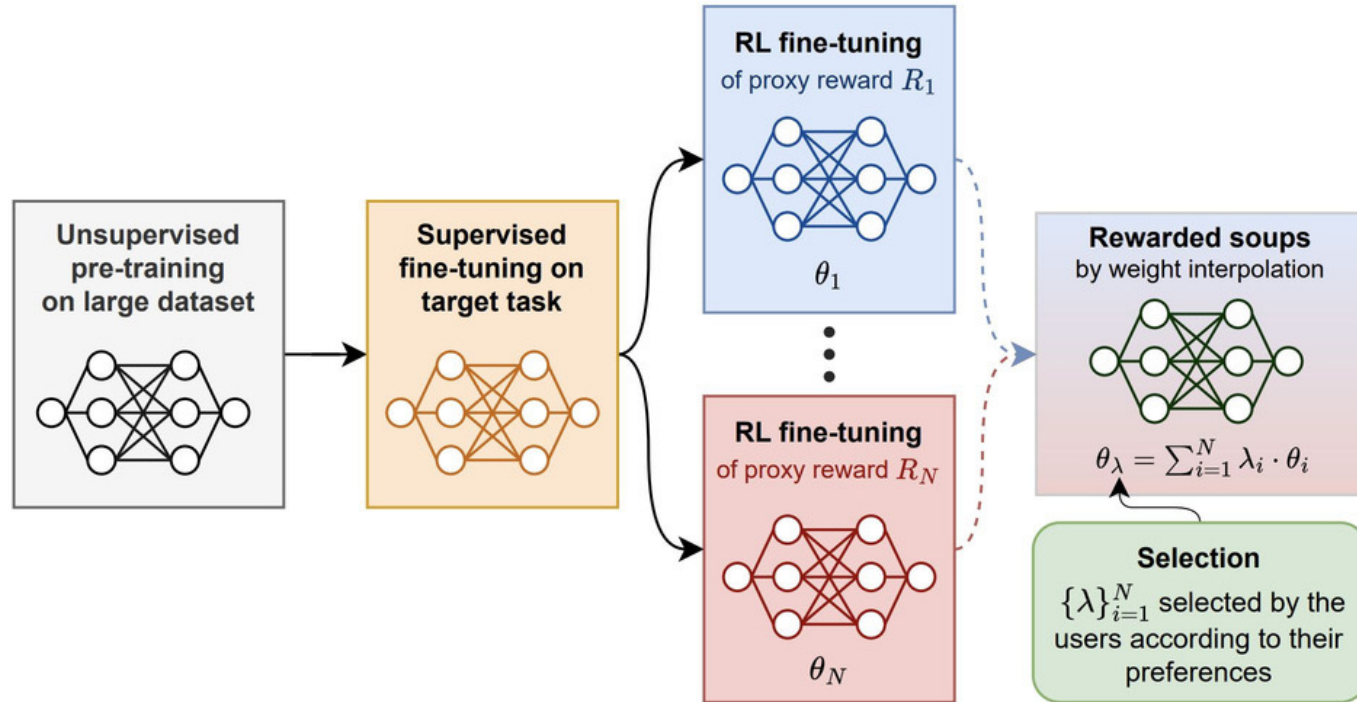
Post-hoc multi-objective optimization for diverse stakeholders

Generalization and Robustness

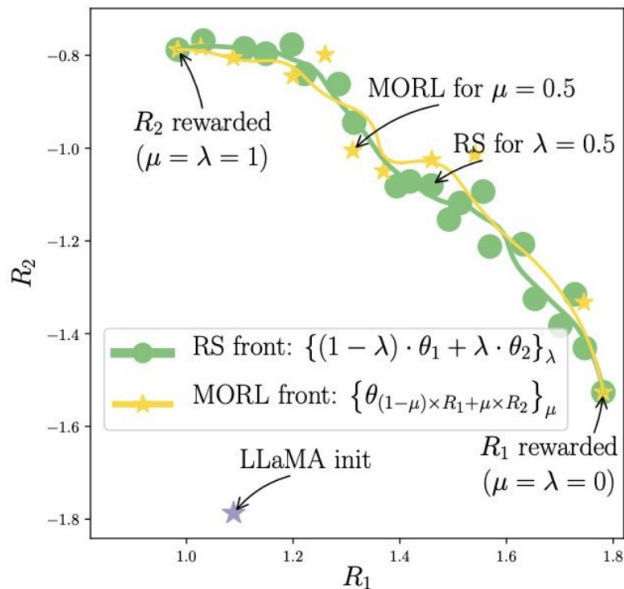
Ensuring robust, and compositionally generalizable behaviour in the real-world

Pluralistic Alignment

Rewarded Soups uses weight interpolation to resolve diverse and conflicting rewards, yielding Pareto-optimal alignment



Rewarded Soups can be a cost-efficient alternative of Multi-Objective Reinforcement Learning (MORL)



LLaMA RLHF for News Summarization

* RS matches the costly yellow front of multi-objective (MORL)

Feature	MORL	RS
Timing of Combination	A Priori: Objectives are combined before training	A Posteriori: Policies (weights) are combined after training
What is Interpolated	Rewards: Linearly interpolates the proxy rewards to create a single objective: $\Sigma \mu_i R_i$	Weights: Linearly interpolates the expert network weights: $\Sigma \lambda_i \theta_i$
Efficiency/Cost	Requires $M \gg N$ trainings (where M is the number of preference weightings) to cover the Pareto front	Requires only N trainings (one for each proxy reward)
Scalability	Unscalable in deep learning due to high computational, memory, and engineering costs.	Highly scalable and efficient.

Model Merging Helps Overcome Core Challenges across the Model development Lifecycle.

Combining Model Capabilities

Multiple tasks, languages, and modalities to integrate

Decentralized Training

Distributed compute and human resources across organizations

Continual Model Development

Continuous training and evolution of models

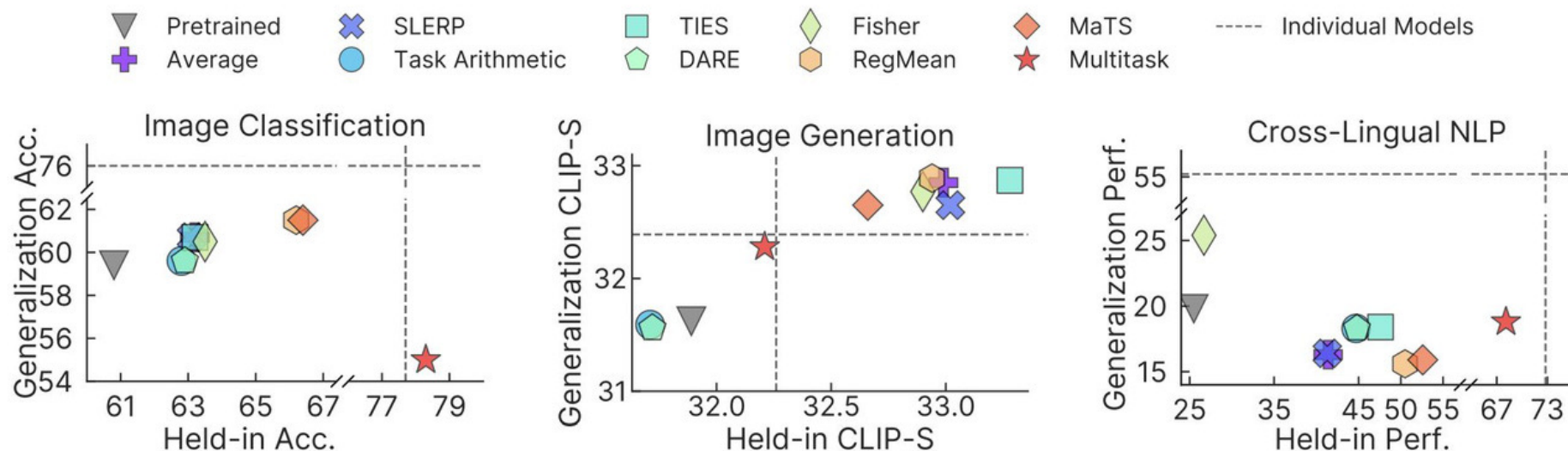
Pluralistic Alignment

Post-hoc multi-objective optimization for diverse stakeholders

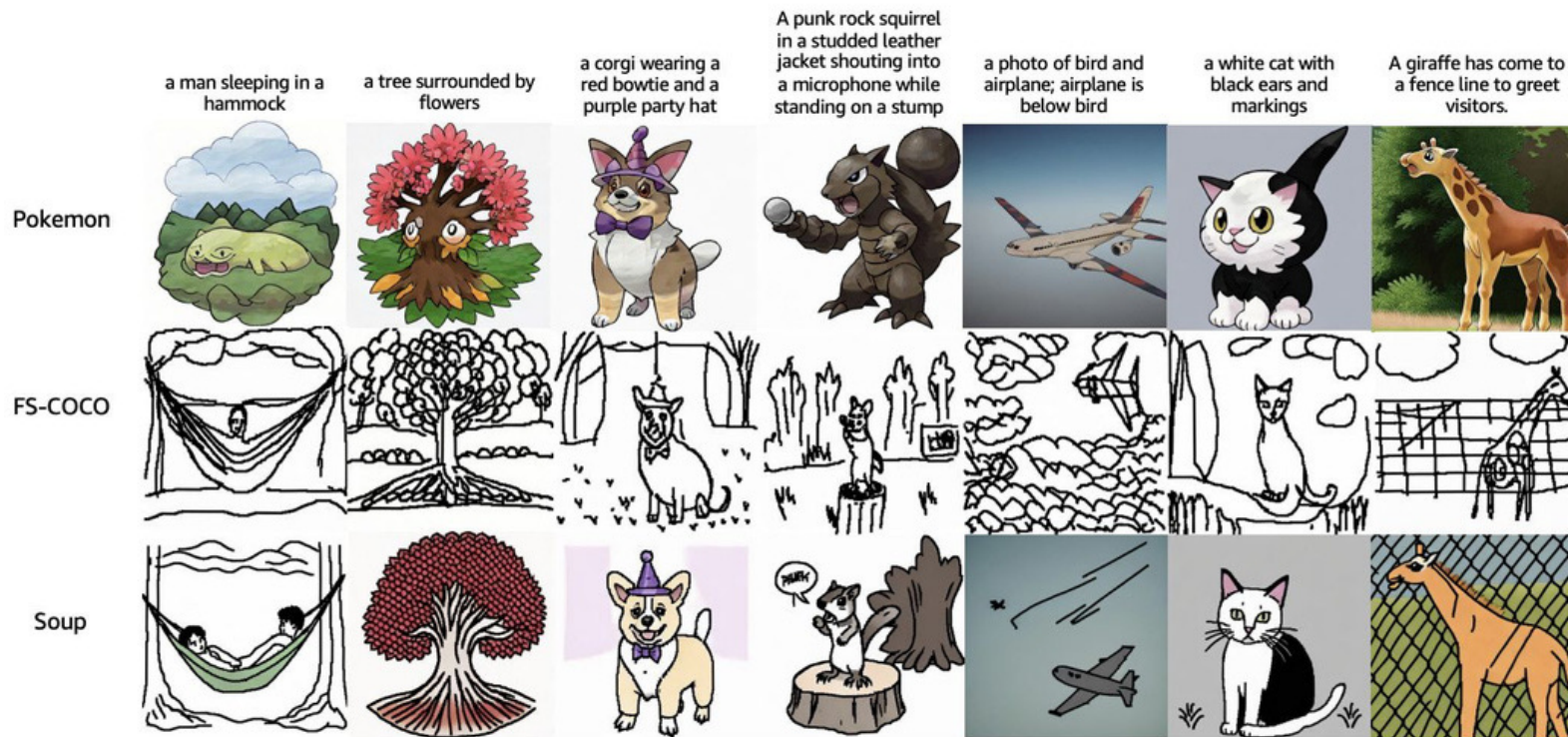
Generalization and Robustness

Ensuring robust, and compositionally generalizable behaviour in the real-world

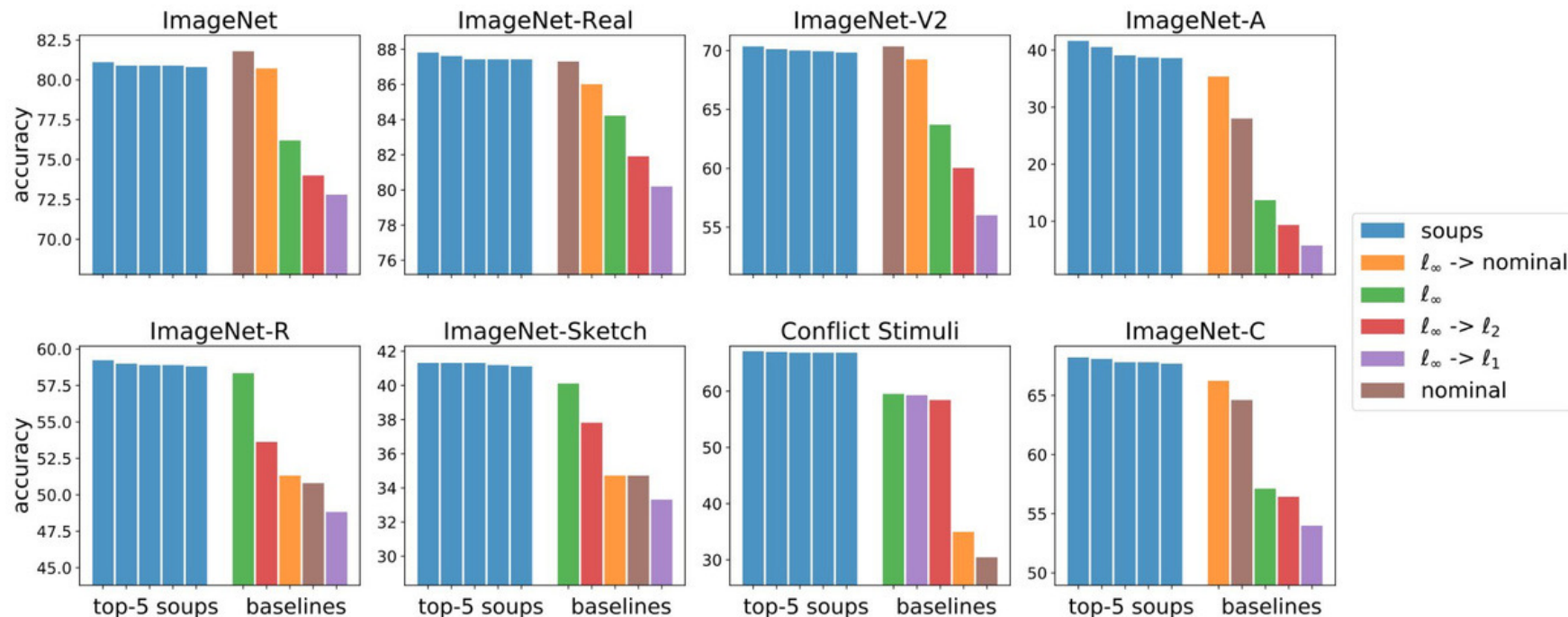
Model merging can help models generalize compositionally, with effectiveness varying by domain



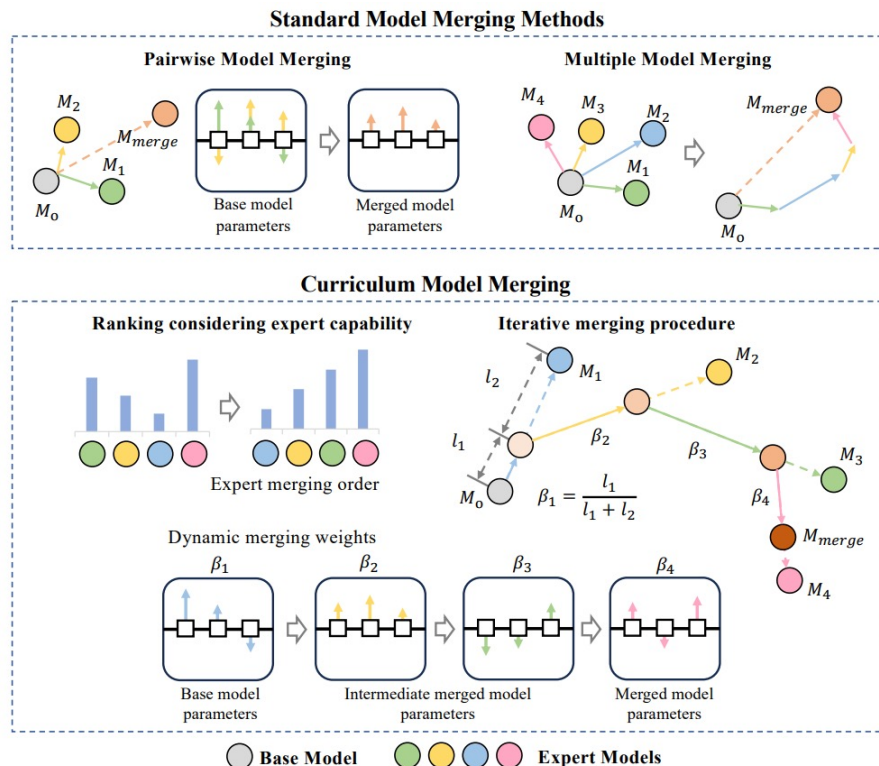
Model merging can blend disparate artistic styles into novel hybrid outputs



Model merging unifies robustness to diverse adversarial attacks in one model



• Curriculum Model Merging in Chemical LLMs



通用化学基座模型 $\rightarrow$ $\left\{ \begin{array}{l} \text{性质预测专家} \\ \text{分子生成专家} \\ \text{逆合成专家} \\ \text{化学推理专家} \end{array} \right\} \rightarrow \text{统一化学模型}$

- *Layer wise merging*

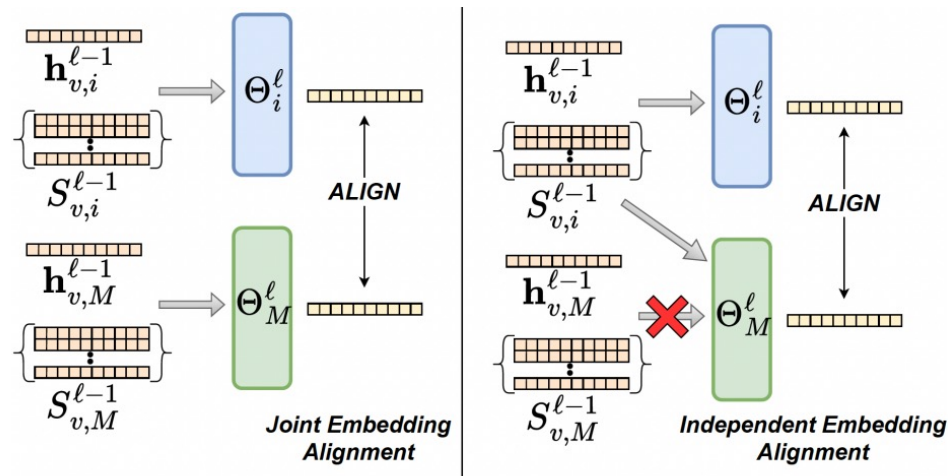
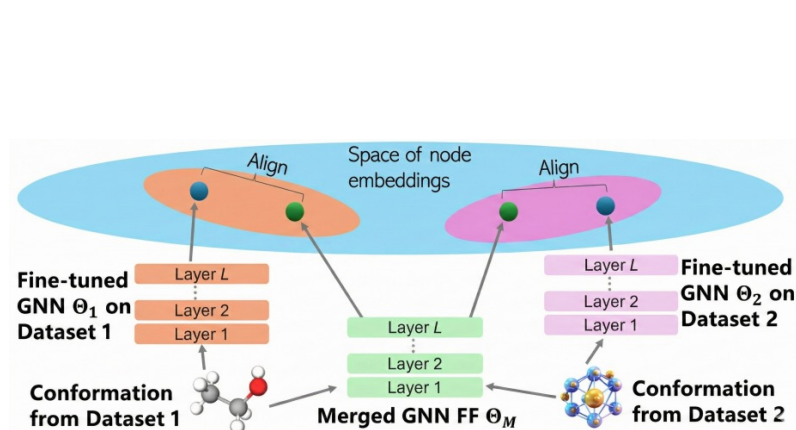


Figure 2. Illustrates the idea of independent layer-wise merging.

• Layer wise merging

2. 不直接对齐权重，而是对齐原子嵌入

GFFMerge的基本观点是：

两个专家的参数数值不一定要相似，但它们在各自数据上产生的原子表征应当被保留下来。

设第 k 个专家在第 ℓ 层对原子 v 产生中间表示：

$$h_{v,k}^{(\ell)}$$

理想的合并模型应该满足：

$$h_{v,M}^{(\ell)} \approx h_{v,k}^{(\ell)}, \quad G \in \mathcal{D}_k.$$

即：

- 输入取Aspirin结构时，合并模型模仿Aspirin专家；
- 输入Uracil结构时，合并模型模仿Uracil专家。

因此，完整目标可以理解为：

$$\min_{\theta_M} \sum_k \sum_{G \in \mathcal{D}_k} \sum_{\ell=1}^L \sum_{v \in V(G)} \|h_{v,M}^{(\ell)}(G) - h_{v,k}^{(\ell)}(G)\|_2^2.$$

论文称其为 **node embedding alignment**. [arXiv](#)

但直接优化这个目标仍需完整反向传播，因为第 ℓ 层合并表示依赖前面所有合并层。

3. 关键近似：逐层独立合并

GFFMerge最重要的技术处理，是将端到端表征对齐改成**逐层独立表征对齐**。

原本第 ℓ 个合并层的输入应该来自合并模型自己：

$$h_M^{(\ell-1)} \rightarrow W_M^{(\ell)} \rightarrow h_M^{(\ell)}.$$

GFFMerge改为使用对应专家模型的上一层表征：

$$h_k^{(\ell-1)} \rightarrow W_M^{(\ell)} \rightarrow \hat{h}_{M,k}^{(\ell)},$$

并要求：

$$\hat{h}_{M,k}^{(\ell)} \approx h_k^{(\ell)}.$$

也就是说，在求第 ℓ 层合并参数时：

- 输入取自源专家；
- 目标输出也取自源专家；
- 只求一个共享的 $W_M^{(\ell)}$ 。

这样各层就可以分别求解，不需要对整个网络反向传播。 [arXiv](#)

这一近似类似于教师强制：闭式求解阶段假设每一层都能接收到“正确的专家输入”，暂时忽略部署时由前面合并层产生的误差累积。

4. 闭式解从哪里来？

考虑某个消息传递层中的线性参数块：

$$y = Wx.$$

第 k 个专家的参数是 W_k ，其期望输出为：

$$y_k = W_k x_k.$$

GFFMerge求一个共享矩阵 W ，使它在每个专家自己的激活分布上模仿该专家：

$$W^* = \arg \min_W \sum_{k=1}^K \sum_{G \in \mathcal{S}_k} \|W x_k(G) - W_k x_k(G)\|_2^2$$

其中 $\mathcal{S}_k$ 是第 k 个领域中的小规模校准样本集。 [arXiv](#)

定义未中心化激活协方差：

$$C_k = \sum_{G \in \mathcal{S}_k} x_k(G) x_k(G)^\top,$$

对最小二乘目标求导并令梯度为零，可以得到：

$$W^* = \left(\sum_{k=1}^K W_k C_k \right) \left(\sum_{k=1}^K C_k \right)^{-1}$$

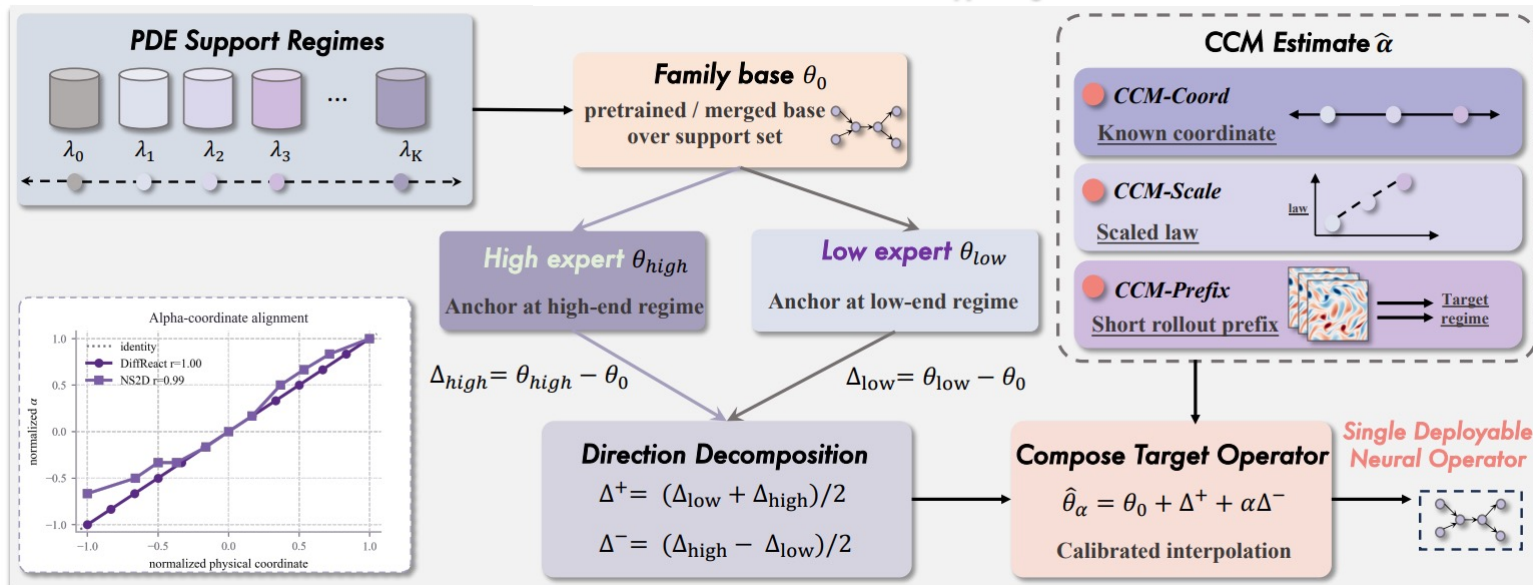
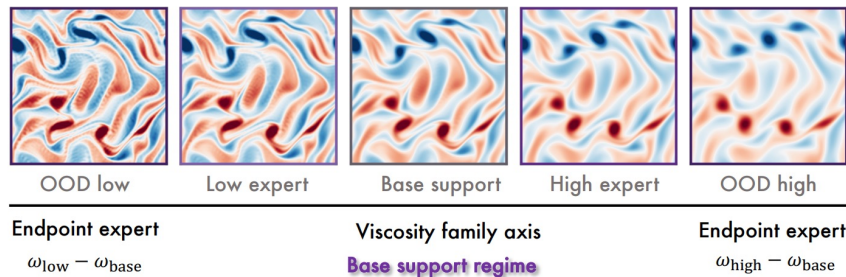
这就是GFFMerge的核心闭式合并公式。 [arXiv](#)

整体流程：： 专家激活收集→逐层闭式合并→切换嵌入→少量后层恢复训练
最终得到：： 多个输入嵌入库+一个共享消息传递主体+一个共享输出头

Applications in ai4s



• Composing Neural PDE Experts



• Composing Neural PDE Experts

3. 构造一条权重空间物理坐标线

对任意坐标 α , 定义:

$$\theta(\alpha) = \theta_0 + \Delta^+ + \alpha \Delta^-.$$

展开后:

$$\theta(\alpha) = \frac{1-\alpha}{2}\theta_{\text{low}} + \frac{1+\alpha}{2}\theta_{\text{high}}$$

因此:

$$\alpha = -1 \Rightarrow \theta(\alpha) = \theta_{\text{low}},$$

$$\alpha = 0 \Rightarrow \theta(\alpha) = \frac{\theta_{\text{low}} + \theta_{\text{high}}}{2},$$

$$\alpha = 1 \Rightarrow \theta(\alpha) = \theta_{\text{high}}.$$

而:

$$|\alpha| > 1$$

表示沿端点定义的方向进行权重外推。 

一个值得注意的数学事实是: 最终的 $\theta(\alpha)$ 可以完全写成两个端点的线性组合, θ_0 在代数上消掉了。

因此anchor的主要作用不是最终合成公式本身, 而是:

1. 确保两个专家具有共同谱系;
2. 将端点更新解释成共同适应与有向变化;
3. 降低因独立训练、置换对称性和不同损失盆地导致的不可合并性。

4. 为什么说它是“物理方向”?

假设理想专家参数随物理坐标 q 平滑变化:

$$q \mapsto \theta^*(q).$$

在中心点附近, 对端点 $q = \pm h$ 做Taylor展开:

$$\theta^*(h) = \theta^*(0) + h\theta^{*'}(0) + \frac{h^2}{2}\theta^{*''}(0) + \dots,$$

$$\theta^*(-h) = \theta^*(0) - h\theta^{*'}(0) + \frac{h^2}{2}\theta^{*''}(0) + \dots.$$

两式相加:

$$\frac{\theta^*(h) + \theta^*(-h)}{2} \approx \theta^*(0),$$

对应共同适应项。

两式相减:

$$\frac{\theta^*(h) - \theta^*(-h)}{2} \approx h\theta^{*'}(0),$$

对应物理参数方向的切向量。

所以低、高端点专家相当于对权重空间中的物理解流形做了一次**中心有限差分**。论文给出了相应误差界: 共同项误差主要是 $O(h^2)$, 方向项对局部切向量的逼近误差主要是 $O(h^3)$, 此外还包含专家训练残差。 

但这只在局部成立。如果端点相距过远、专家路径曲率较大, 线性方向就会失效。

• Composing Neural PDE Experts

1. CCM-Coord: 物理参数已知

如果知道目标物理参数 λ ，先将它映射到归一化坐标：

$$s(\lambda_{\text{low}}) = -1, \quad s(\lambda_{\text{high}}) = 1.$$

直接使用：

$$\hat{\alpha} = \text{clip}(s(\lambda), \alpha_{\min}, \alpha_{\max})$$

例如：

$$\nu_{\text{low}} = 8 \times 10^{-5}, \quad \nu_{\text{high}} = 1.2 \times 10^{-4}.$$

若目标黏度正好在中间：

$$\nu = 10^{-4} \Rightarrow \alpha = 0.$$

若目标位于高端点之外：

$$\nu > 1.2 \times 10^{-4} \Rightarrow \alpha > 1,$$

则做权重外推。 

2. CCM-Scale: 物理坐标和权重坐标尺度不一致

有时物理参数顺序是正确的，但：

$$\alpha = s(\lambda)$$

的变化速度不合适。比如黏度增加10%，理想模型参数不一定恰好沿权重线增加10%。

因此加入一个全局尺度系数：

$$\hat{\alpha} = \text{clip}(\gamma s(\lambda), \alpha_{\min}, \alpha_{\max})$$

其中 γ 在少量验证物理状态上选择一次，之后固定。 

这相当于校准：

物理参数距离 $\rightarrow$ 权重空间距离。

在二维Navier-Stokes黏度实验中，论文主要使用这一形式。

3. CCM-Prefix: 物理参数未知，但有短轨迹

如果不知道 λ ，但可以观察系统最初几步真实演化，就建立候选坐标集合：

$$\mathcal{A} = \{-1.5, -1.25, \dots, 1.25, 1.5\}.$$

对每个候选 α ：

1. 构造checkpoint $\theta(\alpha)$ ；
2. 用它预测前 K 步；
3. 与真实短轨迹比较；
4. 选择前缀误差最小的坐标。

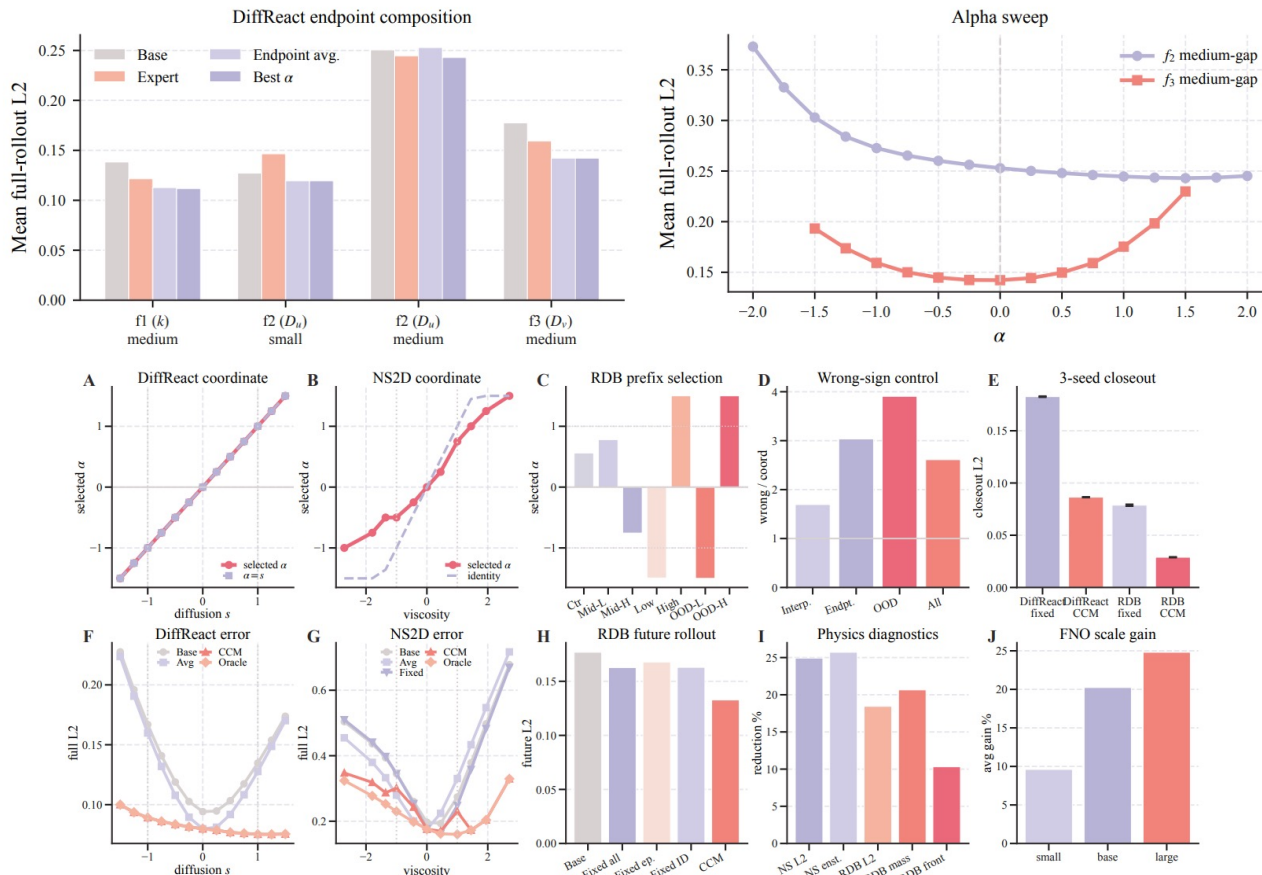
即：

$$\hat{\alpha} = \arg \min_{\alpha \in \mathcal{A}} \mathcal{L}_{\text{prefix}}(\theta(\alpha)).$$

选好后，使用该checkpoint预测剩余未来：

$$\hat{u}_{K+1:T} = \mathcal{F}_{\theta(\hat{\alpha})}(u_0).$$

论文在径向湍流实验中使用 $K = 4$ ，并明确将前4步只用于选择 α ，最终误差只在第5步之后计算。整个过程不更新参数梯度。 



- Model Merging: Foundations, Practice and Applications
- Model Merging in LLMs, MLLMs, and Beyond: Methods, Theories, Applications and Opportunities